BIBLIOTECA IA, SOFTWARE & COMPUTAÇÃO · LIVRO 4

Engenharia de Software e Ciência da Computação

Arquitetura e boas práticas, algoritmos e estruturas de dados, dados e sistemas de recomendação, engenharia reversa e segurança.

Edição de outubro de 2026

Sobre esta coleção

> **Para começar**
> Esta coleção explica inteligência artificial, desenvolvimento de software e computação do zero. Cada capítulo começa em linguagem simples, com analogias do dia a dia, e aprofunda aos poucos. Os livros podem ser lidos separadamente.

O conteúdo reorganiza o que ensinam centenas de aulas e vídeos (do CS50 de Harvard a canais como IBM Technology, Anthropic e muitos outros), artigos técnicos de referência e o blog AkitaOnRails, complementados com pesquisa atualizada. Os vídeos aparecem pelo título, com link; cada capítulo termina com a tabela de fontes. Caixas *Você sabia?* trazem curiosidades e grandes nomes da tecnologia.

Os livros da coleção

1. Livro 1 — Fundamentos de IA e Modelos de Linguagem
2. Livro 2 — Construindo com LLMs
3. Livro 3 — Programando com IA
4. Livro 4 — Engenharia de Software e Ciência da Computação
5. Livro 5 — IA, Negócios e Sociedade
6. Livro 6 — Cultura, Comunidade e História

> **Datas**
> IA muda depressa. Preços, planos, benchmarks e nomes de modelos refletem outubro de 2026. Figuras marcadas como "ilustrativas" mostram a ideia, não medições.

Sumário

1. Engenharia de Software e Arquitetura 4
2. Algoritmos e Ciência da Computação 31
3. Dados, Estatística e Sistemas de Recomendação 48
4. Engenharia Reversa e Segurança 62

CAPÍTULO 1 ·

Engenharia de Software e Arquitetura

> *Engenharia de software é um assunto enorme: linguagens, front-end, APIs, bancos de dados, padrões de projeto, arquitetura, testes, DevOps e Git. Este capítulo percorre esse caminho, do código de uma função até um sistema com milhões de usuários. Um fio condutor atravessa tudo: não existe bala de prata. Cada técnica resolve um problema e cobra um preço, e saber quando não aplicá-la vale cada vez mais agora que agentes de IA geram código em segundos.*

Para começar

Escrever código é assentar tijolos; **engenharia de software** é o que garante que a casa fique de pé, aceite reformas e aguente mais gente do que o previsto. A **arquitetura** é a planta: o que é caro mudar depois. O capítulo conversa com o Livro 3, Capítulo 1 (Programando com IA) e com o Capítulo 2 (Algoritmos).

O que é (e o que não é) arquitetura de software

Para começar

Uma cidade que cresce sem plano vira um labirinto em que qualquer obra trava o trânsito. Um software também pode "funcionar" hoje e ser um pesadelo para mudar amanhã. Arquitetura é o plano que evita isso.

Na aula Fundamentos de Arquitetura de Software para Backend, do canal Fernanda Kipper, arquitetura é a **organização fundamental de um sistema**: componentes, relações entre eles e os princípios que guiam sua evolução. Martin Fowler prefere "o entendimento compartilhado que os desenvolvedores experientes têm do sistema". O vídeo VOCÊ NÃO SABE ARQUITETURA DE SOFTWARE (veja esse vídeo), apoiado no livro *Arquitetura Limpa*, de Robert C. Martin, a define pelo que ela não é: não decide se o sistema funciona (há muitos sistemas funcionando com arquiteturas horríveis) nem se resume a pastas. Seu objetivo é tornar o sistema **fácil de entender, manter e implantar**. Um caso citado: mudar o texto de um endpoint levou duas semanas porque ninguém achava a classe responsável.

Há decisões **estruturais** (monólito ou microsserviços, eventos, serverless, nuvem, banco) e decisões de **design de código** (Clean Architecture, Hexagonal, MVC, padrões, interfaces). A **Lei de Conway** (1967) lembra que sistemas copiam a estrutura de comunicação da organização; como times mudam, é mais estável dividir o sistema por **responsabilidades de negócio** (autenticação, checkout, estoque).

A mesma aula organiza o vocabulário em seis conceitos, que voltam ao longo do capítulo:

- **Stateful × stateless** (com ou sem memória do cliente). *Pergunta:* o servidor precisa lembrar de mim? *Exemplo:* um caixa de padaria que não lembra de você; a cada compra você mostra a comanda de novo.
- **Síncrono × assíncrono**. *Pergunta:* preciso da resposta agora? *Exemplo:* no balcão, o café sai na hora; a revelação de fotos avisa quando ficar pronta.
- **Acoplamento** (o quanto uma parte depende da outra). *Pergunta:* essa dependência faz sentido? *Exemplo:* a escola exigir o pagamento para matricular é normal; mexer no caixa do financeiro, não.
- **Idempotência** (repetir não muda o resultado). *Pergunta:* e se o pedido chegar duas vezes? *Exemplo:* apertar três vezes o botão do elevador chama um elevador só.
- **Cache** (cópia guardada perto para ganhar tempo). *Pergunta:* posso usar uma informação um pouco velha? *Exemplo:* o cardápio impresso pode ser de ontem; o saldo da conta, não.
- **Restrições**. *Pergunta:* quanto dinheiro, tempo e gente eu tenho? *Exemplo:* uma padaria de bairro não precisa de um sistema de rede de supermercados.

Linguagens e fundamentos

> **Para começar**
> O computador só entende código de máquina, então toda linguagem precisa de um tradutor: alguns traduzem tudo antes, outros na hora, outros um pouco de cada. Esta seção mostra esses modelos, o que são **tipos** e os erros clássicos de Java, JavaScript e Python.

Modelos de execução

- **Bytecode + máquina virtual** (Java, Kotlin, C#): o `javac` gera bytecode portável, que a JVM interpreta e compila em tempo de execução (JIT). A JVM também cuida da coleta de lixo.
- **Compilação nativa** (Swift, Rust, C/C++, em geral via LLVM): código de máquina para uma arquitetura específica, rápido, mas não portável.
- **Transpilação** (TypeScript → JavaScript): o `tsc` verifica tipos e emite JS; os tipos desaparecem em tempo de execução.

Tipagem: dois eixos independentes

Estática × dinâmica diz *quando* o tipo é verificado (compilação em Java, TypeScript, Go; execução em Python, JavaScript). **Forte × fraca** diz quanto a linguagem aceita conversões implícitas. Python é dinâmica *e forte* (`"1" + 1` dá erro); JavaScript é dinâmica e fraca (`"1" + 1 === "11"`). Dizer que Python é “fracamente tipado” é um erro comum.

A aula CURSO DE TYPESCRIPT para INICIANTES | Aprenda Typescript na Prática mostra o ganho prático: o editor pega `res.jon(...)` antes de virar erro 500 em produção. Recomendações: `"strict": true`, evitar `any` e usar *union types* de literais. Um contraponto

vem do post Existem linguagens melhores pra programar com LLMs?, do blog AkitaOnRails: num experimento com sete linguagens e quatro modelos de IA, a tipagem forte não reduziu os reparos, porque os erros eram de regra de negócio. O que mais ajudou foi uma suíte de testes. Conclusão do autor: "disciplina vence compilador e vence linguagem".

Java: armadilhas e recursos modernos

Os vídeos Curso de PROGRAMAÇÃO JAVA para INICIANTES | Fundamentos + Programação Orientada Objetos, CURSO de JAVA INTERMEDIÁRIO | Estruturas de dados, Stream API, Generics, Packages, Records e 5 RECURSOS DO JAVA QUE VOCÊ PRECISA CONHECER formam um minicurso. As armadilhas mais comuns:

- **`== × .equals`**: com objetos, `==` compara referências. Duas `new String("Ana")` são diferentes por `==` e iguais por `equals`. Ao sobrescrever `equals`, sobrescreva `hashCode`, senão `HashMap` quebra.
- Casting explícito trunca: `(int) 9.99 == 9`. Arrays têm tamanho fixo; `ArrayList` é dinâmica.
- Coleções: `List` (ordenada), `Set` (sem duplicatas), `Map` (chave→valor; `get` de chave ausente devolve `null`).

O Java moderno trouxe **records** (estáveis no Java 16), *text blocks* (Java 15), switch com seta, lambdas e a **Stream API**:

```
record Livro(String titulo, int paginas) {}   // equals/hashCode/toString prontos
var r = List.of("Fernanda", "Ana").stream()
            .filter(n -> n.startsWith("F"))
            .map(String::toUpperCase)
            .toList();                          // [FERNANDA]
```

Memória: stack, heap e garbage collector

A **stack** guarda os frames de chamada (variáveis locais e referências) e é liberada quando o método retorna; recursão sem fim termina em `StackOverflowError`. O **heap** guarda os objetos, e o **garbage collector** libera os que ninguém mais alcança. O vídeo Entendendo COLETOR DE LIXO DO JAVA (memória Heap, Stack, Garbage Collector) compara os coletores da JVM: Serial (simples, pausas longas), Parallel (throughput), **G1** (padrão desde o Java 9, meta de pausa curta) e ZGC/Shenandoah (pausas abaixo de 1 ms, mais CPU). A escolha é sempre throughput × latência; faça profiling antes de trocar.

JavaScript e Python: erros clássicos

Os vídeos JAMAIS cometa esses ERROS em JavaScript e ERROS COMUNS | Null Pointer e None mostram que muitos bugs vêm de três fontes: **mutação inesperada**, **valores ausentes** e **assincronia**.

Erro	Sintoma	Correção
`arr.sort()` muda o original (JS)	Dados “mudam sozinhos”	`arr.toSorted()` ou `[...arr].sort()`
`x = lista.sort()` (Python)	`x` vira `None`	`sorted(lista)`
Acessar propriedade de `undefined`/`None`	`TypeError`/`AttributeError`	`?.` e `??`; `Optional` + mypy; `dict.get`
Usar `fetch` sem esperar	Recebe uma `Promise` pendente	`async/await`
Variável de ambiente ausente	Falha em produção horas depois	*Fail fast* no boot

Em JS e Python, parâmetros são passados por *valor da referência*: mutar o objeto recebido afeta quem chamou. E dados que vêm de fora devem ser validados em tempo de execução (Zod, Pydantic): tipos estáticos não protegem contra um JSON malformado.

Ferramentas: IDE, CLI e o problema do contexto

Os vídeos What Is an AI IDE? How AI Is Changing Developer & Coding Tools e IDE vs CLI: What Every DevOps Engineer Should Know, do canal IBM Technology, observam que o atrito não está em escolher IDE ou terminal, e sim nas **passagens** entre editor, testes, build e logs, em que o humano vira a “cola” do contexto. As ferramentas de IA úteis executam várias etapas e validam o resultado sozinhas.

Front-end

> **Para começar**
>
> O **front-end** é a vitrine: tudo o que o usuário vê no navegador. Ela é feita de **HTML** (estrutura), **CSS** (aparência) e **JavaScript** (comportamento). Veremos como uma página chega à tela, CSS, React e um pouco de design de interface.

Como a web funciona

A aula CS50x - Lecture 8 - HTML, CSS, JavaScript, de Harvard, resume a base. Roteadores encaminham **pacotes** IP (IPv4 tem 32 bits, ~4,3 bilhões de endereços; IPv6, 128 bits). O **TCP** acrescenta portas (80, 443) e retransmissão. O **DNS** traduz nomes em IPs. O HTTP define verbos (GET, POST, PUT, PATCH, DELETE) e códigos de status:

Código	Significado
200 / 201	OK / Created
301 / 303	Redirecionamento permanente / Post-Redirect-Get
400 / 401 / 403 / 404	Payload inválido, sem autenticação, sem permissão, inexistente
413 / 429	Payload grande demais / Too Many Requests (rate limiting)
500 / 502 / 503 / 504	Bug, upstream fora, sobrecarga, timeout no proxy

O navegador transforma o HTML numa árvore, o **DOM**, que o JavaScript manipula em resposta a eventos. Lição de segurança da aula: **validação no cliente é conveniência, não proteção**. Quem valida de verdade é o servidor.

> **Você sabia?**
> O código HTTP 418 “I'm a teapot” nasceu de uma piada de 1º de abril, a RFC 2324, de 1998, que descrevia um protocolo para controlar cafeteiras. Ele nunca foi usado a sério, mas muitos frameworks o implementam até hoje.

CSS: Flexbox, Grid e responsividade

Flexbox é unidimensional: `justify-content` alinha no eixo principal e `align-items` no secundário; trocar `row` por `column` inverte os eixos, o erro mais comum de iniciante. **Grid** é bidimensional: a unidade `fr` divide o espaço livre, `grid-template-areas` nomeia regiões e `repeat(auto-fill, minmax(200px, 1fr))` cria colunas responsivas sem media queries.

Responsividade importa porque a maior parte do acesso à internet no Brasil é pelo celular. O TUTORIAL COMPLETO CSS MEDIA QUERY - Criando layouts responsivos na PRÁTICA recomenda **mobile first** (estilos base para o celular, `min-width` para telas maiores), poucos breakpoints padronizados (768 e 1024 px) e evitar larguras fixas.

```
.cards { display: grid; grid-template-columns: 1fr; gap: 1rem; }   /* mobile first */
@media (min-width: 768px)  { .cards { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 1024px) { .cards { grid-template-columns: repeat(4, 1fr); } }
```

Entre os frameworks, o **Bootstrap** oferece componentes prontos (`btn btn-primary`, grid de 12 colunas) e o **Tailwind** é *utility-first*, com uma classe por propriedade (`mt-4`, `md:flex`) e um build que mantém só as classes usadas.

React: hooks e dados remotos

O **React** monta a tela com **componentes** e redesenha só o que muda. Os hooks principais:

- `useState`: só o setter avisa o React. Quando o novo valor depende do anterior, use `setCount(prev => prev + 1)`.
- `useEffect(fn, deps)`: roda quando as dependências mudam; o `return` é o *cleanup*. Em desenvolvimento, o `StrictMode` monta duas vezes de propósito, o que explica logs duplicados.
- `useContext` evita passar props por vários níveis; hooks customizados (`useAlgo`) encapsulam lógica reutilizável.

Buscar dados com `useEffect + fetch` obriga a reimplementar loading, erro, retry e cache em cada tela. O **TanStack Query** resolve com `useQuery` e `useMutation`, e `invalidateQueries` recarrega a lista após uma mutação. **Micro front-ends** (vídeo Microfrontends Explicado Simples: A Arquitetura que Está Mudando o Frontend) dão deploy independente a cada time, ao preço de complexidade; como microsserviços, só valem para times grandes.

O tutorial Criando APLICAÇÃO COMPLETA e ESCALÁVEL com Angular + Edge Computing mostra o problema clássico das SPAs: sem JavaScript nada aparece, o que prejudica o SEO. A solução é **Static Site Generation**: cada página vira HTML estático, servido a partir da borda de uma CDN.

UI/UX para quem gera interface com IA

O vídeo 5 BOAS PRÁTICAS de UI/UX para melhorar seus PROMPTS (frontend) mostra que princípios de design melhoram muito a UI gerada por IA: **hierarquia visual** (dois CTAs com o mesmo peso dividem a atenção), **proximidade** (elementos próximos são lidos como grupo), **affordance** (IAs esquecem `cursor: pointer` e `:hover`) e componentes como **máquinas de estado** (vazio → loading → sucesso | erro, sem estados impossíveis).

Back-end e APIs

> **Para começar**
> Se o front-end é o salão, o **back-end** é a cozinha, e a **API** é o garçom: o cardápio com regras pelo qual um programa faz pedidos a outro. Veremos estilos de API, validação, cobranças em dobro, tempo real e a infraestrutura na frente do servidor.

Estilos de API

Para o vídeo What Is an API? Types, Uses, & AI Integration, na era da IA a API é o "canudo" que leva dados até o modelo e permite que ele aja. Três estilos dominam:

- **REST**: cada coisa tem um endereço, e os verbos HTTP dizem o que fazer. Simples e fácil de pôr em cache. Bom para APIs públicas. *Como pedir um prato pelo número do cardápio.*
- **GraphQL**: um endereço só, e o cliente diz quais campos quer, nem mais nem menos. Cache mais difícil. Bom para telas com necessidades variadas. *Como montar o próprio prato no self-service.*
- **gRPC**: chama uma função de outro servidor como se fosse local, com mensagens binárias pequenas (Protocol Buffers). Rápido, porém difícil de depurar. Bom entre serviços que você controla. *Como o interfone direto entre a cozinha e o caixa.*

O vídeo O que fazer quando RESTful não for suficiente faz uma observação que vale para o capítulo todo: quem programa com IA precisa saber *quando pedir* gRPC, filas ou CQRS, porque a IA dificilmente vai sugerir sozinha.

DTOs e validação

Um **DTO** (*Data Transfer Object*) é um contrato simples que valida e transporta dados (vídeo DTOs, o que é e porque usar). Use DTOs separados para entrada (`CreateUser` tem `password`) e saída (`UserResponse` **nunca** tem). Passar o corpo da requisição direto para o ORM permite **mass assignment**: o atacante envia `"is_admin": true`. Validar e-mail mostra o limite das regex (Como Validar Email do Jeito Certo!): uma regex completa da RFC 5322 é gigante e arrisca **ReDoS**. A receita é regex simples, biblioteca no back e, sempre, um **e-mail de confirmação**.

Idempotência e double submit

Uma operação é **idempotente** quando repeti-la tem o mesmo efeito de executá-la uma vez. Pela semântica HTTP, GET, PUT e DELETE devem ser; POST não. Mas os vídeos Todo DEV precisa entender isso: IDEMPOTÊNCIA e Como sistemas evitam requisições duplicadas alertam: não presuma pelo verbo. Clique duplo, retry após timeout e rede instável causam **double submit**, e as defesas se empilham:

Camada	Técnica	Limite
Tela	Desabilitar o botão após o clique	Quem quiser consegue burlar
HTTP	Post/Redirect/Get (303)	Só evita o reenvio ao apertar F5
Servidor	Chave de idempotência (um "número do pedido") guardada num lugar **compartilhado**, como Redis ou banco	É preciso definir o que é "repetido" e por quanto tempo
Banco	Restrição `UNIQUE`	Exige um campo realmente único, como o CPF

Para cadastros, redirect + unique bastam. Para pagamentos e estoque, a chave de idempotência é obrigatória, e nunca na memória local de um servidor só.

Tempo real: polling, SSE, WebSocket e webhooks

Na web comum o cliente pergunta e o servidor responde. Para o servidor avisar novidades, há opções (vídeos SERVER-SENT EVENTS: Aprenda a Construir Sistemas que se Comunicam em Tempo Real, O que é WebSocket? Entenda na prática em 20 minutos (Guia para programadores) e WEBHOOKS vs WEBSOCKETS (na prática)):

Técnica	Direção	Uso típico
Polling / long polling	cliente → servidor	Perguntar "já chegou?" de tempos em tempos; serve para poucos usuários
SSE	servidor → cliente	Uma resposta que não fecha; o texto de um chatbot aparecendo aos poucos
WebSocket	bidirecional	Canal aberto nos dois sentidos, como numa ligação; chats e jogos
Webhook	servidor → servidor	O outro sistema avisa você, como o banco que manda SMS quando o Pix cai

Em webhooks, confira a assinatura da mensagem (HMAC, um selo que prova quem enviou) e trate avisos repetidos sem efeito duplicado. Dois erros típicos com SSE: chamar `res.end()` após cada mensagem (vira polling disfarçado) e abrir uma conexão Redis por usuário quando um único assinante Pub/Sub basta.

```
app.get('/stream', (req, res) => {
  res.setHeader('Content-Type', 'text/event-stream');
  const onMsg = (msg) => res.write(`data: ${msg}\n\n`);
  subscriber.on('message', onMsg);                  // um único cliente Redis
  req.on('close', () => subscriber.off('message', onMsg));
});
```

CORS

CORS diz se uma página de um site pode *ler* respostas de outro (vídeo O que é CORS? Análise frontend e backend). Ele existe só no navegador; curl e serviços entre si não têm

"erro de CORS". Requisições não simples disparam um **preflight** `OPTIONS`, e o `*` não vale com cookies. CORS não substitui autenticação nem proteção contra CSRF (cookies `SameSite`, tokens).

Proxy reverso, CDN e rate limiting

Um **proxy reverso** é a recepção do prédio: fica na frente dos servidores e cuida de roteamento, TLS, cache e balanceamento. O vídeo Apache vs NGINX contrasta o Apache (processo ou thread por conexão no modelo clássico) com o Nginx, criado em 2004 para o "problema C10K", com poucos workers orientados a eventos. Em APRENDA NGINX DO ZERO | Curso de NGINX em 1 HORA, a pegadinha clássica é a barra final do `proxy_pass`, que decide se o prefixo é repassado:

```
upstream frontend { server localhost:3000; server localhost:3001; }  # round-robin
server {
  listen 80;
  location /     { proxy_pass http://frontend; }
  location /api/ { proxy_pass http://localhost:8000/; }  # barra final remove /api
}
```

Uma **CDN** é uma rede de depósitos perto do usuário: servidores de **borda** guardam cópias de arquivos estáticos e buscam na origem só no *miss* (vídeo O que é uma CDN? Para que serve a CDN?).

Rate limiting limita requisições por cliente num intervalo. No vídeo Rate Limiter | Explicação Completa, o autor conta um ataque cujo único objetivo era gerar custo num sistema serverless. Os algoritmos clássicos são **fixed window** (simples, mas permite rajada dupla na virada), **sliding window** (preciso, usa mais memória), **token bucket** (permite rajadas controladas) e **leaky bucket** (saída constante). A resposta padrão é **429** com `Retry-After`, e a mesma ideia serve para **orçamentos de tokens** por usuário em apps com LLM.

Estado global, connection pooling e event loop

Estado global no back-end gera bugs difíceis de reproduzir: o vídeo Como Uma Requisição Contamina Outra mostra uma variável `current_user` compartilhada entre requisições paralelas. Prefira funções puras, contexto por requisição e injeção de dependência.

Abrir uma conexão com o banco é caro (TCP, TLS, autenticação). Com **connection pooling** (Como 1.000 Requests Usam 20 Conexões | Connection Pooling), mil requisições compartilham, por exemplo, 20 conexões. O pool é criado fora do handler, e o `release` vai num `finally`. Em serverless, use um pool externo (PgBouncer, RDS Proxy).

```
const pool = new Pool({ max: 20 });          // uma vez, no módulo
app.get('/users/:id', async (req, res) => {
  const client = await pool.connect();
  try { res.json((await client.query('SELECT * FROM users WHERE id=$1', [req.pa-
rams.id])).rows[0]); }
  finally { client.release(); }
});
```

Até `console.log` pesa: no Node, escrever em arquivo ou terminal é síncrono e atrasa o *event loop*. Nos testes do vídeo Console log pode TRAVAR a sua aplicação (explicação), cem logs por requisição derrubaram a vazão de um servidor de ~1,3 milhão para ~71 mil requisições no mesmo intervalo. Use logs assíncronos e estruturados (Pino).

Por fim, cada dependência externa traz riscos de cadeia de suprimentos (o caso *xz*, de 2024), atualizações quebradas e imagens maiores. A regra de O custo de usar muitas dependências: não instale uma biblioteca para uma única função; fixe versões com lockfile e audite no CI.

Bancos de dados

> **Para começar**
> Um **banco de dados** é o arquivo-mestre da empresa: guarda as informações de forma organizada e segura. Há bancos **relacionais**, como planilhas com colunas fixas consultadas em SQL, e **NoSQL**, como fichas livres. Veremos garantias, distribuição, busca, evolução do esquema, cache e sharding.

Relacional × documento, ACID × BASE

O MySQL organiza dados em tabelas com esquema; o MongoDB guarda documentos JSON flexíveis (MySQL vs MongoDB). Os vídeos ACID e BASE explicado de forma simples e Como Escolher o Banco de Dados Correto pra sua Aplicação | System Design & Arquitetura de Software mostram que a escolha começa pelas garantias. **ACID**: **atomicidade** (a transferência debita e credita, ou nada), **consistência** (o estoque nunca fica negativo), **isolamento** (duas compras do último item: uma falha) e **durabilidade** (o confirmado sobrevive a um crash). **BASE** troca correção imediata por disponibilidade e escala, com **consistência eventual**. Pagamentos, estoque e ingressos pedem consistência forte; likes, analytics e recomendações aceitam a eventual. Na prática as fronteiras se misturam: o MongoDB tem transações ACID e o DynamoDB oferece leitura consistente sob demanda.

CAP e PACELC

Quando um banco roda em várias máquinas e a rede entre elas cai, cada lado precisa decidir: continua atendendo com dado talvez velho ou para até a linha voltar? O teorema **CAP** (Brewer, 2000) diz que, durante uma **partição**, é preciso escolher entre **consistência** e **disponibilidade**. Um banco prefere ficar indisponível a mostrar saldo errado (CP); uma rede social prefere mostrar curtidas atrasadas (AP). O **PACELC** (Abadi, 2012) completa: *sem* partição, a escolha diária é entre **latência** e consistência (replicação síncrona é consistente e lenta; assíncrona, rápida e eventual).

Banco	Classificação usual	Mecanismo
Cassandra / ScyllaDB	PA/EL (ajustável)	Cada consulta escolhe quantas réplicas confirmam (ONE, QUORUM, ALL)
MongoDB	PC/EC (padrão)	Replica set com eleição de primário
DynamoDB	Por requisição	`ConsistentRead` true/false
CockroachDB, Spanner	PC/EC	As réplicas votam antes de confirmar (Raft/Paxos)

Com fator de replicação RF, se os nós lidos (R) e escritos (W) somam mais que RF, toda leitura cruza com uma réplica atualizada. Dois erros comuns: confundir o "C" de ACID (invariantes) com o do CAP (todas as réplicas devolvem a última escrita), e chamar o Postgres de "CA". Um nó só não sofre partição; com réplicas, a escolha volta a existir.

Índices e Full-Text Search

Um **índice** é como o índice remissivo de um livro. O vídeo O QUE NINGUÉM TE ENSINOU SOBRE BUSCAS INTELIGENTES NO BANCO DE DADOS! mostra que `LIKE '%termo%'` busca substrings ("anel" acha "panela") e, com o curinga no início, não usa índice B-tree: lê a tabela inteira. O **Full-Text Search** tokeniza, remove *stopwords*, reduz palavras ao radical (*stemming*) e monta um **índice invertido**.

```
-- PostgreSQL
CREATE INDEX idx_fts ON products USING GIN (to_tsvector('portuguese', name || ' ' || description));
SELECT * FROM products
 WHERE to_tsvector('portuguese', name || ' ' || description) @@ plainto_tsquery('portuguese', 'capa dura');
```

A lição dos números do vídeo: **sem o índice**, o FTS foi *mais lento* que o LIKE (139 ms contra 4,9 ms); com o índice GIN, caiu para 0,3 ms. Para relevância de grande e-commerce, use um motor dedicado (Elasticsearch/OpenSearch); para substrings tolerantes a erro, a extensão `pg_trgm`.

ORMs, SQL injection e migrações

Uma **ORM** traduz objetos em SQL (`user.save()`). O vídeo Como ORMs Mentem pra Você lista o que ela esconde. O principal é o **N+1**: um loop que acessa `order.customer.name` dispara uma consulta por pedido. A solução é *eager loading* (`select_related` no Django, `include` no Prisma, `JOIN FETCH` no JPA). Inspecione o SQL gerado com `EXPLAIN ANALYZE` e use SQL puro nas consultas críticas. Contra **SQL injection**, a defesa são **consultas parametrizadas** (`WHERE id = $1`), não "sanitizar" strings (O que é SQL injection e como resolver).

Rodar `ALTER TABLE` à mão em produção não é revisável nem reproduzível (Como Fazer Migrações de Banco de Dados do Jeito Certo). **Migrações** ficam no repositório, passam por PR e o banco registra qual versão aplicou. Para mudar sem downtime, use **expand/contract**: adicione a coluna nova, escreva nas duas, faça o backfill em lotes, mude as leituras e só depois remova a antiga. Já o **database branching** (Pare de quebrar o banco em

produção: Aprenda Database Branching) dá a cada branch de código uma cópia *copy-on-write* do banco de staging, que só duplica os blocos alterados.

Cache

Cache é deixar o sal ao lado do fogão em vez de buscá-lo na despensa. Funciona por causa das ordens de grandeza de latência citadas em CACHE-ASIDE: Escalabilidade, Performance e Arquitetura de Software pra Mandar Bem na Entrevista!: RAM ~100 ns, SSD ~100 µs, disco rígido ~10 ms. Mesmo pela rede, uma leitura no Redis costuma ser bem mais rápida que uma consulta com joins.

No **cache-aside**, a aplicação lê o cache; no *miss*, lê o banco e grava no cache com **TTL**. Alternativas: **write-through** (grava nos dois, cache sempre atual, escrita mais lenta) e **write-behind** (grava no cache e persiste depois, com risco de perda). Armadilhas: o **cache stampede** (milhares de misses quando uma chave quente expira; use lock ou TTL com *jitter*) e cache local em várias instâncias, cada uma com uma versão diferente.

O caso da Timeline do Facebook (ARQUITETURA BACKEND DO FACEBOOK | System Design) mostra que cache sem estratégia não basta: a solução combinou desnormalização, separação entre dados quentes e frios e um agregador rodando na própria máquina do banco, levando a computação até os dados.

Sharding

Réplicas de leitura escalam leituras, mas não escritas nem volume. Quando o primário não aguenta, vem o **sharding**: os dados são fatiados entre bancos independentes. O vídeo Aprenda a Escalar Bancos de Dados Usando SHARDING | System Design & Arquitetura de Software propõe uma ordem: escala vertical, índices, cache, otimização de queries, réplicas e, por último, sharding.

- A **shard key** precisa de alta cardinalidade, distribuição uniforme e afinidade (usuário numa rede social). `created_at` por ano concentra a carga no shard atual (*hotspot*).
- `hash(key) % N` remapeia quase tudo quando N muda; o **consistent hashing** move só ~1/N das chaves.
- Desafios: o "problema da celebridade", consultas entre shards e transações distribuídas, que viram **Sagas** com compensações.

Design de código: orientação a objetos bem feita

> **Para começar**
> Orientação a objetos organiza o programa em peças que juntam dados e comportamento, como blocos de LEGO. Peças bem desenhadas podem ser trocadas sem desmontar o castelo. Esta seção mostra os sintomas de um código apodrecendo e três ferramentas: **SOLID**, **Object Calisthenics** e **padrões de projeto**.

Sintomas de código apodrecendo

Robert C. Martin descreveu quatro sintomas, retomados em 4 Sintomas de que Seu Código Está APODRECENDO e Você Precisa Identificar Agora!: **rigidez** (toda mudança gera efeito dominó), **fragilidade** (mexer no frete cancela pedidos), **imobilidade** (impossível reaproveitar um módulo) e **viscosidade** (a gambiarra é mais fácil que o caminho certo). A saída começa por cobrir as áreas sensíveis com testes e refatorar; depois, extrair partes aos poucos atrás de uma fachada, o padrão **Strangler Fig**.

SOLID

Princípio	Enunciado	Violação típica
SRP	Um módulo tem **um único motivo para mudar**	Classe que matricula, cobra e manda e-mail
OCP	Aberto para extensão, fechado para modificação	Cadeia de `if (tipo == "ICMS")`
LSP	Um subtipo substitui o tipo base sem quebrar quem o usa	Um "pássaro" pinguim que lança erro ao voar
ISP	Nenhum cliente depende de métodos que não usa	Pagamento com cartão obrigado a ter `generateQRCode`, que só o Pix usa
DIP	Regras dependem de abstrações, não de detalhes	`new StripeClient()` dentro da regra de negócio

As visões divergem. Renato Augusto (por exemplo, em O Princípio SOLID mais CRÍTICO de TODOS! | Princípio de Substituição de LISKOV) considera LSP e ISP os mais negligenciados. Augusto Galego, em Princípios SOLID Pelos Olhos de um Dev Sr., dá nota máxima a Liskov e é cético com ISP e DIP, que aumentam o boilerplate. Os dois concordam que os princípios **não devem ser seguidos à risca em software pequeno**. Na aula Dominando príncipios de design de código | SOLID, o ganho é concreto: com um `Notifier` injetado, uma queda do e-mail deixa de bloquear matrículas, e trocar e-mail por WhatsApp não mexe na regra.

> **Você sabia?**
> O "L" do SOLID homenageia Barbara Liskov, que formulou o princípio de substituição numa palestra de 1987. Ela também criou a linguagem CLU, pioneira em tipos abstratos de dados, e recebeu o Prêmio Turing de 2008.

Object Calisthenics

As nove regras de Jeff Bay (*The ThoughtWorks Anthology*, 2008) são **exercícios**, não leis, comentados em Separando Dev Júnior de Dev Senior (Object Calisthenics). As mais úteis: um nível de indentação por método; não usar `else` (*guard clauses*, retornar cedo); encapsular primitivos em **Value Objects** que se autovalidam (Email, CPF); um ponto por linha (**Lei de Demeter**: peça `order.getCustomerZipCode()` em vez de navegar a cadeia); e evitar getters/setters (**Tell, Don't Ask**). Regras como "no máximo duas variáveis de instância" são consideradas exageradas.

```
// Anêmico: a regra mora num "Service"
if (product.getPrice() > 50) product.setPrice(product.getPrice() - 10);
// Rico: a entidade protege a própria regra
product.applyDiscount(Money.of("10.00"));
```

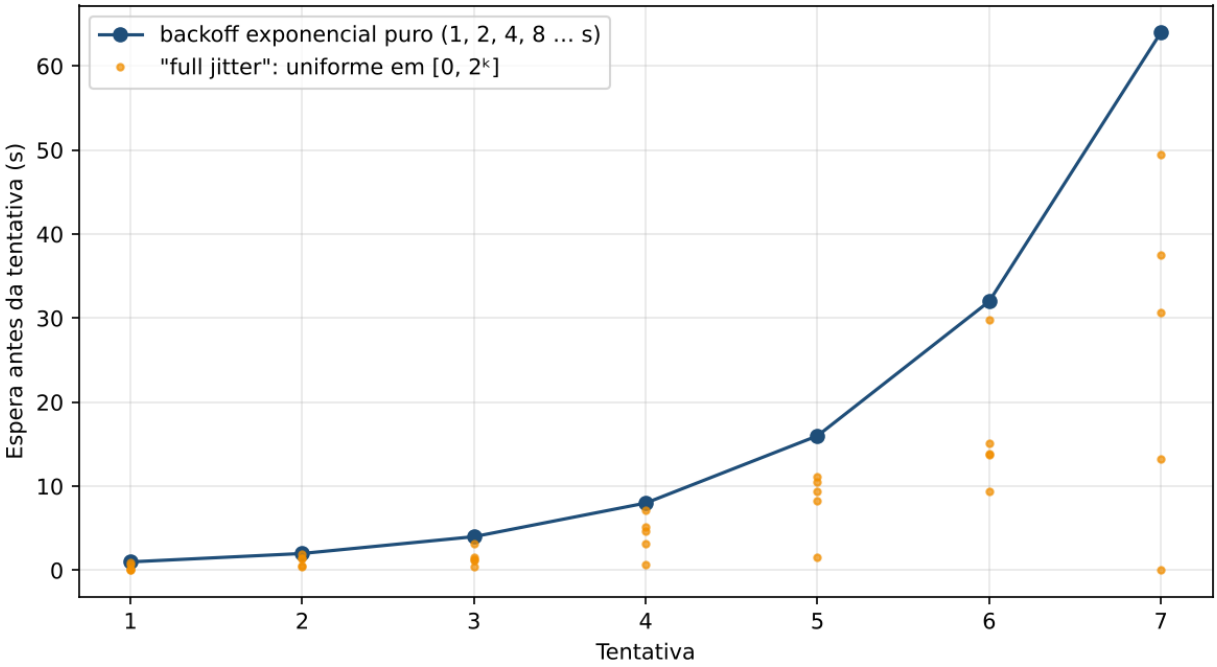
Nunca use `float` para dinheiro: 0,1 + 0,2 ≠ 0,3 em ponto flutuante binário. Use `BigDecimal`, centavos inteiros ou um tipo Money.

Padrões de projeto

Um **padrão de projeto** é uma solução com nome para um problema recorrente, como uma jogada ensaiada. O catálogo do GoF (1994) os divide em criacionais, estruturais e comportamentais. Regra de ouro: aplique quando o problema aparecer, nunca preventivamente.

Padrão	Problema que resolve	Cuidado
Factory	Centralizar a criação; o cliente só conhece a interface	Métodos estáticos atrapalham testes
Singleton	Uma instância por processo (pool de conexões)	É estado global
Adapter	Traduzir uma biblioteca externa para a sua interface, como um adaptador de tomada	Mais uma camada para manter
Strategy	Trocar a regra sem if/switch, como escolher o tipo de desconto	Alguém ainda precisa escolher a regra (em geral, uma Factory)
State	Estados de um pedido como objetos com transições válidas	Mais classes; alternativa: tabela de transições
Observer	O emissor não conhece quem escuta	Fluxo implícito, difícil de rastrear

Em 5 Design Patterns Que Todo Dev Deveria Conhecer, Augusto Galego acrescenta dois padrões de **resiliência** indispensáveis ao chamar APIs instáveis (pagamentos, LLMs). O **retry com backoff exponencial e jitter** espera 1, 2, 4 s... e sorteia o momento dentro de cada intervalo para que milhares de clientes não retentem juntos (veja a figura a seguir); só retente erros transitórios e operações idempotentes. O **circuit breaker**, após N falhas, “abre” e faz as chamadas falharem na hora, deixando passar uma chamada de teste de tempos em tempos.



Backoff exponencial puro × backoff com *full jitter* (cinco clientes simulados): o jitter desfaz a sincronização das retentativas. Fonte: elaboração própria com dados do vídeo “5 Design Patterns Que Todo Dev Deveria Conhecer”.

Acoplamento: um espectro, não um pecado

Para Augusto Galego, em Quando Vale a Pena Desacoplar Seu Código?, o método “receita de bolo” que salva, cobra, reserva estoque e envia e-mail em sequência é ótimo para um MVP. Cada nível de desacoplamento (interface + injeção → fila → pub/sub) compra resiliência e paga em indireção. Três frases dele: “abstração é útil quando abstrai alguma coisa” (`UserService` + `UserServiceImpl` sem segunda implementação é cerimônia); “fila não dá velocidade, dá **vazão**”; e IAs tendem a desacoplar ao máximo até software simples, então dê pushback.

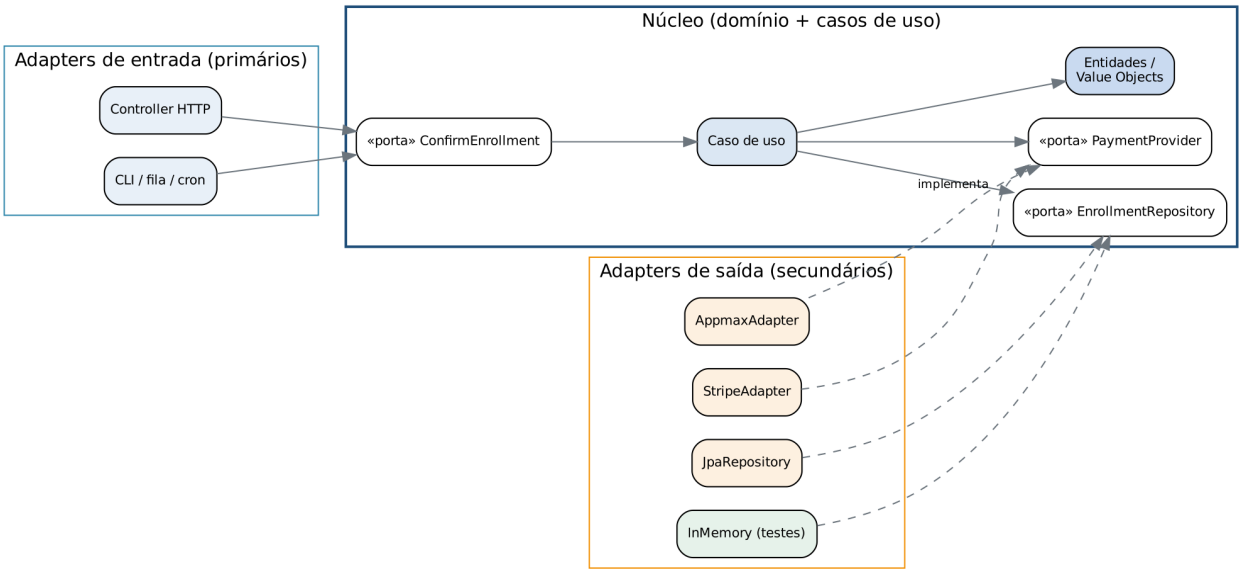
Arquitetura de aplicação

> **Para começar**
> Agora olhamos a montagem inteira. O coração do sistema (as regras de negócio) não deveria depender da “roupa” que ele veste (framework, banco, provedor de pagamento). Veremos as arquiteturas que protegem esse núcleo, a escolha entre monólito e microsserviços e dois padrões avançados: CQRS e Event Sourcing.

Clean Architecture e Hexagonal

As duas fazem as dependências apontarem para dentro. A **Clean Architecture** (Robert C. Martin) organiza o código em anéis: entidades, casos de uso, adaptadores e frameworks. A **Hexagonal** (*Ports and Adapters*, Alistair Cockburn, 2005) tem um núcleo que define **portas** (interfaces de entrada e de saída) e **adaptadores** que as implementam, como mostra a figura a seguir. Os vídeos Me de 30 minutos para aprender Arquite-

tura Limpa e Hexagonal e Arquitetura Hexagonal | Explicação de um Dev Sr. mostram as duas na prática: no fundo, é DIP + Adapter aplicados à aplicação inteira.



Ports and Adapters: o núcleo define as portas, e os adaptadores de entrada e saída (inclusive um InMemory para testes) dependem dele, nunca o contrário. Fonte: elaboração própria.

Ganha-se testabilidade (casos de uso rodam com adaptadores em memória) e liberdade para trocar fornecedores sem tocar no núcleo. Paga-se em arquivos e indireção. Não é organização de pastas: o que importa é o núcleo depender só de interfaces que ele mesmo define.

Monólito, monólito modular e microsserviços

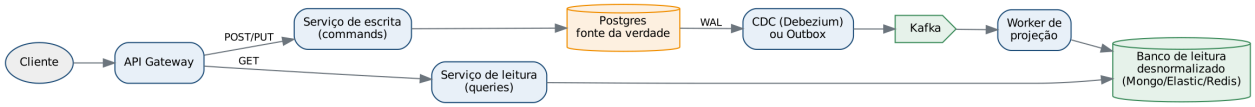
Um **monólito** é uma unidade de deploy. Um **monólito modular** mantém o deploy único, mas cria fronteiras internas com contratos mínimos: no exemplo de Aprenda MONOLITO MODULAR ANTES de MICROSERVIÇOS, matrículas só chama `cobrancas.isConfirmed(pedido)`, e cobranças pode trocar de provedor sem afetar ninguém. **Microsserviços** separam também deploy, processo e banco. A pergunta-chave, citada em Quando usar Monolitos vs Microserviços, é: **“que benefício justifica trocar uma chamada de função por uma chamada de rede?”**. O custo vem em autenticação entre serviços, tracing, latência, consistência entre bancos e infraestrutura.

Dor no monólito	Solução sem microsserviço	Quando o microsserviço se justifica
Carga alta	Load balancer + réplicas	Perfis de carga muito diferentes
Conflitos entre muitos devs	Módulos com donos	Vários times realmente autônomos
Uma parte precisa de GPU	Extrair só essa parte	Serviço de IA separado

A Shopify segue como monólito modular famoso, e a Amazon Prime Video relatou em 2023 que voltar um pipeline de monitoramento para um processo único cortou o custo em 90%. Dividir mal produz um **monólito distribuído**, o pior dos dois mundos. O consenso: comece pelo monólito modular e extraia serviços quando a dor for real.

CQRS e Event Sourcing

Numa biblioteca, o balcão de devoluções (escrita) e o catálogo de consulta (leitura) têm necessidades diferentes. O **CQRS** (*Command Query Responsibility Segregation*), explicado em CQRS: O PADRÃO ARQUITETURAL QUE TODO PROGRAMADOR PRECISA CONHECER!, separa o modelo de escrita do de leitura. Faz sentido quando há **assimetria**: uma página de produto agrega dezenas de tabelas, enquanto a escrita toca uma ou duas. Réplicas de leitura ajudam, mas não são CQRS, porque mantêm o mesmo modelo.



CQRS completo: escrita no Postgres (fonte da verdade), propagação por CDC/Outbox → Kafka → worker de projeção → banco de leitura desnormalizado. Fonte: elaboração própria com dados do vídeo “CQRS: O PADRÃO ARQUITETURAL QUE TODO PROGRAMADOR PRECISA CONHECER!”.

O ponto delicado é a sincronização. Gravar no banco e depois publicar no Kafka cria o **dual write**: se a segunda operação falha, os lados divergem. As soluções são o **CDC** (ler o log de transações do banco) ou o **Transactional Outbox** (gravar o evento numa tabela *na mesma transação* e publicá-lo depois). O resultado é consistência eventual; validações críticas, como saldo, consultam a fonte da verdade.

Event Sourcing (EVENT SOURCING: APRENDA A CONSTRUIR SISTEMAS QUE VÃO ALÉM DO CRUD, CQRS e EVENT SOURCING | Design de Software Maduro) leva a ideia ao limite: o estado é a soma de eventos imutáveis (ContaAberta, DepósitoFeito…), como um extrato. Um UPDATE apaga a história; um evento a preserva. **Projeções** geram modelos de leitura, **snapshots** evitam reprocessar tudo e o **replay** permite reconstruir o passado, por exemplo para estornar tarifas declaradas indevidas. Correções são eventos inversos, nunca deleções. Ambos são decisões de **domínio** (auditoria, finanças), não modismos: num CRUD simples só multiplicam a complexidade.

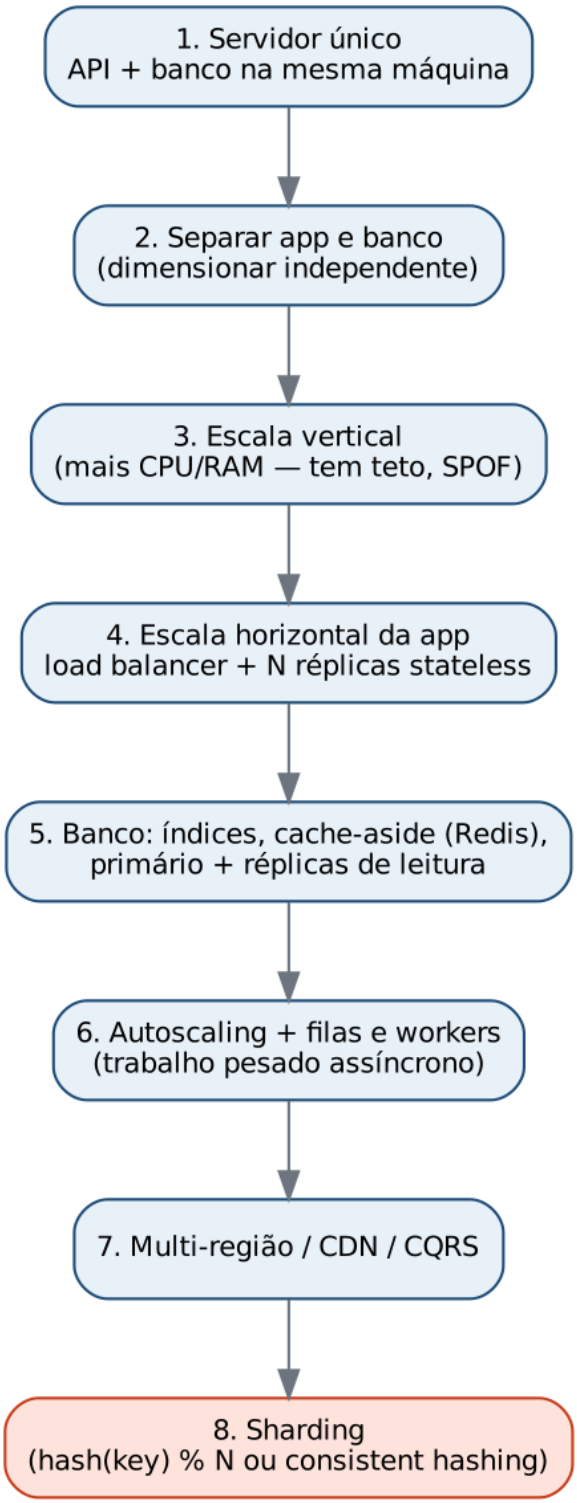
System design: de um servidor a milhões de usuários

> **Para começar**
> Uma barraca de pastel atende dez clientes por hora; para mil, não basta fritar mais rápido: são necessárias mais fritadeiras, uma fila organizada e talvez filiais. **System design** é planejar essa expansão para software. Veremos o roteiro de entrevista, como medir disponibilidade e três estudos de caso.

Roteiro e evolução típica

O vídeo É isso que um SENIOR precisa saber sobre SYSTEM DESIGN sugere um roteiro: requisitos funcionais e não funcionais, estimativas de capacidade, modelo de dados, API, desenho de alto nível e, por fim, gargalos. Avalia-se a capacidade de discutir **trade-offs**: se a leitura leva 60 ms e a meta é 500 ms, colocar Redis só traz custo.

Escala vertical (máquina maior) tem teto e mantém um ponto único de falha; **escala horizontal** (réplicas atrás de um load balancer) exige aplicação **stateless**. A figura a seguir resume a evolução descrita no livro *System Design Interview*, de Alex Xu, recomendado em System Design: Escalando uma Arquitetura do Zero a Um Milhão de Usuários | Arquitetura de Software: load balancer, autoscaling, réplicas de leitura (com consistência eventual; leia do primário logo após escrever), cache, filas com workers para tarefas lentas e, por último, sharding e múltiplas regiões.



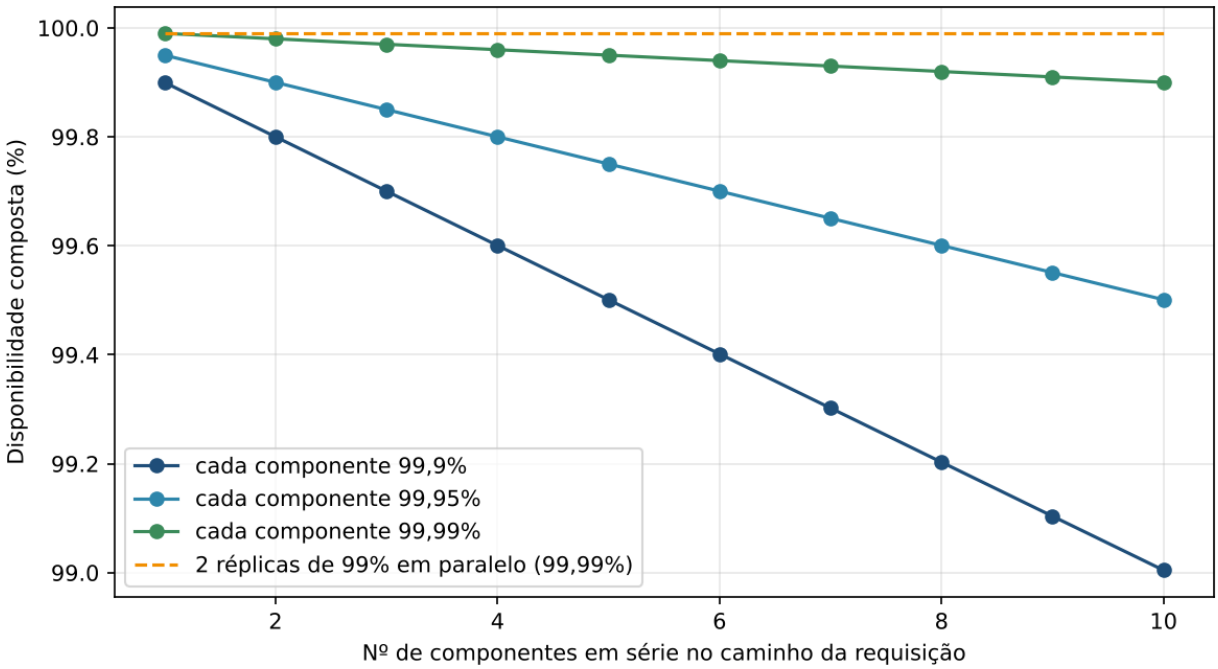
Evolução típica de uma arquitetura até milhões de usuários. Sharding fica por último. Fonte: elaboração própria.

Disponibilidade e o cálculo dos “noves”

O **SLI** é a medida real (fração de requisições boas), o **SLO** é a meta interna (99,9% em 30 dias) e o **SLA** é o contrato com o cliente, geralmente mais frouxo. A diferença 1 − SLO é o **error budget**: enquanto houver orçamento, o time arrisca deploys; quando acaba, prioriza confiabilidade. O downtime permitido é `(1 − disponibilidade) × período`:

Disponibilidade	Downtime/ano	Downtime/mês (30 d)
99%	3,65 dias	7,2 h
99,9%	8,76 h	43,2 min
99,99%	52,6 min	4,32 min
99,999%	5,26 min	25,9 s

O que mais pesa é a **composição**. Componentes em **série** multiplicam disponibilidades: LB (99,99%) × API (99,9%) × banco (99,95%) ≈ **99,84%**, pior que qualquer um isolado. Réplicas em **paralelo** combinam como $1 - (1 - A_1)(1 - A_2)$: duas de 99% dão **99,99%**, como mostra a figura a seguir. Mas só se as falhas forem independentes: duas réplicas na mesma zona, com o mesmo bug, caem juntas.



Disponibilidade composta de N componentes em série (multiplicação) comparada a duas réplicas de 99% em paralelo. Fonte: elaboração própria (cálculo).

A disponibilidade também se escreve A = MTBF / (MTBF + MTTR). Na prática, reduzir o tempo de recuperação (MTTR) com alertas, rollback rápido e runbooks costuma ser mais barato que evitar toda falha.

Estudos de caso

Notificações em larga escala. No exercício do vídeo de system design, é preciso avisar até milhões de inscritos em menos de 5 s quando sai um vídeo novo. A arquitetura separa ingestão, descoberta e entrega com filas: um validador responde rápido ao webhook, um

processador quebra a lista de assinantes em lotes, workers por canal (push, e-mail, SMS) entregam e uma **dead-letter queue** guarda as falhas para reprocessar. Para poucos usuários, um endpoint basta.

Upload e streaming de vídeo. Em ARQUITETANDO O YOUTUBE NA PRÁTICA | SYSTEM DESIGN, as soluções ingênuas falham: vídeo em base64 dentro de JSON cresce 33%, e um POST único esbarra em 413 e timeouts. O desenho que funciona: a API gera **URLs pré-assinadas** para *multipart upload* direto no armazenamento de objetos; o cliente envia pedaços em paralelo; um evento dispara a **transcodificação** em várias resoluções e segmentos curtos (manifestos DASH ou HLS); o player alterna a resolução conforme a banda (**Adaptive Bitrate**); e uma CDN serve os segmentos.

Sistemas com LLM. Em System Design para Aplicações com IA, os princípios são clássicos aplicados a um fornecedor caro, instável e não determinístico: o provedor atrás de um adapter; **saídas estruturadas** (JSON Schema) para domar o não determinismo; orçamentos de tokens por usuário; roteamento para modelos baratos em tarefas simples; cache; jobs assíncronos; máquinas de estado para fluxos; e observabilidade de custo e qualidade, com *failover* quando um modelo degrada.

Testes

> **Para começar**
>
> Um **teste automatizado** é um programa que confere se outro faz o que deveria, como o checklist do piloto antes de decolar. Não garante que nada dará errado, mas pega erros conhecidos de forma rápida e repetível. Com IA escrevendo código, ele vira a principal prova de que o código funciona.

Por que testar, e o que muda com IA

Para o vídeo Code Quality in the Age of AI: Why Great Code Isn't Enough, a confiança no código vem "por evidência, não por autoria", e os testes são a prova primária. O experimento do post do AkitaOnRails citado em "Tipagem" reforça o ponto. A **pirâmide de testes** segue útil:

Nível	Testa	Ferramentas
Unitário (muitos)	Uma peça isolada, com dependências simuladas	JUnit + Mockito, Jest/Vitest, pytest
Integração	Peças reais juntas, como código + banco	Testcontainers
Contrato	Se quem chama e quem responde a API falam a mesma língua	Pact, OpenAPI
E2E (poucos)	Jornada completa no navegador	Playwright, Cypress

Exemplos: Spring e React

No vídeo COMEÇANDO com TESTES UNITÁRIOS no JAVA SPRING | Junit + Mockito, as dependências do serviço viram `@Mock`, o serviço recebe `@InjectMocks` e o teste segue *Arrange, Act, Assert*. Uma lição: foi preciso extrair a chamada ao autorizador externo para um

serviço próprio antes de conseguir mocká-la. Testabilidade exige design (é o DIP na prática). Para queries com sintaxe específica do Postgres, prefira Testcontainers a um banco em memória.

```
@Test
void transfere_quando_autorizado() {
    when(authService.authorize(any(), any())).thenReturn(true);          // Arrange
    service.createTransaction(new TransactionDTO(new BigDecimal("10"), 1L, 2L)); // Act
    verify(repository, times(1)).save(any());                             // Assert
    assertEquals(new BigDecimal("0"), sender.getBalance());
}
```

No front, a filosofia da Testing Library (Elevando os Testes Unitários da sua aplicação React usando Jest & Testing Library) é testar o que o usuário vê, não a implementação. Prefira `getByRole('button', { name: /salvar/i })` a IDs de teste, o que verifica acessibilidade de graça; para chamadas assíncronas, use `findBy*` ou `waitFor`.

TDD e testes como especificação

O **Test-Driven Development**, de Kent Beck, segue o ciclo **red → green → refactor**. Com agentes de IA, o teste vira a **especificação executável** que impede o agente de "trapacear". Daí regras de AGENTS.md (o arquivo de instruções para agentes) como as de Práticas do Akita com IA. Deu bom? (Boas Práticas de IA): "use TDD", "nunca delete testes sem aprovação", "rode os linters".

> **Você sabia?**
> Em 1968, uma conferência da OTAN em Garmisch, na Alemanha, popularizou o termo "engenharia de software" para enfrentar a "crise do software": projetos atrasados, caros e cheios de defeitos.

DevOps, entrega e operação

> **Para começar**
> O código ainda precisa chegar aos usuários e continuar funcionando. **DevOps** une quem desenvolve e quem opera, como uma linha de montagem com inspeções automáticas e sensores que avisam quando algo quebra. Veremos contêineres, CI/CD, nuvem e monitoramento.

Contêineres e Docker

Uma **imagem** é um sistema de arquivos em camadas, imutável, descrito por um `Dockerfile`. Um **contêiner** é um processo isolado que compartilha o kernel do host, e não uma VM. O **Compose** descreve vários serviços (app, banco, cache) para desenvolvimento local reproduzível.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci                  # camada cacheada enquanto o lockfile não mudar
COPY . .
RUN npm run build

FROM node:22-alpine         # imagem final sem ferramentas de build
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/node_modules ./node_modules
USER node                   # não rodar como root
CMD ["node", "dist/server.js"]
```

Boas práticas: copiar o manifesto antes do código, multi-stage, imagens base enxutas e fixadas, `.dockerignore` e nenhum segredo na imagem.

CI/CD

A **integração contínua** roda build, lint e testes a cada push. A **entrega contínua** deixa cada versão pronta para produção; a **implantação contínua** a publica automaticamente (CD é *delivery* ou *deployment*, não "development"). O vídeo Como Eu Começo Um Projeto do Zero recomenda **publicar o "Hello World" em produção no primeiro dia**, com deploy a cada merge na `main`, para que os problemas de "funciona na minha máquina" apareçam quando ainda são baratos.

```
# .github/workflows/ci.yml (esqueleto)
on: [pull_request, push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci && npm run lint && npm test
  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps: [ { uses: actions/checkout@v4 }, { run: ./scripts/deploy.sh } ]
```

Regras de arquitetura e segurança devem morar no pipeline (análise estática, *policy as code*), não numa wiki: o objetivo é "tornar o caminho certo o mais fácil". A infraestrutura também vira código (**IaC**: Terraform, manifestos do Kubernetes), versionado e revisado. Na nuvem, o serverless "escala pelo cartão de crédito", então rate limiting e alarmes de cobrança são obrigatórios.

Observabilidade

Observabilidade é entender o sistema pelos sinais que ele emite, como o painel de um carro: **logs** estruturados (com *correlation id* e sem dados sensíveis), **métricas** e **traces** distribuídos (OpenTelemetry). Os *golden signals* do Google SRE são latência (em percentis p95/p99, nunca só a média), tráfego, erros e saturação. O **monitoramento sintético** (Synthetic Monitoring Explained: A Guide to Reliable DevOps Workflows) simula usuários com scripts agendados, de várias regiões, e pega o caso "está no ar, mas inutilizável".

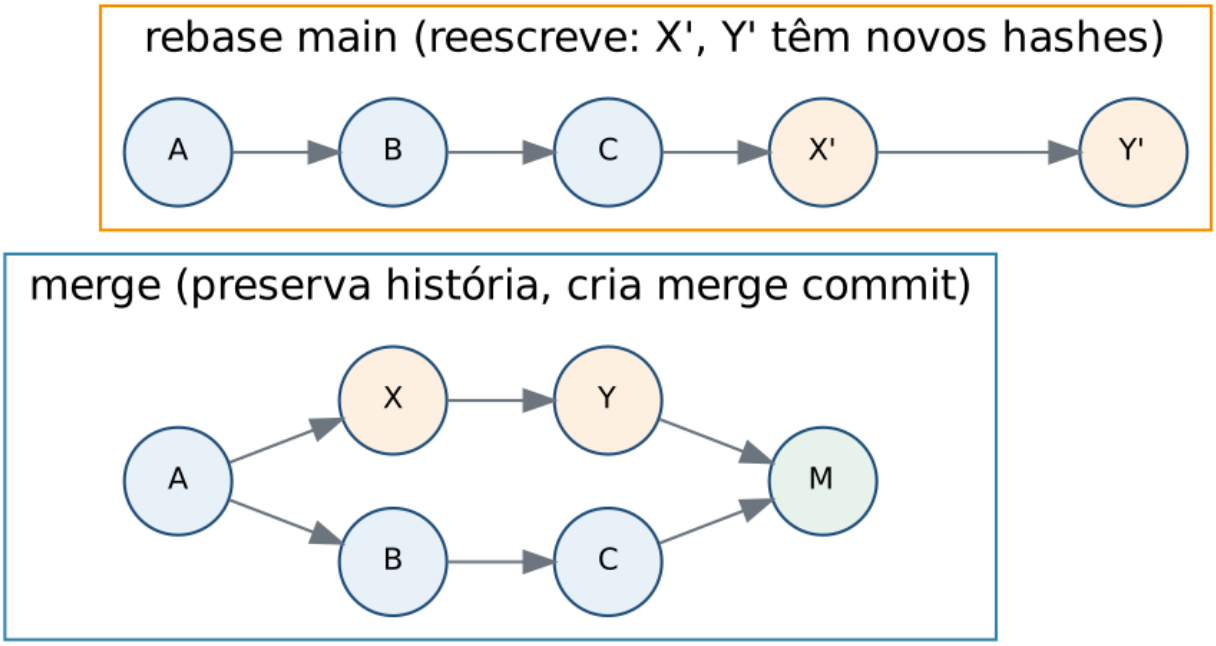
Comece por 3 a 5 jornadas críticas e alerte sobre *sintomas* que afetam o usuário (error budget queimando rápido), não sobre causas como CPU em 80%.

Git no dia a dia

> **Para começar**
> O **Git** guarda cada mudança como um **commit** (uma foto do projeto) e permite linhas paralelas de trabalho (**branches**) que depois se juntam. Depois do básico, todo mundo esbarra em "minha branch ficou desatualizada" ou "preciso só daquele commit".

O vídeo TUTORIAL GIT | Git Rebase, Git Cherry Pick, Git Ammend, Git Rebase Interativo apresenta quatro comandos:

- **`git rebase main`** reaplica seus commits sobre a ponta da main, com **novos hashes** e histórico linear; o `merge` preserva a história com um *merge commit* (veja a figura a seguir). Para atualizar um PR: `git fetch`, `git rebase origin/main`, resolver conflitos, `git rebase --continue` e `git push --force-with-lease`.
- **`git stash`** guarda o trabalho em andamento para atender um hotfix; `stash pop` o traz de volta.
- **`git cherry-pick <hash>`** traz um commit específico.
- **`git commit --amend`** refaz o último commit (também muda o hash).



Merge × rebase: o merge cria o commit M e preserva os dois ramos; o rebase reescreve X e Y como X′ e Y′ sobre C. Fonte: elaboração própria com dados do vídeo "TUTORIAL GIT | Git Rebase, Git Cherry Pick, Git Ammend, Git Rebase Interativo".

Regra de ouro: **nunca reescreva histórico público**. Na sua própria branch, prefira `--force-with-lease`, que recusa o push se houver commits que você não viu. Outras práticas: PRs pequenos (algumas centenas de linhas), *Conventional Commits*, `main` protegida com

CI obrigatório e **git worktrees** para trabalhar em várias branches ao mesmo tempo, hoje muito usados para rodar agentes de IA em paralelo (ver Livro 3, Capítulo 1, Programando com IA).

> **Você sabia?**
> Linus Torvalds escreveu a primeira versão do Git em abril de 2005, em poucos dias, depois que o kernel Linux perdeu a licença gratuita do BitKeeper. “Git” é gíria britânica para pessoa teimosa e desagradável, e Torvalds brincou que batiza projetos em sua própria homenagem.

Boas práticas, documentação e engenharia na era da IA

> **Para começar**
> Bom profissional é quem deixa o trabalho fácil de entender e de continuar, como o cozinheiro que limpa a bancada e anota a receita. Esta seção reúne hábitos de código, documentos de decisão, código legado e o que a IA muda no ciclo de vida do software.

Código que um revisor confia

O vídeo 5 Erros que Todo Júnior comete NO CÓDIGO que o Impedem de virar PLENO (E Como Resolvê-los) resume o básico: **early return** em vez de `else`, tratar erros, **nomes** que se explicam, conhecer `map`/`filter`/`reduce` e **DRY**. Uma ressalva: DRY é sobre *conhecimento* duplicado; duas funções parecidas que mudam por motivos diferentes não devem ser fundidas (“a duplicação é melhor que a abstração errada”). Some a isso **KISS** (mantenha simples) e **YAGNI** (você não vai precisar disso).

Documentação: PRD, RFC, Design Doc e ADR

O vídeo Como Manter CONTEXTO em 1 MILHÃO de Linhas de CÓDIGO propõe documentos proporcionais ao risco:

Documento	Responde	Regra de ouro
PRD	O quê, para quem, por quê	Uma página, com métricas de sucesso; nunca a solução técnica
RFC	Qual caminho?	Só para decisões caras ou irreversíveis; inclua a opção “não fazer nada”
Design Doc	Como construir	Alternativas, riscos, testes, monitoramento e rollback
ADR	O que decidimos e por quê	Uma página, imutável; se substituído, marque como revogado

Mudança pequena e reversível vai direto para a spec; pagamentos e dados pessoais pedem o pacote completo. Documento desatualizado é pior que nenhum. Para trabalhar com agentes, mantenha curtos o **README** (como instalar, rodar e testar) e o **AGENTS.md** (regras para o agente); um AGENTS.md de mil linhas dilui as instruções.

Começando do zero e modernizando o legado

O checklist de "dia 1" de Como Eu Começo Um Projeto do Zero: escolher a stack pelo objetivo (para ganhar dinheiro, use o que já domina; frameworks "baterias incluídas" como Django, Rails e Laravel ajudam o dev solo), planejar o MVP num documento no repositório, publicar o Hello World com CD, configurar migrações, pôr testes no pipeline e escrever README e AGENTS.md.

Código **legado** funciona, mas não tem testes, documentação nem quem entenda todas as regras, e os especialistas em COBOL estão se aposentando. O vídeo What Is Legacy Code? How AI Is Modernizing Legacy Systems defende modernização **incremental**. A IA acelera a descoberta, a tradução e a geração de testes, mas uma tradução sintaticamente correta pode se comportar errado; por isso o primeiro passo continua sendo cobrir o comportamento atual com **testes de caracterização**.

O ciclo de vida do software com IA

Os vídeos AI in the SDLC: Rethinking AI Coding Tools & AI Agents e What Is RAD? Why It Matters in the Age of AI Coding partem de um dado incômodo: num ensaio controlado da METR (2025), desenvolvedores experientes se *sentiram* ~20% mais rápidos com IA, mas foram ~20% mais *lentos*. Acelerar a digitação adianta pouco se o tempo vai para as esperas entre etapas e para revisar milhares de linhas não lidas. A **superdelegação** ("construa um e-commerce") deixa decisões implícitas; a **subdelegação** desperdiça a ferramenta. O caminho é redesenhar o ciclo: **spec-driven development** no lugar do vibe coding, testes gerados a partir das histórias de usuário e métricas de resultado, não de linhas de código. O RAD de James Martin, dos anos 1980, já previa protótipos rápidos com o usuário; faltavam ferramentas, e os agentes agora as oferecem.

A **revisão de código**, segundo How AI Is Changing Code Reviews & Software Development, foi das inspeções formais de Michael Fagan na IBM, nos anos 1970, aos pull requests e agora à IA. O foco migrou de "como construímos?" para "construímos o que queríamos?", com base em evidências: testes, checagens automáticas e dados de execução.

> **Em resumo**
> Qualidade de implementação está ficando barata; qualidade de decisão é o diferencial. Diante de uma feature gerada por IA, pergunte: síncrona ou assíncrona? O que acontece se o serviço downstream cair? Quantos retries? E se os usuários passarem de 10 mil para 10 milhões? Este capítulo existe para que você saiba responder.

Fontes deste capítulo

Vídeos e posts citados neste capítulo, todos públicos.

Título	URL	Canal	Ponto principal
CS50x - Lecture 8 - HTML, CSS, JavaScript	https://www.youtube.com/watch?v=yYst7puZXjw	CS50	Internet, HTTP e o trio HTML/CSS/JS.

Título	URL	Canal	Ponto principal
Elevando os Testes Unitários da sua aplicação React usando Jest & Testing Library	https://www.youtube.com/watch?v=0dRPPmRYM3o	Fernanda Kipper	Testes de componentes React com Testing Library.
Curso de PROGRAMAÇÃO JAVA para INICIANTES	Fundamentos + Programação Orientada Objetos	https://www.youtube.com/watch?v=nODe5lFcGpg	Fernanda Kipper	Fundamentos de Java e orientação a objetos.
É isso que um SENIOR precisa saber sobre SYSTEM DESIGN	https://www.youtube.com/watch?v=yaNjUYg6yY0	Fernanda Kipper	Roteiro de system design e notificações em escala.
ARQUITETANDO O YOUTUBE NA PRÁTICA	SYSTEM DESIGN	https://www.youtube.com/watch?v=JBivKeZVex0	Renato Augusto	Upload multipart, transcodificação e streaming adaptativo.
APRENDA NGINX DO ZERO	Curso de NGINX em 1 HORA	https://www.youtube.com/watch?v=gd_cUmwzgEM	Fernanda Kipper	Nginx como proxy reverso e balanceador.
Como Escolher o Banco de Dados Correto pra sua Aplicação	System Design & Arquitetura de Software	https://www.youtube.com/watch?v=bhw4-Kq_RPs	Renato Augusto	Escolher banco: replicação, consistência, CAP, PACELC.
CURSO de JAVA INTERMEDIÁRIO	Estruturas de dados, Stream API, Generics, Packages, Records	https://www.youtube.com/watch?v=jLropHUgI1A	Fernanda Kipper	Collections, generics, records e Stream API.
Aprenda a Escalar Bancos de Dados Usando SHARDING	System Design & Arquitetura de Software	https://www.youtube.com/watch?v=xJllDyCIyws	Renato Augusto	Sharding: escalar o banco horizontalmente.
Criando APLICAÇÃO COMPLETA e ESCALÁVEL com Angular + Edge Computing	https://www.youtube.com/watch?v=o2_RBMABx4s	Fernanda Kipper	Angular com geração estática servida na borda.
System Design: Escalando uma Arquitetura do Zero a Um Milhão de Usuários	Arquitetura de Software	https://www.youtube.com/watch?v=9g7twJrXqoY	Renato Augusto	Evolução de um servidor a milhões de usuários.
CURSO DE TYPESCRIPT para INICIANTES	Aprenda Typescript na Prática	https://www.youtube.com/watch?v=QoqDr4H2G8U	Fernanda Kipper	TypeScript: tipagem estática e tsconfig.
COMEÇANDO com TESTES UNITÁRIOS no JAVA SPRING	Junit + Mockito	https://www.youtube.com/watch?v=T6ChO8LQxRE	Fernanda Kipper	JUnit 5 e Mockito no Spring Boot.
SERVER-SENT EVENTS: Aprenda a Construir Sistemas que se Comunicam em Tempo Real	https://www.youtube.com/watch?v=uiT4oK19hu4	Renato Augusto	SSE com Redis Pub/Sub para tempo real.
O QUE NINGUÉM TE ENSINOU SOBRE BUSCAS INTELIGENTES NO BANCO DE DADOS!	https://www.youtube.com/watch?v=UYOr-rpQs1I	Renato Augusto	Full-Text Search no MySQL e PostgreSQL.
EVENT SOURCING: APRENDA A CONSTRUIR SISTEMAS QUE VÃO ALÉM DO CRUD	https://www.youtube.com/watch?v=jzMghF8n26E	Renato Augusto	Event store, projeções, snapshots e replay.
System Design para Aplicações com IA	https://www.youtube.com/watch?v=2Ls_uc0fo-Y	Augusto Galego	Princípios de system design para apps com LLM.
Como Manter CONTEXTO em 1 MILHÃO de Linhas de CÓDIGO	https://www.youtube.com/watch?v=FY8dRVejhD0	O Novo Programador	PRD, RFC, Design Doc e ADR por feature.
CACHE-ASIDE: Escalabilidade, Performance e Arquitetura de Software pra Mandar Bem na Entrevista!	https://www.youtube.com/watch?v=vRO0UfvsbDw	Renato Augusto	Cache-aside com Redis para aliviar o banco.
Rate Limiter	Explicação Completa	https://www.youtube.com/watch?v=mlzPJlQeqBM	Augusto Galego	Algoritmos de rate limiting e onde aplicá-los.
TUTORIAL GIT	Git Rebase, Git Cherry Pick, Git Ammend, Git Rebase Interativo	https://www.youtube.com/watch?v=7Y5ywzYIfG4	Fernanda Kipper	Rebase, stash, cherry-pick e amend.
Entendendo COLETOR DE LIXO DO JAVA (memória Heap, Stack, Garbage Collector)	https://www.youtube.com/watch?v=cgUfurMJosE	Fernanda Kipper	Stack, heap e coletores de lixo do Java.
CQRS: O PADRÃO ARQUITETURAL QUE TODO PROGRAMADOR PRECISA CONHECER!	https://www.youtube.com/watch?v=8Z8JAtEEan0	Renato Augusto	Quando separar comandos de consultas (CQRS).
ARQUITETURA BACKEND DO FACEBOOK	System Design	https://www.youtube.com/watch?v=6bTkTy6DjOQ	Fernanda Kipper	Como o Facebook escalou a Timeline.
5 BOAS PRÁTICAS de UI/UX para melhorar seus PROMPTS (frontend)	https://www.youtube.com/watch?v=Qdw3Dt_-tzc	Fernanda Kipper	Princípios de UI/UX para interfaces geradas por IA.
TUTORIAL COMPLETO CSS MEDIA QUERY - Criando layouts responsivos na PRÁTICA	https://www.youtube.com/watch?v=S-TqflGxHd4	Fernanda Kipper	Media queries, mobile first e breakpoints.
Fundamentos de Arquitetura de Software para Backend	https://www.youtube.com/watch?v=jwBpiEoo8rI	Fernanda Kipper	Definição de arquitetura e seis conceitos-base.

Título	URL	Canal	Ponto principal
Me de 30 minutos para aprender Arquitetura Limpa e Hexagonal	https://www.youtube.com/watch?v=HTrLgUKQLvs	Fernanda Kipper	Clean e Hexagonal protegem o núcleo de negócio.
Quando usar Monolitos vs Microserviços	https://www.youtube.com/watch?v=ooJjxNsQnK4	Augusto Galego	Que benefício justifica uma chamada de rede?
CQRS e EVENT SOURCING | Design de Software Maduro	https://www.youtube.com/watch?v=0B20k31Gyq0	Augusto Galego	CQRS e eventos como fonte da verdade.
Todo DEV precisa entender isso: IDEMPOTÊNCIA	https://www.youtube.com/watch?v=YNHKO_74sLU	Augusto Galego	Idempotência e chaves de idempotência em pagamentos.
O que é WebSocket? Entenda na prática em 20 minutos (Guia para programadores)	https://www.youtube.com/watch?v=LBTY5BaWLH0	Attekita Dev	WebSocket: conexão persistente e bidirecional.
O Princípio SOLID mais CRÍTICO de TODOS! | Princípio de Substituição de LISKOV	https://www.youtube.com/watch?v=f6-5ANuTkys	Renato Augusto	Liskov: pré-condições, pós-condições e invariantes.
Dominando príncipios de design de código | SOLID	https://www.youtube.com/watch?v=5_fmI1nmWOQ	Fernanda Kipper	SOLID num sistema de matrículas em Java.
Princípios SOLID Pelos Olhos de um Dev Sr.	https://www.youtube.com/watch?v=2yqHlJ2HbTo	Augusto Galego	SOLID com olhar crítico, sem dogma.
JAMAIS cometa esses ERROS em JavaScript	https://www.youtube.com/watch?v=n8RLfLunQzc	Fernanda Kipper	Erros clássicos: mutação, null, this, assincronia.
Microfrontends Explicado Simples: A Arquitetura que Está Mudando o Frontend	https://www.youtube.com/watch?v=HBi6jJUltKQ	Attekita Dev	Micro front-ends: benefícios, custos e integração.
Como sistemas evitam requisições duplicadas	https://www.youtube.com/watch?v=xsCOMiUwpeg	Augusto Galego	Camadas de defesa contra requisições duplicadas.
5 Design Patterns Que Todo Dev Deveria Conhecer	https://www.youtube.com/watch?v=9sI8KHEjs0M	Augusto Galego	Strategy, Observer, Adapter, retry e circuit breaker.
5 Erros que Todo Júnior comete NO CÓDIGO que o Impedem de virar PLENO (E Como Resolvê-los)	https://www.youtube.com/watch?v=3ICKCyda8sE	O Novo Programador	Hábitos que minam a confiança do revisor.
Quando Vale a Pena Desacoplar Seu Código?	https://www.youtube.com/watch?v=ecev-RuuKVg	Augusto Galego	Acoplamento é espectro: quando abstrair ou enfileirar.
Separando Dev Júnior de Dev Senior (Object Calisthenics)	https://www.youtube.com/watch?v=Ss6LAFC_9pw	Augusto Galego	As nove regras do Object Calisthenics.
ACID e BASE explicado de forma simples	https://www.youtube.com/watch?v=8KUyqiSkQIQ	Augusto Galego	Garantias ACID e BASE e seus trade-offs.
Como Fazer Migrações de Banco de Dados do Jeito Certo	https://www.youtube.com/watch?v=Go7rKLtJ0ec	Augusto Galego	Migrações versionadas, nunca à mão em produção.
Como ORMs Mentem pra Você	https://www.youtube.com/watch?v=upn1vRXg-Og	Augusto Galego	ORMs escondem N+1 e joins caros.
ERROS COMUNS | Null Pointer e None	https://www.youtube.com/watch?v=sVfFxC5pC8k	Augusto Galego	Causas comuns de erro por None.
Arquitetura Hexagonal | Explicação de um Dev Sr.	https://www.youtube.com/watch?v=LHhm1gzIzOE	Augusto Galego	Ports and Adapters explicado na prática.
WEBHOOKS vs WEBSOCKETS (na prática)	https://www.youtube.com/watch?v=k0QTuaK6LX4	Augusto Galego	Webhook é API invertida; WebSocket, conexão persistente.
Como Uma Requisição Contamina Outra	https://www.youtube.com/watch?v=5vKcNaWXtCY	Augusto Galego	Estado global contamina requisições paralelas.
Pare de quebrar o banco em produção: Aprenda Database Branching	https://www.youtube.com/watch?v=llC6b8g6FgY	Fernanda Kipper	Banco isolado por branch com copy-on-write.
Aprenda MONOLITO MODULAR ANTES de MICROSERVIÇOS	https://www.youtube.com/watch?v=CwIyDq6nhdk	Fernanda Kipper	Módulos com fronteiras num único deploy.
Console log pode TRAVAR a sua aplicação (explicação)	https://www.youtube.com/watch?v=V3tyJSEDiuM	Augusto Galego	console.log síncrono pode bloquear o event loop.
4 Sintomas de que Seu Código Está APODRECENDO e Você Precisa Identificar Agora!	https://www.youtube.com/watch?v=S2mMFopw_ww	Renato Augusto	Rigidez, fragilidade, imobilidade e viscosidade.
5 RECURSOS DO JAVA QUE VOCÊ PRECISA CONHECER	https://www.youtube.com/watch?v=WpVF3S4kuK0	Fernanda Kipper	Text blocks, records, switch com seta, lambdas.
DTOs, o que é e porque usar	https://www.youtube.com/watch?v=jpeUKuCexGc	Augusto Galego	DTO como contrato para validar e transportar dados.
O que fazer quando RESTful não for suficiente	https://www.youtube.com/watch?v=Cvr0cGtq3fQ	Augusto Galego	gRPC e Protocol Buffers comparados a REST.

Título	URL	Canal	Ponto principal
VOCÊ NÃO SABE ARQUITETURA DE SOFTWARE (veja esse vídeo)	https://www.youtube.com/watch?v=SWgCLOW1bCE	Fernanda Kipper	Arquitetura torna o sistema fácil de manter.
Como Validar Email do Jeito Certo!	https://www.youtube.com/watch?v=RsSE7ZWUC6k	Augusto Galego	Validar e-mail de verdade exige confirmação.
Como 1.000 Requests Usam 20 Conexões | Connection Pooling	https://www.youtube.com/watch?v=aqmt_3UWItA	Augusto Galego	Pool de conexões e o problema no serverless.
Como Eu Começo Um Projeto do Zero	https://www.youtube.com/watch?v=PADhI0fZnTI	Augusto Galego	Primeiro dia: stack, MVP, deploy e migrações.
O custo de usar muitas dependências	https://www.youtube.com/watch?v=nJpXU6RSvO8	Augusto Galego	Custos ocultos de cada dependência externa.
How AI Is Changing Code Reviews & Software Development	https://www.youtube.com/watch?v=c57vAe-mMLo	IBM Technology	Cinco eras da revisão de código.
Code Quality in the Age of AI: Why Great Code Isn't Enough	https://www.youtube.com/watch?v=GRYZR2R20FI	IBM Technology	Com IA, confiança vem de evidência, não autoria.
O que é uma CDN? Para que serve a CDN?	https://www.youtube.com/watch?v=pOFToA0Upek	Fernanda Kipper	CDN entrega arquivos perto do usuário.
AI in the SDLC: Rethinking AI Coding Tools & AI Agents	https://www.youtube.com/watch?v=4wMRXmLpdA8	IBM Technology	Ganhos da IA somem nas esperas entre etapas.
What Is RAD? Why It Matters in the Age of AI Coding	https://www.youtube.com/watch?v=J0zbWsutyA8	IBM Technology	RAD de James Martin revisitado com agentes.
IDE vs CLI: What Every DevOps Engineer Should Know	https://www.youtube.com/watch?v=5RtmQSKKQo8	IBM Technology	O atrito está nas passagens entre ferramentas.
What Is an AI IDE? How AI Is Changing Developer & Coding Tools	https://www.youtube.com/watch?v=fWZylhIHF9w	IBM Technology	IDEs e o impacto da IA integrada.
What Is Legacy Code? How AI Is Modernizing Legacy Systems	https://www.youtube.com/watch?v=CKC47S92wA4	IBM Technology	IA na modernização incremental de código legado.
Apache vs NGINX	https://www.youtube.com/watch?v=9nyiY-psbMs	IBM Technology	Apache × Nginx, proxy reverso e balanceamento.
What Is an API? Types, Uses, & AI Integration	https://www.youtube.com/watch?v=Qw8qVSZEWBc	IBM Technology	O que é uma API e seu papel.
O que é CORS? Análise frontend e backend	https://www.youtube.com/watch?v=0gXiTWMikDU	Augusto Galego	CORS, preflight e por que só no navegador.
Synthetic Monitoring Explained: A Guide to Reliable DevOps Workflows	https://www.youtube.com/watch?v=fPWjWTFveBQ	IBM Technology	Simular usuários para detectar falhas antes deles.
MySQL vs MongoDB	https://www.youtube.com/watch?v=OdgZ0jr4jpM	IBM Technology	MySQL relacional × MongoDB de documentos.
O que é SQL injection e como resolver	https://www.youtube.com/watch?v=THwPB7OrMyw	Augusto Galego	SQL injection e consultas parametrizadas.
Práticas do Akita com IA. Deu bom? (Boas Práticas de IA)	https://www.youtube.com/watch?v=ytTJtfg9HCw	Augusto Galego	PRs pequenos, AGENTS.md enxuto e TDD.
Existem linguagens melhores pra programar com LLMs?	https://akitaonrails.com/2026/10/11/existem-linguagens-melhores-pra-programar-com-llms/	Blog AkitaOnRails (11/10/2026)	Tipagem não reduziu reparos; testes sim.

CAPÍTULO 2 ·

Algoritmos e Ciência da Computação

> **Para começar**
>
> Pense numa cozinha: a receita é o **algoritmo**, a despensa é a **memória** e o jeito de organizar a despensa são as **estruturas de dados**. Ciência da computação, no dia a dia, é escolher boas receitas e boas despensas para que o jantar saia a tempo mesmo com mil convidados em vez de dez.

> *Modelos de linguagem escrevem código cada vez melhor, mas quem decide se o código serve ainda precisa saber por que um laço aninhado não escala, por que `0.1 + 0.2` não dá `0.3` e por que um caixeiro-viajante com 20 cidades esgota qualquer força bruta. Este capítulo se apoia nas aulas do CS50 de Harvard, em vídeos de canais brasileiros e em um post de Fabio Akita sobre provas formais, e segue a ordem natural do assunto: algoritmo, custo, busca e ordenação, recursão, memória, estruturas de dados, problemas difíceis, programação dinâmica e entrevistas técnicas.*

O que é um algoritmo e o que a máquina faz

> **Para começar**
>
> Um algoritmo é uma receita tão precisa que até quem não tem bom senso, como um computador, consegue segui-la. "Tempere a gosto" não serve; "coloque 5 g de sal" serve.

Na definição usada por Doug Lloyd em CS50 for Business - Lecture 1 - Analyzing Algorithms, um **algoritmo** é uma sequência de passos claros, ordenados e sem ambiguidade, com entradas e saídas definidas. A definição clássica de Knuth também exige que ele *termine*.

A base teórica vem de 1936, quando Alan Turing descreveu uma máquina abstrata com uma fita infinita, uma cabeça de leitura e escrita e uma tabela finita de transições. A **tese de Church-Turing** afirma que tudo o que é "efetivamente computável" pode ser computado por essa máquina, e o lambda-cálculo de Church tem o mesmo poder. Consequência prática: em qualquer linguagem Turing-completa, recursão e iteração são intercambiáveis.

Na **máquina determinística**, cada situação tem no máximo uma ação; na **não determinística**, pode haver várias, e a máquina aceita se *algum* caminho aceitar. A classe P reúne os problemas resolvíveis em tempo polinomial pela primeira; NP, aqueles cuja solução proposta pode ser *verificada* em tempo polinomial.

Compilação e limites físicos

Um **compilador** traduz o texto que humanos escrevem nas instruções numéricas que o processador executa. A aula CS50x em Português - Aula 1 - C mostra o ciclo `code hello.c`, `make hello`, `./hello` e insiste em três critérios de qualidade: **correção**, **design** (sem trabalho redundante) e **estilo** (legibilidade).

```
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
}
```

Números no computador têm tamanho fixo. Um `int` de 32 bits tem cerca de 4 bilhões de valores; somar 1 ao maior causa *integer overflow*, e em C o comportamento com tipos com sinal é indefinido. Casos reais:

Caso	Causa	Consequência
Boeing 787 (2015)	Contador de centésimos de segundo em inteiro de 32 bits com sinal	Estouro após 248 dias ligado; a FAA exigiu reinícios periódicos
Pac-Man (1980)	Número da fase em 1 byte	Nível 256 corrompe metade da tela
Ano 2038	`time_t` de 32 bits com sinal	Em 19/01/2038 03:14:07 UTC volta para 1901

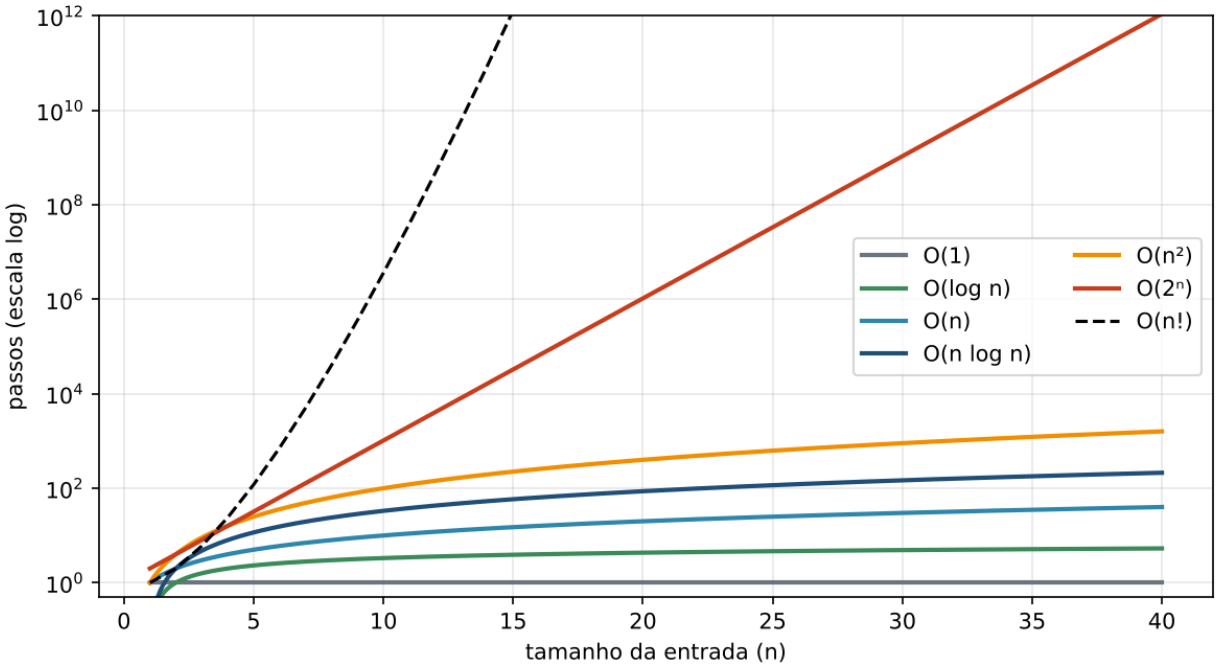
Medindo custo: notação assintótica

> **Para começar**
> Procurar um nome entre 10 convidados é fácil com qualquer método; entre 10 milhões, o método decide se leva um segundo ou um dia. O "Big-O" diz quanto o trabalho cresce quando a entrada cresce, sem depender do computador.

Conta-se o número de **passos** em função do tamanho da entrada n. Na abertura de CS50x em Português - Aula 3 - Algoritmos, contar a plateia de um em um leva n passos; com pares que somam suas contagens enquanto metade se senta, leva $\log_2 n$, e dobrar a plateia custa só um passo a mais.

- **O(f(n))**: limite superior; o custo cresce *no máximo* como f(n), a menos de constantes.
- **Ω(f(n))**: limite inferior; cresce *pelo menos* como f(n).
- **Θ(f(n))**: os dois limites coincidem.

Formalmente, $T(n) \in O(f(n))$ se existem $c > 0$ e n_0 tais que $T(n) \le c \cdot f(n)$ para todo $n \ge n_0$. Constantes e termos de menor ordem somem: o termo dominante decide. Cuidado com a simplificação "O = pior caso, Ω = melhor caso": pior, melhor e médio são *cenários*; O, Ω e Θ são *tipos de limite* que valem para qualquer cenário. Em entrevistas, diga de qual caso está falando.



Crescimento das classes de complexidade em escala logarítmica. Com n = 40, um algoritmo O(n log n) faz cerca de 200 passos, um $O(2^n)$ cerca de 10^{12} e um O(n!) passa de 10^{47}. Fonte: elaboração própria com dados de "CS50 for Business - Lecture 1 - Analyzing Algorithms", "Big O Notation fácil de entender!" e "Entenda FUNDAMENTOS do CURSO de CIÊNCIA da COMPUTAÇÃO".

Classe	Nome	Exemplo do dia a dia	n = 1.000
O(1)	constante	abrir o livro na página marcada	1
O(log n)	logarítmica	achar uma palavra no dicionário abrindo no meio	~10
O(n)	linear	procurar a chave olhando gaveta por gaveta	1.000
O(n log n)	log-linear	ordenar provas dividindo em montinhos e juntando	~10.000
$O(n^2)$	quadrática	cada convidado cumprimenta todos os outros	1.000.000
$O(2^n)$	exponencial	testar todas as combinações de coberturas de pizza	$\sim 10^{301}$
O(n!)	fatorial	testar todas as ordens de sentar os convidados	$\sim 4\times 10^{2567}$

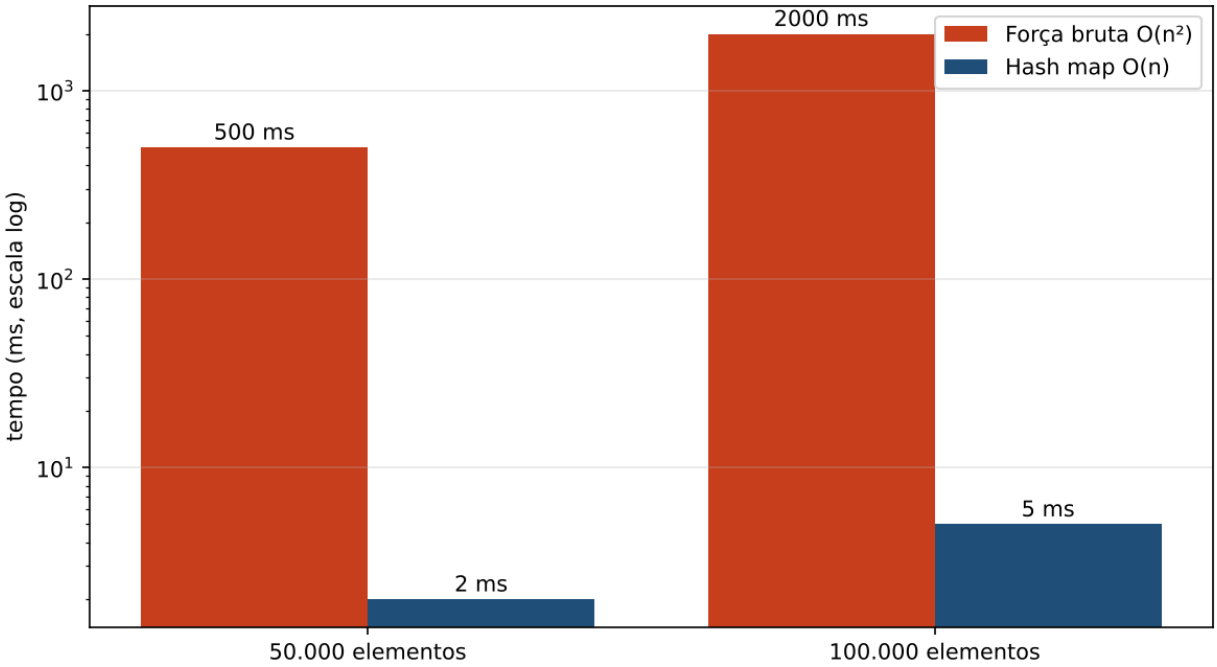
Algoritmos exponenciais não ficam viáveis com hardware melhor: um computador mil vezes mais rápido acrescenta só uns 10 elementos ao maior n tratável por um $O(2^n)$. Pelo mesmo motivo a criptografia funciona: cada bit a mais na chave dobra o trabalho da força bruta.

Espaço e desempenho real

Memória se analisa do mesmo jeito. No vídeo Big O Notation fácil de entender! (Complexidade de Tempo e Espaço na Programação), achar duplicados comparando todos os pares custa $O(n^2)$ de tempo e O(1) de espaço; com um conjunto de itens já vistos, o tempo cai para O(n) ao preço de O(n) de memória.

```
def find_duplicates(items):
    seen, dups = set(), set()
    for x in items:          # O(n) tempo
        if x in seen:        # O(1) em média
            dups.add(x)
        seen.add(x)          # O(n) espaço
    return dups
```

Em Quando um algoritmo ruim é melhor do que um bom..., Augusto Galego lembra que Big-O mede **escalabilidade**, não velocidade absoluta. O hash map tem custos fixos (alocar, calcular hashes, tratar colisões); com poucos elementos, dois laços aninhados sobre um bloco contíguo em cache vencem.



Two Sum em Go: força bruta O(n²) contra hash map O(n). Com 100 mil elementos, cerca de 2 s contra 5 ms; abaixo de 10 elementos, empate. Fonte: elaboração própria com dados de "Quando um algoritmo ruim é melhor do que um bom...".

> **Dica**
> Para prever o efeito de uma mudança de escala, aplique a classe: num O(n²), dobrar n quadruplica o tempo. Se a medição não bate com a classe, o gargalo está em outro lugar (alocação, I/O, cache, GC).

Busca e ordenação

> **Para começar**
> Um dicionário de papel é rápido porque está em ordem: você abre no meio e sabe para que lado ir. Numa pilha bagunçada, só resta olhar folha por folha.

A **busca linear** verifica os elementos um a um e funciona com qualquer arranjo: O(n) no pior caso, Ω(1) no melhor. A **busca binária** exige dados ordenados: olha o meio, descarta a metade que não contém o alvo e repete, em O(log n). Com um milhão de nomes bastam cerca de 20 comparações.

```
def binary_search(a, target):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = lo + (hi - lo) // 2   # evita overflow em linguagens com int fixo
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

O cálculo `(lo + hi) / 2` estourou inteiros na biblioteca do Java por quase uma década (bug relatado por Joshua Bloch em 2006), e errar entre < e <= gera laços infinitos. Ordenar para depois buscar só compensa se a busca se repete: ordenar custa O(n log n), e esse custo se dilui (é *amortizado*) em muitas buscas de O(log n).

Algoritmos de ordenação

- **Selection sort**: acha o menor restante e o põe na posição i; sempre $n(n-1)/2$ comparações.
- **Bubble sort**: troca vizinhos fora de ordem; com parada antecipada, é $\Omega(n)$ em entrada ordenada.
- **Insertion sort**: insere cada elemento no prefixo ordenado; ótimo em dados quase ordenados e usado em híbridos para pedaços pequenos.
- **Merge sort**: ordena as metades e mescla; $\log_2 n$ níveis com n trabalho cada, $\Theta(n \log n)$ sempre, com O(n) de memória extra.
- **Quicksort**: particiona em torno de um pivô; $O(n \log n)$ em média, $O(n^2)$ com pivô ruim, muito rápido na prática por usar bem a cache.
- **Counting/Radix sort**: não comparam elementos e chegam perto de O(n), mas exigem chaves inteiras de faixa limitada.

Algoritmo	Melhor	Médio	Pior	Espaço extra	Estável?
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	O(1)	não
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	O(1)	sim
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	O(n)	sim
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	O(log n)	não
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	O(1)	não
Timsort (Python)	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	O(n)	sim

Em resumo

Nenhuma ordenação por comparações tem pior caso melhor que $\Omega(n \log n)$: há $n!$ ordens possíveis e cada comparação no máximo divide as possibilidades ao meio. Na prática, use o `sort` da biblioteca.

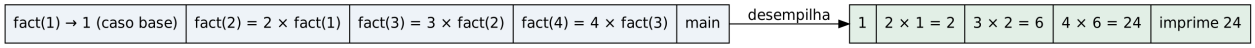
Você sabia?

Edsger Dijkstra contou que criou seu algoritmo de caminho mínimo em 1956, em cerca de vinte minutos, num café em Amsterdã, sem lápis nem papel. Ele atribuía a elegância do algoritmo justamente à falta de papel.

Recursão

> **Para começar**
> Numa fila, você descobre sua posição perguntando a de quem está à frente e somando 1, até chegar à primeira pessoa, que sabe que é a número 1. Isso é **recursão**: resolver um problema por uma versão menor dele, até um caso trivial.

Uma função **recursiva** chama a si mesma com um problema menor e tem um **caso base** que encerra a descida. O fatorial é n! = n × (n−1)!, com 1! = 1; merge sort e busca binária são recursivos por natureza. Sem caso base, a pilha estoura.



Pilha de chamadas de fact(4): cada chamada empilha um frame até o caso base; na volta, os frames são desempilhados multiplicando 1, 2, 6, 24. Fonte: elaboração própria com dados de "Mentiram pra você sobre recursão..." e "CS50 for Business - Lecture 1 - Analyzing Algorithms".

Em Mentiram pra você sobre recursão..., Augusto Galego questiona o mito de que a iteração é sempre mais eficiente. Pela tese de Church-Turing, qualquer recursão vira laço com uma pilha explícita. No assembly, o laço vira um `jmp` e a recursão um `call`, que empilha o endereço de retorno. Quando a chamada recursiva é a *última* operação, a **tail call optimization (TCO)** reaproveita o frame:

```
// não é tail call: precisa voltar para multiplicar
long fact(long n) { return n <= 1 ? 1 : n * fact(n - 1); }

// tail call: o resultado parcial viaja no acumulador
long fact_acc(long n, long acc) { return n <= 1 ? acc : fact_acc(n - 1, n * acc); }
```

TCO depende da linguagem: Scheme a garante, C e C++ a fazem na prática com otimização ligada, Python não a faz (e limita a profundidade a cerca de 1.000 chamadas), e a JVM também não. Mesmo assim, travessias de árvore, DFS e parsers ficam muito mais claros na forma recursiva.

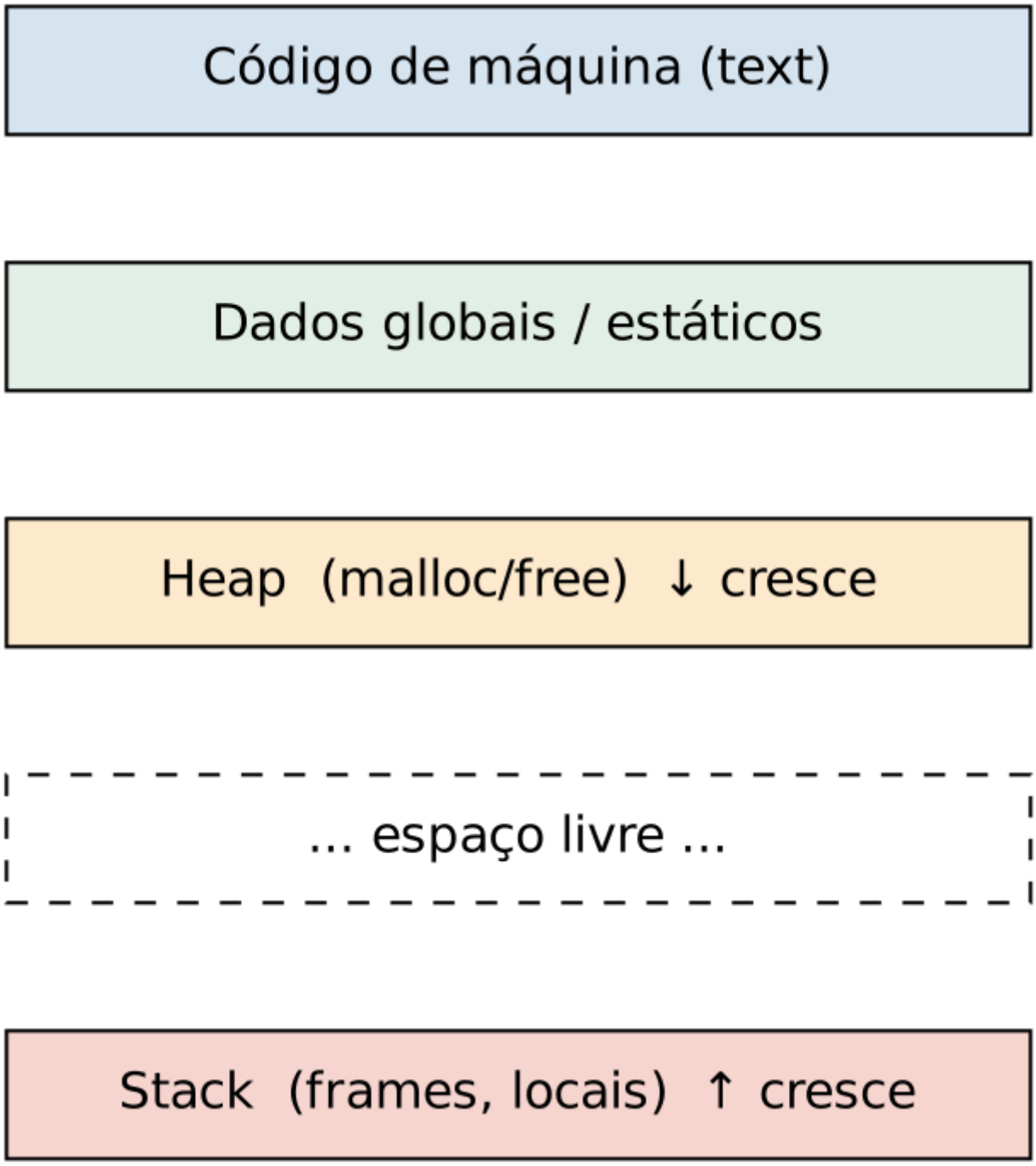
Memória, ponteiros e representação

> **Para começar**
> A memória é uma rua enorme de casas numeradas, cada uma guardando um byte. O número da casa é o **endereço**, e um **ponteiro** é um papel com um endereço anotado.

Na aula CS50x em Português - Aula 4 - Memória, endereços aparecem em **hexadecimal**, porque cada dígito hex representa exatamente 4 bits (`0xFF` = 255). Em C, `&x` dá o endereço de `x` e `*p` segue o ponteiro. Uma `string` é só `char *`: o endereço do primeiro caractere, com o fim marcado por `\0`. Daí as regras:

- `s == t` compara *endereços*; para comparar conteúdo, use `strcmp`.
- `t = s` copia o endereço, não o texto; a cópia real exige `malloc` e `strcpy`.

- `malloc` retorna `NULL` em caso de falha, e todo `malloc` precisa de um `free`. O Valgrind acusa vazamentos e escritas fora do array.



Layout clássico da memória de um processo: heap (alocação dinâmica) e stack (frames de função) crescem um em direção ao outro. Fonte: elaboração própria com dados de "CS50x em Português - Aula 4 - Memória".

A **stack** guarda variáveis locais e é liberada quando a função termina; o **heap** guarda o que foi pedido com `malloc`. Recursão profunda causa *stack overflow*, e escrever além do fim de um array causa *buffer overflow*, raiz de boa parte das vulnerabilidades históricas em C.

Valor, referência e mutabilidade

Um `swap(int a, int b)` troca cópias; para alterar as originais, a função recebe endereços (`swap(&x, &y)`). Em Finalmente aprenda PONTEIROS (C/Rust/Python), Augusto Galego mostra que em Python tudo é referência e o que muda é a **mutabilidade**: depois de `y = x; y.append(5)`, a lista `x` também mudou. Rust torna isso explícito, e o **borrow checker** proíbe, em tempo de compilação, um empréstimo mutável junto com outro imutável.

Linguagem	Modelo	Segurança de memória
C	ponteiros crus, `malloc/free`	nenhuma garantia
Python / JS / Java	referências + coletor de lixo	sem uso após free; há aliasing de mutáveis
Rust	ownership + borrow checker	garantida na compilação (fora de `unsafe`)

Ponto flutuante: por que não usar float para dinheiro

Em Por Que Você NUNCA Deve Usar FLOAT pra Representar Dinheiro no Código — Como Corrigir do Jeito Certo, Renato Augusto explica por que `0.1 + 0.2` dá `0.30000000000000004` em qualquer linguagem que siga o IEEE 754. O float é notação científica em base 2, e 1/10 em binário é uma dízima periódica, tão impossível de representar exatamente quanto 1/3 em decimal.

O caso do míssil Patriot em Dhahran (1991) é o exemplo clássico: segundo o relatório do GAO, o tempo era multiplicado por 1/10 em **ponto fixo de 24 bits**, e o erro de truncamento acumulou cerca de 0,34 s após 100 horas ligado. O míssil não interceptou o Scud, e 28 soldados morreram. O mecanismo não era IEEE 754, mas a lição é a mesma.

Representação	Precisão em decimais	Use para
float/double	aproximada (~7 e ~15 dígitos)	física, gráficos, ML, medições
inteiro em centavos	exata na menor unidade	preços, saldos
Decimal/BigDecimal	exata, com arredondamento controlado	juros, câmbio, impostos

> **Dica**
> Nunca compare floats com ==; use tolerância (`math.isclose`). Em finanças, defina por escrito o modo de arredondamento.

Cache e hierarquia de memória

A CPU lê linhas de cache de 64 bytes numa hierarquia L1 → L2 → L3 → RAM → SSD, em que cada nível é maior e mais lento. Em Cache explicado - O que é e como funciona cache?, Augusto Galego usa a analogia dos best-sellers duplicados na estante da frente e lembra o custo da duplicação: *invalidar* a cópia quando o original muda.

Nível	Latência típica
L1	~1 ns
L3	~10–20 ns
RAM	~80–100 ns
SSD NVMe	~20–100 µs
Rede intercontinental	~50–150 ms

Por isso um array contíguo costuma vencer uma lista encadeada mesmo onde a teoria favorece a lista: cada nó pode custar uma ida à RAM. O mesmo raciocínio vale para CDNs e para Redis/Memcached.

Estruturas de dados

> **Para começar**
> Livros podem ficar numa pilha no chão, em estantes em ordem alfabética ou num fichário que diz onde está cada título. Todas as formas guardam os mesmos livros, mas encontrar, acrescentar e retirar custa diferente em cada uma.

A aula CS50x em Português - Aula 5 - Estruturas de Dados separa o **tipo abstrato** (o comportamento) da **estrutura** que o implementa. Uma *fila* é FIFO, uma *pilha* é LIFO e um *dicionário* associa chaves a valores.

Arrays dinâmicos (`list` do Python, `std::vector`) **dobram** de tamanho quando enchem, em vez de crescer de 1 em 1; assim cada `append` custa O(1) amortizado, pois as cópias somam menos de 2n. **Listas encadeadas** guardam em cada nó o dado e um ponteiro para o próximo: inserir no início custa O(1), mas buscar custa O(n) e não há busca binária.

Numa **árvore de busca binária** (BST), tudo à esquerda de um nó é menor e tudo à direita é maior. Balanceada, busca em O(log n); com inserção já ordenada, degenera numa lista e volta a O(n). Árvores autobalanceadas (AVL, rubro-negra) e B-trees, que sustentam os índices de bancos de dados, evitam isso.

Tabelas hash

Uma **função hash** mapeia um domínio enorme num intervalo de índices. A tabela é um array de "baldes", e as **colisões** são tratadas por *encadeamento* (uma lista por balde) ou *endereçamento aberto*.

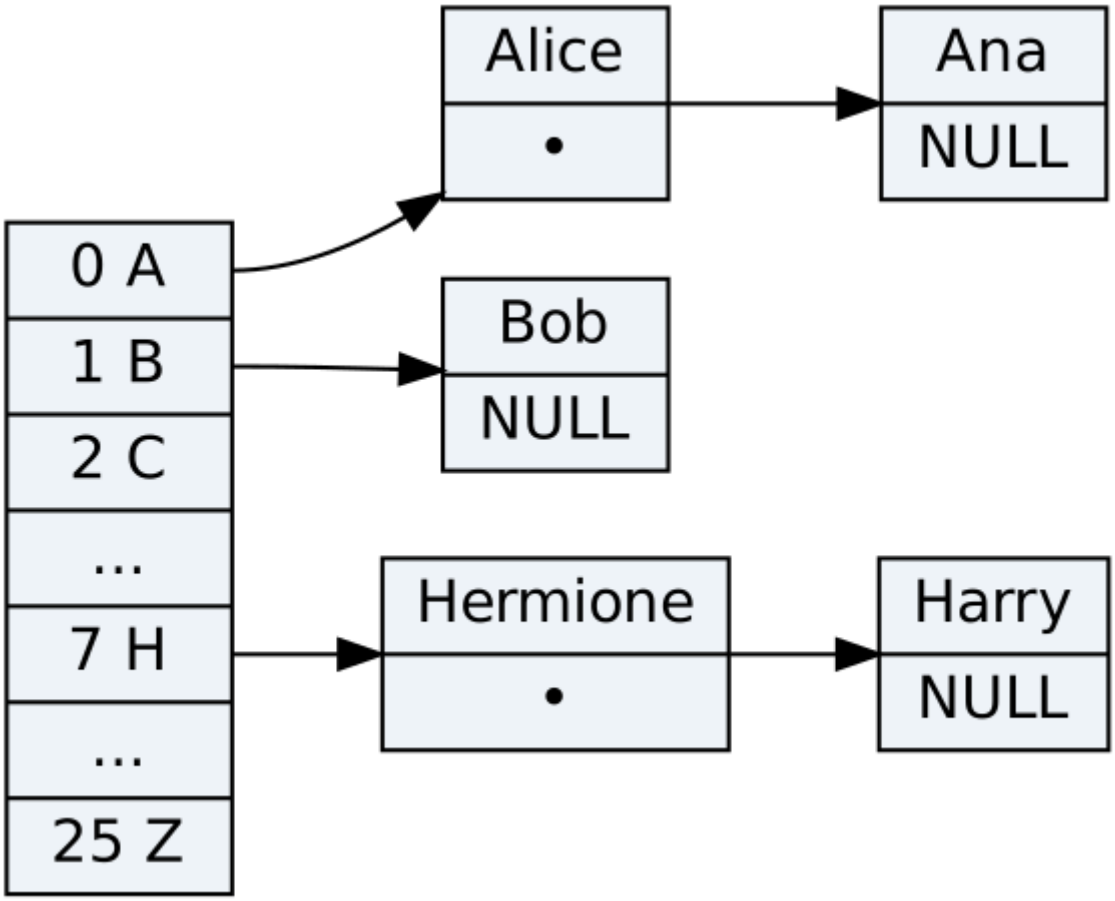


Tabela hash com encadeamento: a primeira letra escolhe o balde e as colisões viram uma lista ligada. Fonte: elaboração própria com dados de "CS50x em Português - Aula 5 - Estruturas de Dados" e "Implementando um hashmap do ZERO em Python".

Em Implementando um hashmap do ZERO em Python, Augusto Galego conta que implementar um hash map do zero já foi pedido em entrevista da Amazon. Versão didática:

```
class HashMap:
    def __init__(self, size=8):
        self.size, self.count = size, 0
        self.buckets = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def put(self, key, value):
        b = self.buckets[self._hash(key)]
        for i, (k, _) in enumerate(b):
            if k == key:
                b[i] = (key, value); return
        b.append((key, value))
        self.count += 1
        if self.count / self.size > 0.75:   # fator de carga
            self._resize(self.size * 2)

    def get(self, key):
        for k, v in self.buckets[self._hash(key)]:
            if k == key: return v
        raise KeyError(key)

    def _resize(self, new_size):
        old = [kv for b in self.buckets for kv in b]
        self.size, self.count = new_size, 0
        self.buckets = [[] for _ in range(new_size)]
        for k, v in old: self.put(k, v)
```

O ponto central é o **fator de carga** $\alpha = n/k$. Mantendo α limitado com redimensionamento, inserção e busca ficam em O(1) *em média*; o pior caso continua O(n). Linguagens como Python e Rust usam hash com semente aleatória (SipHash) para impedir que um atacante force colisões (*hash flooding*).

Tries e grafos

Uma **trie** é uma árvore em que cada nível corresponde a um caractere; a busca custa O(L), sendo L o comprimento da palavra, independentemente de quantas palavras existem. Variantes compactas (radix tree) aparecem em roteamento IP e autocompletar.

Um **grafo** generaliza árvores: vértices ligados por arestas com ou sem direção e peso. Os algoritmos essenciais são **BFS** (menor caminho em número de arestas), **DFS** (ciclos, ordenação topológica de dependências), **Dijkstra** (menor caminho com pesos não negativos, O((V+E) log V)) e **A*** (Dijkstra guiado por heurística, usado em jogos e GPS).

Estrutura	Busca	Inserção	Observações
Array ordenado	O(log n)	O(n)	contíguo, ótimo para cache
Array dinâmico	O(n)	O(1) amortizado no fim	sobra de capacidade
Lista encadeada	O(n)	O(1) na cabeça	ruim para cache
BST balanceada	O(log n)	O(log n)	mantém ordem
Heap binário	O(n)	O(log n)	fila de prioridade
Tabela hash	O(1) médio	O(1) médio	sem ordem
Trie	O(L)	O(L)	busca por prefixo

Em resumo

Toda escolha de estrutura troca tempo, espaço e complexidade de código. Saber que `x in lista` é O(n) e `x in conjunto` é O(1) é a otimização mais barata que existe.

De C a Python: tempo humano contra tempo de máquina

A aula CS50x em Português - Aula 6 - Python refaz em Python os exercícios feitos em C. O corretor ortográfico, que em C exige escrever a tabela hash à mão, cabe em cerca de 20 linhas com `set()` e rodou em 1,87 s contra 1,32 s em C: menos tempo humano, mais tempo de CPU. O preço da conveniência aparece na memória: cada `int` do CPython é um objeto de 28 bytes no heap.

Conceito	C	Python
Inteiros	tamanho fixo, overflow	precisão arbitrária
Divisão	`/` entre ints trunca	`/` dá float, `//` é inteira
Strings	`char *`, `strcmp`	`str` imutável, `==`
Erros	valor de retorno (`NULL`, `−1`)	exceções
Memória	`malloc/free`	contagem de referências + GC

Você sabia?

Donald Knuth começou "The Art of Computer Programming" em 1962 e a obra ainda não terminou. Insatisfeito com a tipografia de uma reedição, ele parou para criar o TeX, usado até hoje em artigos científicos.

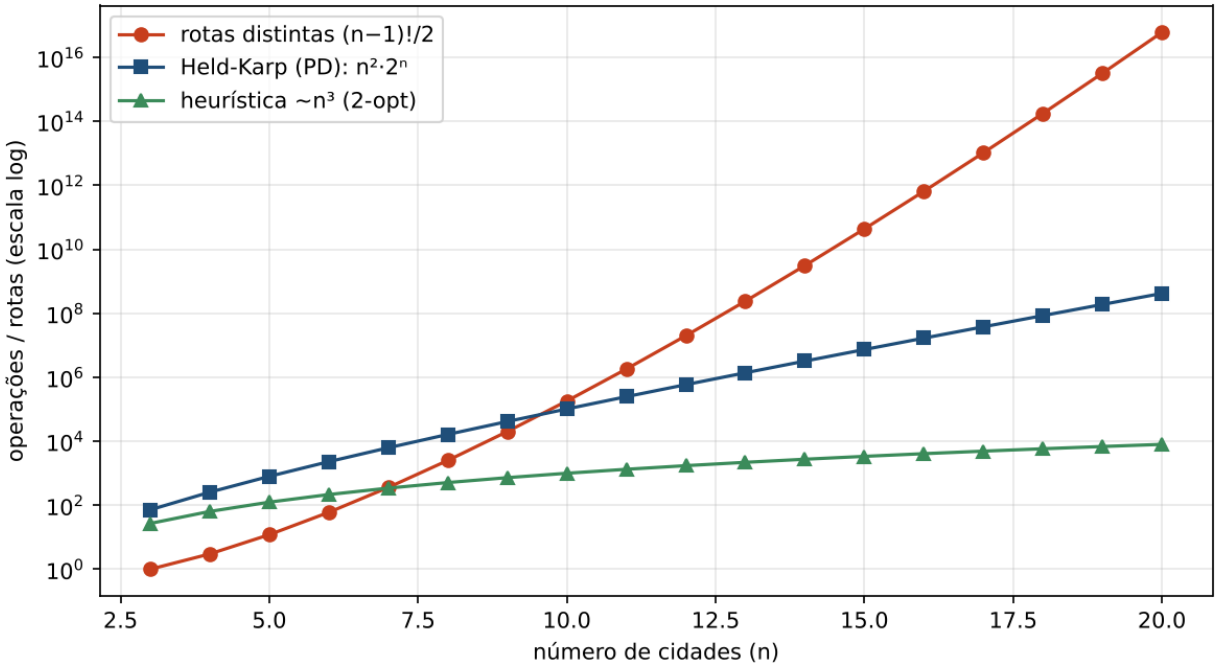
Problemas difíceis: NP, caixeiro-viajante e restrições

Para começar

Conferir um sudoku preenchido leva um minuto; resolvê-lo do zero pode levar uma hora. Muitos problemas práticos, como rotas de entrega e escalas de plantão, têm essa cara: fácil checar, difícil encontrar.

A pergunta P = NP, um dos Problemas do Milênio do Clay Institute (prêmio de US$ 1 milhão), pergunta se todo problema fácil de *verificar* também é fácil de *resolver*. Um problema é **NP-completo** se está em NP e todo problema de NP se reduz a ele em tempo polinomial. Cook e Levin (1971) mostraram que SAT é NP-completo, e Karp (1972) listou outros 21, como a mochila e a coloração de grafos. Se um deles tiver algoritmo polinomial, todos terão.

No **problema do caixeiro-viajante (TSP)**, busca-se o ciclo mais curto que passa por n cidades. Por força bruta há $(n-1)!/2$ rotas: 12 com 5 cidades, cerca de 6×10^{16} com 20.



Número de rotas do TSP por força bruta comparado à programação dinâmica de Held-Karp ($n^2 \cdot 2^n$) e a uma heurística polinomial típica (2-opt). Fonte: elaboração própria; tema de "A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3".

No episódio A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3, Amir Husain resolve com uma GPU barata o TSP, uma escala de equipe médica por *satisfação de restrições* e a formação de times balanceados. O argumento é que a saída de um solver é **determinística e verificável**, ao contrário de pedir a escala a um LLM; um agente pode delegar o problema ao solver.

Estratégia	Garante ótimo?	Exemplo
Força bruta	sim	testar todas as rotas; só até 10–12 cidades
Programação dinâmica	sim	anotar trechos já calculados; cerca de 25 cidades
Branch and bound	sim (exponencial no pior caso)	descartar cedo rotas que já perdem; milhares de cidades
Aproximação	fator conhecido	rota no máximo 1,5 vez maior que a melhor
Heurísticas	não	ir sempre à cidade mais próxima
Solvers genéricos	depende	programa pronto que recebe as regras e busca a solução

Antes do solver vem o **modelo matemático**. Em Unlock the Power of Data-Driven Decision Making with Decision Optimization, a IBM usa uma barraca de limonada: variáveis de decisão (preço, quantidade), um objetivo (lucro) e restrições (orçamento). Uma escala de enfermagem segue o mesmo formato, com variáveis booleanas "a pessoa p trabalha no turno t":

```
# esboço com OR-Tools CP-SAT
from ortools.sat.python import cp_model
m = cp_model.CpModel()
x = {(p, t): m.NewBoolVar(f"x_{p}_{t}") for p in people for t in shifts}
for t in shifts:
    m.Add(sum(x[p, t] for p in nurses) >= MIN_ENF)    # cobertura mínima
for p in people:
    m.Add(sum(x[p, t] for t in shifts) <= MAX_TURNOS)      # carga máxima
solver = cp_model.CpSolver(); solver.Solve(m)
```

Programação dinâmica

> **Para começar**
>
> Se você acabou de calcular 1+1+1+1+1 = 5 e alguém pede mais um "+1", você não recomeça: soma 1 ao 5. **Programação dinâmica** é anotar respostas de subproblemas para não recalculá-las.

A PD, termo de Richard Bellman, se aplica quando há **subestrutura ótima** e **subproblemas sobrepostos**. O Fibonacci recursivo ingênuo faz cerca de $1{,}618^n$ chamadas (mais de 29 milhões para n = 35); com memoização, faz n.

```
from functools import lru_cache

@lru_cache(maxsize=None)        # top-down: memoização
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def fib_tab(n):                 # bottom-up: tabulação, O(1) de espaço
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Roteiro: defina o **estado** (ex.: dp[i][w] = melhor valor com os i primeiros itens e capacidade w), escreva a **transição** (na mochila 0/1, dp[i][w] = max(dp[i−1][w], dp[i−1][w−p_i] + v_i)), fixe os **casos base** e calcule o custo como estados × custo por transição.

Problema clássico	Estado	Complexidade
Troco mínimo	dp[valor]	O(n·moedas)
Mochila 0/1	dp[i][capacidade]	O(n·W), pseudo-polinomial
Distância de edição, LCS (`diff`)	dp[i][j]	O(n·m)
TSP (Held-Karp)	dp[subconjunto][último]	$O(n^2 \cdot 2^n)$

A mochila em O(n·W) não prova P = NP: o custo é polinomial no *valor* de W, não no número de bits que o representam.

LeetCode e entrevistas técnicas

> **Para começar**
> Entrevistas técnicas funcionam como uma prova prática de cozinha: um problema curto e alguém observando como você pensa. O **LeetCode** reúne milhares desses problemas para treinar.

Two Sum: a resposta depende do contexto

```
def two_sum(nums, target):        # O(n) tempo, O(n) espaço
    seen = {}                     # valor -> índice
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
```

A força bruta ($O(n^2)$ de tempo, $O(1)$ de espaço) não está "errada": perde em escala e ganha em memória e em entradas minúsculas. O conselho de Augusto Galego é **perguntar ao entrevistador** o tamanho e o padrão dos dados; com o array ordenado, dois ponteiros resolvem em $O(n)$ de tempo e $O(1)$ de espaço.

Single Number e off-by-one

No problema tratado em LeetCode 136 | Explicação em Português, todo número aparece duas vezes exceto um, com espaço $O(1)$. Como $x \oplus x = 0$ e $x \oplus 0 = x$, aplicar XOR em todos anula os pares e sobra o número sem par.

```
from functools import reduce
from operator import xor
def single_number(nums):
    return reduce(xor, nums)      # O(n) tempo, O(1) espaço
```

Em LeetCode: O erro mais comum, o tema é o erro de "off-by-one". A pergunta a fazer sempre é "o intervalo é inclusivo ou exclusivo, e começa em 0 ou em 1?". Em Python, `range(a, b)` e fatias excluem o fim, mas `random.randint(a, b)` o inclui. Uma janela de tamanho k num array de tamanho n tem `n - k + 1` posições. Dijkstra defendia intervalos **semiabertos** [a, b): o tamanho é $b - a$ e intervalos adjacentes se encaixam sem sobreposição.

Padrão	Sinal no enunciado	Ferramenta
Hash map / set	"já vi este valor?", contagens	`dict`, `Counter`
Dois ponteiros	array ordenado, palíndromo	índices nas duas pontas
Janela deslizante	"maior subarray contíguo com..."	expandir e contrair
Busca binária	espaço ordenado ou monotônico	`bisect`
BFS / DFS	grade, árvore, dependências	fila / recursão
Heap	"k maiores", agendamento	`heapq`
Programação dinâmica	"quantas formas", mínimo/máximo	tabela ou memo

> **Dica: roteiro de entrevista**
> Repita o problema e pergunte os limites; diga a força bruta e seu custo; proponha a otimização e o trade-off; programe falando; teste os casos de fronteira; diga a complexidade final de tempo e espaço.

Com IA gerando código, ainda vale estudar isso? Em Como Browsers Funcionam?, Augusto Galego pediu a um LLM que "criasse um browser" e recebeu só um invólucro de um componente pronto; a versão "avançada" nem rodou. Julgar se uma solução está certa, se escala e onde vai quebrar continua exigindo fundamentos.

Concorrência: o custo que Big-O não mostra

Em Concorrência é DIFERENTE de Paralelismo!, Augusto Galego separa **concorrência** (intercalar tarefas, ilusão de simultaneidade), **paralelismo** (executar ao mesmo tempo em vários núcleos) e **assincronia** (não bloquear à espera de um resultado). Três `await fetch` independentes em sequência desperdiçam tempo; `await Promise.all([...])` os torna concorrentes. Pela lei de Amdahl, se uma fração s do trabalho é sequencial, o ganho máximo com infinitos núcleos é 1/s.

Provas formais: quando testar não basta

> **Para começar**
> Testar uma ponte com dez caminhões mostra que ela aguentou aqueles dez; o cálculo estrutural garante qualquer carga dentro da especificação. Em software, o equivalente é a **prova formal**, conferida por um programa.

Testes mostram a presença de bugs, nunca a ausência (Dijkstra). No post "Por que não fazemos provas formais definitivas pra ter código sem bug?", Fabio Akita discute a **verificação formal** no contexto dos agentes de IA. O exemplo canônico do custo é o microkernel **seL4**: cerca de 8.700 linhas de C exigiram cerca de 200 mil linhas de prova em Isabelle/HOL e cerca de 20 pessoas-ano (20 pessoas trabalhando um ano). Em assistentes como Lean 4 e Coq, um programa que compila é uma prova da proposição expressa no seu tipo (correspondência de Curry-Howard).

A virada vem das **provas geradas por IA**: um LLM pode alucinar, mas não engana um verificador. Se o Lean aceita a prova, ela vale. Escrever a prova, a parte cara, passa a ser automatizável; o humano fica com escrever a *especificação* certa.

```
-- Lean 4: a prova é verificada pelo compilador
theorem add_comm' (a b : Nat) : a + b = b + a := by
  omega
```

> **Em resumo**
> Solvers de restrição, verificadores de prova e verificadores de tipos seguem o mesmo princípio: o resultado pode ser checado mecanicamente. Com agentes gerando código, o valor de um componente depende cada vez mais de quão barato é *verificar* o que ele produz.

Você sabia?

Durante o pouso da Apollo 11, alarmes 1202 e 1201 indicaram sobrecarga do computador de bordo. O software da equipe de Margaret Hamilton, no MIT, descartou tarefas de baixa prioridade e manteve as essenciais, e o pouso seguiu.

Fontes deste capítulo

Título	URL	Canal / autor	Ponto principal
CS50x em Português - Aula 1 - C	https://www.youtube.com/watch?v=eepzDoovmAw	CS50	Compilação, qualidade de código, overflow
CS50x em Português - Aula 3 - Algoritmos	https://www.youtube.com/watch?v=1dV61IMUwIc	CS50	Buscas, O/Ω/Θ, ordenação, recursão
CS50x em Português - Aula 4 - Memória	https://www.youtube.com/watch?v=DweS4KseeDE	CS50	Hexadecimal, ponteiros, malloc/free, heap/stack
CS50x em Português - Aula 5 - Estruturas de Dados	https://www.youtube.com/watch?v=pttqNxchp9k	CS50	Listas, árvores, tabelas hash e tries
CS50x em Português - Aula 6 - Python	https://www.youtube.com/watch?v=ilG8j9kWIlY	CS50	C versus Python: tempo humano e de máquina
CS50 for Business - Lecture 1 - Analyzing Algorithms	https://www.youtube.com/watch?v=fFzWME39Mag	CS50	Definição de algoritmo e classes de complexidade
Big O Notation fácil de entender! (Complexidade de Tempo e Espaço na Programação)	https://www.youtube.com/watch?v=FR44uWofQ7o	Attekita Dev	Complexidade de tempo e espaço
Entenda FUNDAMENTOS do CURSO de CIÊNCIA da COMPUTAÇÃO	https://www.youtube.com/watch?v=R1IivfF7cu8	Fernanda Kipper	Máquina de Turing, complexidade e criptografia
Mentiram pra você sobre recursão...	https://www.youtube.com/watch?v=5Zmc6f420rg	Augusto Galego	Pilha de chamadas, Church-Turing e TCO
Finalmente aprenda PONTEIROS (C/Rust/Python)	https://www.youtube.com/watch?v=8Pkvm8r3nUQ	Augusto Galego	Mutabilidade, aliasing e borrow checker
Por Que Você NUNCA Deve Usar FLOAT pra Representar Dinheiro no Código — Como Corrigir do Jeito Certo	https://www.youtube.com/watch?v=vFBUWtrzz48	Renato Augusto	IEEE 754, centavos inteiros e Decimal
Cache explicado - O que é e como funciona cache?	https://www.youtube.com/watch?v=r-tiD2MYnWE	Augusto Galego	Hierarquia de cache e invalidação
Implementando um hashmap do ZERO em Python	https://www.youtube.com/watch?v=J4ELMYEGVS0	Augusto Galego	Colisões, fator de carga e resize
Quando um algoritmo ruim é melhor do que um bom...	https://www.youtube.com/watch?v=2ObVc2QvZG0	Augusto Galego	Big-O mede escalabilidade, não velocidade
LeetCode 136 | Explicação em Português	https://www.youtube.com/watch?v=mVcGSzGxrYM	Augusto Galego	Single Number com XOR
LeetCode: O erro mais comum	https://www.youtube.com/watch?v=EQfcAeR2R6I	Augusto Galego	Off-by-one e intervalos inclusivos/exclusivos
Concorrência é DIFERENTE de Paralelismo!	https://www.youtube.com/watch?v=74RLgpAV6LQ	Augusto Galego	Concorrência, paralelismo e assincronia
Unlock the Power of Data-Driven Decision Making with Decision Optimization	https://www.youtube.com/watch?v=jhSejeNslP8	IBM Technology	Modelagem: variáveis, objetivo, restrições, solvers
Como Browsers Funcionam?	https://www.youtube.com/watch?v=fv_B3FTXwxo	Augusto Galego	IA não substitui os fundamentos
A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3	https://www.youtube.com/watch?v=2ILc4WY86us	Deep Tech with Amir Husain	TSP e escalas resolvidos por solver determinístico
Por que não fazemos provas formais definitivas pra ter código sem bug? (24/10/2026)	https://akitaonrails.com/2026/10/24/por-que-nao-fazemos-provas-formais-definitivas-de-codigo-sem-bug/	Fabio Akita (blog AkitaOnRails)	Custo do seL4 e provas geradas por IA
Patriot Missile Defense: Software Problem Led to System Failure at Dhahran, Saudi Arabia (GAO/IMTEC-92-26)	https://www.gao.gov/products/imtec-92-26	U.S. General Accounting Office	Erro de truncamento em ponto fixo

CAPÍTULO 3 ·

Dados, Estatística e Sistemas de Recomendação

> *Todo modelo de IA é uma função dos dados que o alimentam. Este capítulo percorre a cadeia completa: guardar e consultar dados (SQL, NoSQL), movê-los com segurança (pipelines, batch, streaming), resumi-los (estatística, PCA, anomalias), transformar probabilidades em decisões (classificadores calibrados, cascatas) e, por fim, o artefato de dados mais lucrativo da internet: o sistema de recomendação.*

Para começar

Numa cozinha de restaurante, alguém compra, guarda, lava e descarta ingredientes antes de qualquer prato sair. A receita (o modelo) importa, mas nada salva um prato feito com ingrediente ruim. Este capítulo é sobre essa cozinha dos dados.

Por que dados vêm antes de modelos

A maior parte do ciclo de vida de um projeto de IA não é treinar modelos, e sim coletar, limpar, documentar e mover dados. O vídeo Big Data Rules For AI: Essential Data Management Principles, da IBM Technology, transforma essa constatação em objetivo de engenharia: reduzir esse tempo com padronização e automação. Em LLM + Data: Building AI with Real & Synthetic Data, o mesmo canal lembra que cada decisão de curadoria (quais categorias, quais exemplos, como rotular) define *quem* é representado. Escala não garante diversidade nem qualidade.

A frase "*data is the new oil*", atribuída a Clive Humby (2006), é útil num ponto pouco lembrado: petróleo bruto não serve sem refino, e dados crus também não.

Bancos de dados relacionais e SQL

Para começar

Uma planilha já é quase um banco de dados: linhas são registros, colunas são atributos. Um **banco relacional** é uma coleção de tabelas que se referenciam, com regras que impedem bagunça, e o **SQL** é a linguagem em que você descreve o que quer e o banco descobre como buscar.

Do arquivo plano ao SQL

A aula CS50x em Português - Aula 7 - SQL parte de um CSV com respostas de um formulário. Um CSV é um **banco de arquivo plano**: universal e legível, mas cada pergunta exige um programa. Contar votos em Python leva várias linhas; em SQL, uma consulta:

```
SELECT language, COUNT(*) AS n
FROM favorites
GROUP BY language
ORDER BY n DESC;
```

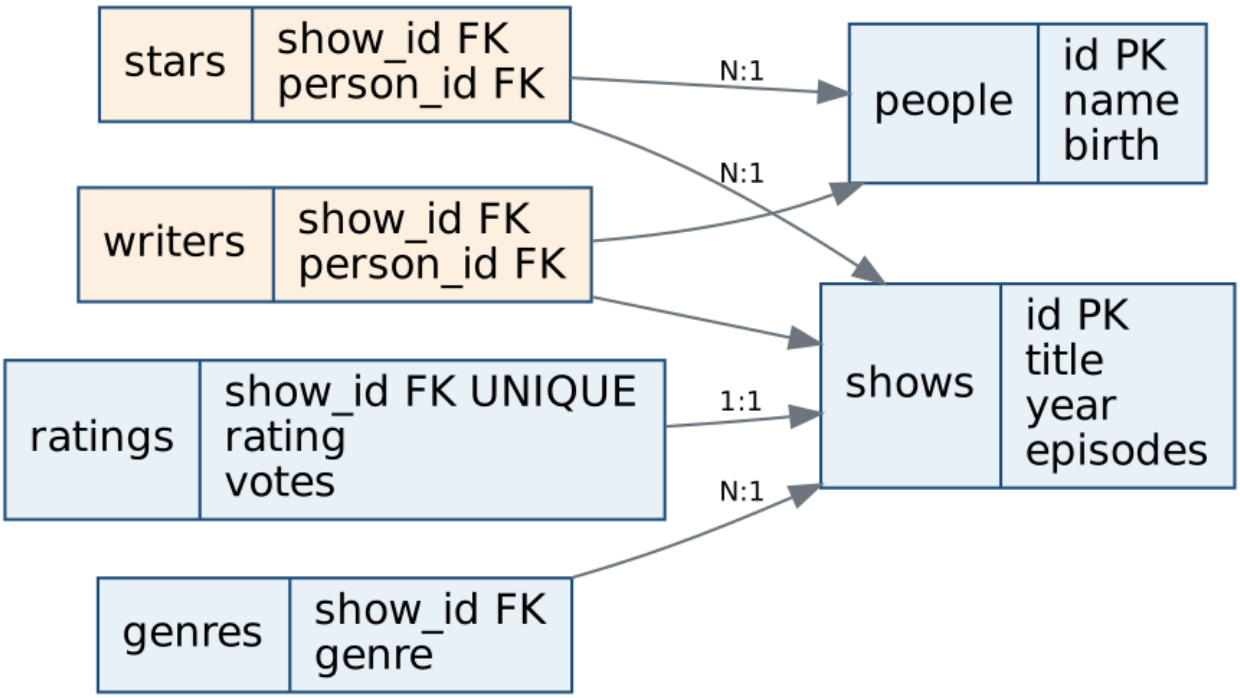
Python é **imperativo** (você diz *como* calcular); SQL é **declarativo** (você diz *o que* quer, e o otimizador escolhe índices, ordem de junção e algoritmos). Por isso a mesma consulta funciona sobre 300 linhas ou 300 milhões. As operações básicas formam o acrônimo **CRUD**: `INSERT`, `SELECT`, `UPDATE`, `DELETE`. O CS50 usa o SQLite, um banco embutido num único arquivo, presente em todo celular e navegador.

Construção	Para que serve
`WHERE`, `LIKE`	Filtrar linhas, casar padrões
`COUNT`, `AVG`, `MAX`, `DISTINCT`	Agregar
`GROUP BY` / `HAVING`	Agregar por grupo / filtrar grupos
`ORDER BY`, `LIMIT`	Ordenar e cortar
`IS NULL`	Testar ausência de valor

`NULL` não é igual a nada, nem a si mesmo; por isso existe `IS NULL`. E `DELETE` ou `UPDATE` sem `WHERE` afeta a tabela inteira.

Modelagem: chaves e normalização

A aula modela um subconjunto do IMDb com cerca de 250 mil séries. Uma planilha com colunas "ator1, ator2, ator3..." é o anti-padrão clássico. A solução é **normalizar**: cada entidade numa tabela, com **chave primária**, e as relações expressas por **chaves estrangeiras**. As formas normais de Codd formalizam a ideia de "cada fato em um único lugar", o que evita anomalias de atualização.



Esquema relacional do exemplo IMDb do CS50: `stars` e `writers` são tabelas de junção (muitos-para-muitos), `ratings` é um-para-um e `genres` é um-para-muitos. Fonte: elaboração própria com dados de "CS50x em Português - Aula 7 - SQL".

Para cruzar tabelas, o **JOIN** é a forma idiomática:

```
SELECT shows.title
FROM shows
JOIN stars  ON stars.show_id  = shows.id
JOIN people ON people.id      = stars.person_id
WHERE people.name = 'Steve Carell';
```

Sem índice, buscar um título exige varrer a tabela, em O(n). Com `CREATE INDEX`, o banco mantém uma **B-tree** (árvore larga e rasa, feita para ler blocos de disco) e a busca cai para O(log n); na aula, a consulta ficou cerca de 42 vezes mais rápida. Índices ocupam espaço e deixam escritas mais lentas, então indexe as colunas usadas com frequência em `WHERE`, `JOIN` e `ORDER BY`, e confira o plano com `EXPLAIN`.

Injeção de SQL e concorrência

Regra de ouro: **nunca** monte SQL com concatenação de strings; use *placeholders* (`?`). Num login como `malan@harvard.edu'--`, o apóstrofo fecha a string e o `--` comenta o resto da consulta, inclusive a verificação de senha. A defesa é estrutural: consultas parametrizadas e menor privilégio.

O outro problema clássico é a **condição de corrida**: dois servidores leem "100 curtidas", somam 1 e gravam 101, e uma curtida se perde. As soluções são operações atômicas (`UPDATE posts SET likes = likes + 1`), locks e **transações**, que garantem as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade).

Você sabia?

O modelo relacional foi proposto por Edgar F. Codd, da IBM, em 1970, mas a própria IBM demorou a adotá-lo. O primeiro banco SQL comercial veio da Relational Software, futura Oracle, em 1979.

NoSQL: "not only SQL"

Para começar

Uma loja que vende TVs, camisetas e livros tem produtos com características bem diferentes. Em vez de uma tabela cheia de colunas vazias, bancos **de documentos** guardam cada registro como um "cartão" flexível em JSON.

O vídeo Master NoSQL: Scalable Databases for Modern Applications, da IBM, desfaz um mito: NoSQL significa "*not only SQL*", e bancos de documentos também têm índices e junções. O princípio é **modelar para o padrão de acesso**: gastar mais lógica na escrita para tornar a leitura barata.

Cenário	Padrão em documentos
Loja que vende TVs e camisetas	Campos comuns a todos e uma parte livre para cada tipo
Termômetro que mede a cada segundo	Juntar as leituras de cada hora num só documento
Post com comentários	Guardar os primeiros comentários dentro do post; o resto, à parte
Curtidas de um post	Somar 1 ao total a cada curtida, em vez de contar tudo de novo

É o oposto da normalização relacional, que otimiza integridade e flexibilidade de consulta. Se você ainda não sabe quais consultas fará, comece relacional (o Postgres tem colunas `JSONB` indexáveis) e migre quando a junção for o gargalo medido.

Movendo dados: pipelines, batch e streaming

Para começar

Dados raramente nascem onde serão usados. O caminho automatizado que os leva de um lugar a outro, limpando no trajeto, é o **pipeline**. O estilo **batch** é como o caminhão de lixo que passa uma vez por dia; o **streaming** é a água correndo no cano o tempo todo.

ETL resiliente

Um pipeline ETL (*extract, transform, load*) move dados de A para B. Em Scaling Data Pipelines: Memory Optimization & Failure Control, a IBM parte de um caso realista: o tráfego triplica e o pipeline estoura a memória. As práticas para memória são ler e escrever em blocos (*chunking*), converter strings repetitivas para o tipo `category` e trocar laços por operações vetorizadas. Para falhas, a mentalidade é "o pipeline *vai* falhar": validar o schema na entrada, repetir cada fase algumas vezes antes de desistir e gravar um **checkpoint** fora do pipeline para retomar do ponto onde parou.

```
import pandas as pd
ckpt = load_checkpoint()            # ex.: último id gravado externamente
for chunk in pd.read_csv("sales.csv", chunksize=200_000,
                         dtype={"region": "category"}):
    chunk = chunk[chunk.id > ckpt]
    chunk = validate_schema(chunk)   # descarta linhas inválidas cedo
    agg = chunk.groupby("product", observed=True)["amount"].sum()
    with_retry(lambda: write(agg), tries=3)
    save_checkpoint(chunk.id.max())
```

Em resumo

Pipelines robustos são **idempotentes** (rodar duas vezes dá o mesmo resultado), **retomáveis** e **defensivos**. Polars, DuckDB ou Spark resolvem problemas de memória, mas não substituem esses princípios.

Big data, fast data e streaming

Em Big Data vs Fast Data: Optimize Your AI Strategy, a pergunta-chave é se você otimiza para profundidade ou para velocidade. Big data extrai valor da **agregação ao longo do tempo** (treinar modelos, histórico); fast data extrai valor **no momento** (fraude, personalização). Os dois coexistem: **modelos são treinados com big data e aplicados a fast data**.

	Big data	Fast data
Chegada dos dados	Batch (em lotes, de tempos em tempos)	Streaming (contínuo)
Processamento	Grandes cálculos agendados	Cada evento analisado ao chegar
Uso	Relatórios, treino de IA	Alertas, decisões automáticas
Exemplo	Relatório de vendas do mês	Bloquear na hora um cartão roubado

Em What Is Real-Time Data Streaming? AI & Machine Learning Applications, a tese central é que **o valor do dado decai com o tempo**. O processador filtra, **enriquece** (junta ao registro qual máquina, onde, em que operação) e analisa; e não convém ser um "acumulador de dados": persista anomalias e pontos de interesse, não cada leitura bruta. Conceitos essenciais de streaming incluem tempo de evento versus tempo de processamento, janelas e semântica de entrega (*at-least-once*, *exactly-once*).

O Apache Hadoop, tema de What is Apache Hadoop?, popularizou o processamento distribuído em hardware comum, com HDFS (blocos replicados), YARN (recursos) e MapReduce (processamento). O MapReduce grava resultados intermediários em disco; o Spark os mantém em memória e é bem mais rápido em ML iterativo. Hoje o HDFS muitas vezes deu lugar a object storage com Spark ou motores SQL, mas a ideia de processar onde o dado está e tolerar falhas de nós permanece.

Governança e produtos de dados

O vídeo sobre regras de big data para IA propõe quatro salvaguardas: **documentação padronizada** (tipo do dado, dono, retenção), **ingestões automatizadas** e implantadas como código, **rastreamento de mudanças** com alertas e **marcação** de qual dado foi combinado com qual modelo. Em RAG, marque os dados *antes* de vetorizar. Em Data Products Explained: Key Characteristics & Benefits, a IBM usa a analogia do supermercado

para o **produto de dados**: reutilizável, pertencente a um domínio, bem "embalado", com controle de acesso, SLA de qualidade e vários canais de entrega. É a ideia central do *data mesh*.

Estatística descritiva: resumir antes de modelar

> **Para começar**
> Ninguém lê uma planilha de 100 mil linhas. Resumos como "a idade média é 34" ou "metade das compras fica abaixo de R$ 80" são o primeiro passo diante de qualquer dado novo, como o médico que mede a pressão antes de pedir exames.

O "Data Desk" mostrado em A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3 ilustra o fluxo exploratório: carrega-se uma planilha e a ferramenta calcula estatísticas descritivas, frequências, correlações, PCA, agrupamentos e anomalias. É o roteiro que vale para qualquer dataset novo.

Medida	O que é, num exemplo
Média	Soma dividida pelo total; um bilionário na sala distorce a média salarial
Mediana	O valor do meio; prefira para salários e preços
Moda	O valor mais comum, como o tamanho de camiseta mais vendido
Desvio padrão	Quanto os valores se espalham; notas 7, 7 e 7 têm desvio zero
Quartis, IQR	Dividem os dados em quatro partes iguais; base do boxplot
Percentis (p95, p99)	p95: 95% das páginas abrem mais rápido que esse tempo

A correlação de Pearson mede associação *linear* entre -1 e 1; a de Spearman, aplicada aos postos, capta relações monotônicas. Correlação não é causalidade, $r \approx 0$ não significa independência (uma parábola tem $r \approx 0$), e duas séries temporais com tendência quase sempre parecem correlacionadas: correlacione as *variações*, não os níveis.

> **Atenção**
> O quarteto de Anscombe (1973) tem quatro conjuntos com médias, variâncias e correlação idênticas e gráficos completamente diferentes. Sempre plote os dados.

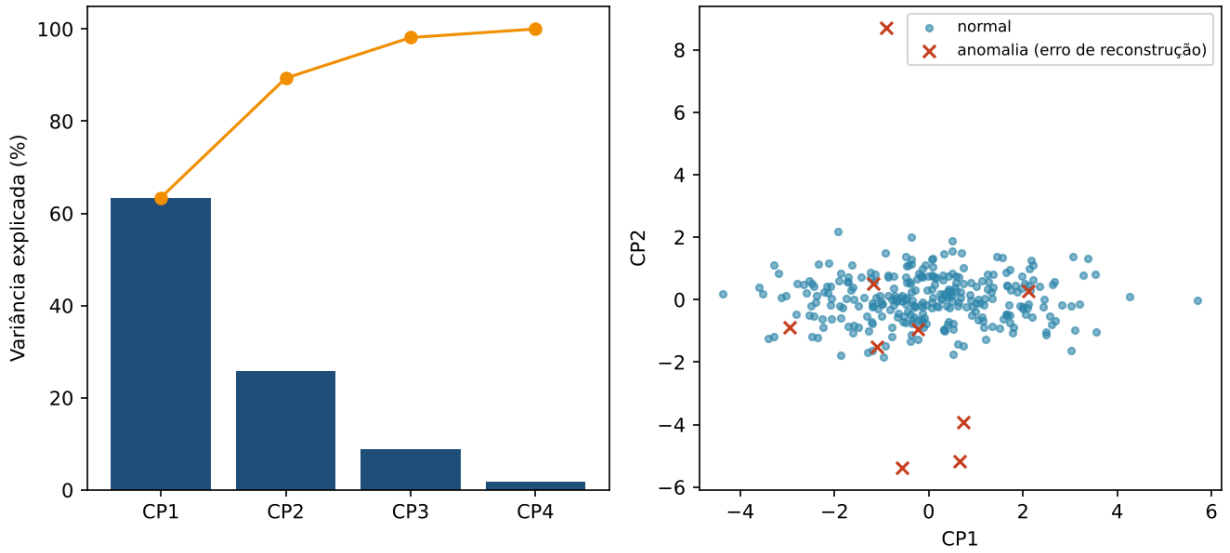
PCA: enxergando a estrutura

> **Para começar**
> A sombra de uma mão na parede transforma 3D em 2D e, com a lanterna no lugar certo, ainda revela a forma. A **PCA** procura esse "ângulo de lanterna" para reduzir dezenas de colunas a duas ou três que dá para desenhar.

A Análise de Componentes Principais encontra direções (combinações lineares das variáveis) ordenadas pela variância que capturam: o CP1 é a de maior variância, o CP2 a maior ortogonal ao CP1, e assim por diante. O algoritmo: **padronizar** cada coluna (senão a

de maior escala domina), calcular a decomposição em valores singulares, medir a **variância explicada** de cada componente e projetar nos k primeiros.

```
import numpy as np
Xs = (X - X.mean(0)) / X.std(0)
U, S, Vt = np.linalg.svd(Xs, full_matrices=False)
explained = S**2 / (S**2).sum()
Z = Xs @ Vt[:2].T                  # coordenadas em CP1, CP2
recon = Z @ Vt[:2]                 # volta ao espaço original
error = ((Xs - recon)**2).sum(1)    # erro de reconstrução por linha
```



Esquerda: variância explicada por componente num conjunto sintético de 4 variáveis correlacionadas. Direita: projeção em CP1×CP2, com os pontos de maior erro de reconstrução marcados como anomalias. Fonte: elaboração própria, ilustrando o fluxo do "Data Desk" do VectorFall Ep. 3.

A PCA é linear e sensível a outliers. Para estruturas não lineares, t-SNE e UMAP visualizam melhor, mas distorcem distâncias globais. Para agrupar, k-means após PCA é um combo clássico. E a fatoração de matrizes dos recomendadores é, essencialmente, uma SVD truncada.

Detecção de anomalias

> **Para começar**
>
> Uma **anomalia** é um dado que destoa do resto, como uma compra de R$ 30 mil num cartão que gasta R$ 200. Detectá-las é como o porteiro que conhece os moradores e repara no rosto estranho.

Família	Método	Exemplo do dia a dia
Estatística	Z-score robusto, regra de Tukey	Conta de luz dez vezes maior que a de costume
Distância/densidade	Mahalanobis, LOF	Carro potente e levíssimo: cada dado é normal, a combinação não
Reconstrução	PCA, autoencoder	Peça que não se encaixa no padrão do quebra-cabeça
Isolamento	Isolation Forest	Quem está sozinho num canto da festa é achado logo
Séries temporais	Resíduo de previsão, EWMA	Loja vazia numa sexta-feira que costuma lotar

O z-score clássico tem um defeito: o próprio outlier infla a média e o desvio, mascarando-se. A versão robusta usa mediana e MAD. A distância de Mahalanobis considera correlações: um carro potente e pesado é normal; potente e levíssimo, não. Lembre que "anomalia" é estatística e "problema" é de negócio: a Black Friday é anômala e legítima.

Da probabilidade à decisão: classificadores calibrados

Detectores produzem *pontuações*; o negócio precisa de *decisões*. Os vídeos da IBM sobre o Jev, da startup TypeSafe (What Is Jev? The AI Model That Doesn't Generate Text e New frontier AI models, TypeSafe's Jev AI, & NASA's IBM collab), tornam essa ponte explícita. Um **classificador** escolhe entre respostas pré-definidas em vez de gerar texto. Para um e-mail "fui cobrado duas vezes", o Jev devolve de uma vez: reembolso? 0,9; equipe? financeiro 0,85; urgência? alta.

O argumento técnico é a **calibração**: um modelo é calibrado se, entre os casos em que diz "80%", acerta cerca de 80%. Segundo a TypeSafe, LLMs de chat tendem a ser sobreconfiantes; recompensar o modelo com uma **regra de pontuação própria** (log-loss ou Brier) é o que incentiva calibração.

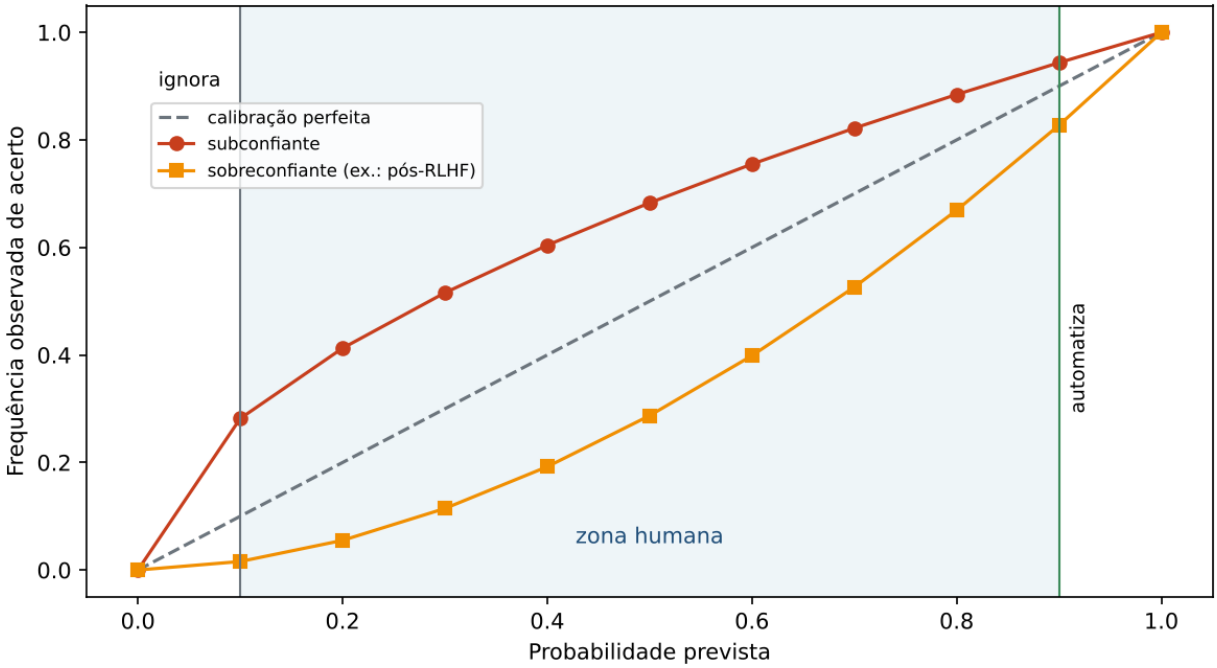


Diagrama de confiabilidade: a diagonal é o ideal e, abaixo dela, o modelo é sobreconfiante. As linhas verticais marcam a política de limiares: abaixo de 0,1 ignora, acima de 0,9 automatiza, no meio envia a um humano. Curvas ilustrativas. Fonte: elaboração própria a partir de "What Is Jev? The AI Model That Doesn't Generate Text".

Com probabilidades calibradas, a política é simples: **acima de 0,9 automatiza, entre 0,1 e 0,9 vai para humano, abaixo de 0,1 descarta**. Formalmente, age-se quando p > custo/(ganho + custo). Calibração se mede com o ECE e se corrige com *Platt scaling* ou *temperature scaling*. Fabio Akita, em LLMs + Agentes = competente o suficiente, mostra que modelos pequenos e locais empatam ou chegam perto dos líderes hospedados em tarefas de classificação: um modelo especializado e calibrado costuma vencer um gerador grande em custo e latência.

Atenção

Em The Problem with Jev: Can You Trust AI Decisions?, a IBM lembra que **confiança não é prova**. Modelos de decisão sobre texto continuam vulneráveis a *prompt injection* nos dados. Trate-os como componentes não confiáveis, registre entradas e saídas e mantenha controles determinísticos em volta.

Cascatas: barato primeiro, caro só quando necessário

Em Fraud Detection with AI: Ensemble of AI Models Improve Precision & Speed, a decisão "fraude sim/não" precisa sair em menos de ~200 ms. O primeiro estágio é **ML tabular** (regressão logística, gradient boosting) sobre valor, horário, local e histórico: barato, escalável e auditável. Só os casos de baixa confiança sobem para um **LLM encoder** (BERT, RoBERTa), que lê também o texto, como um memo "investimento urgente, ROI garantido de 200%".

Em resumo

A maioria dos casos é fácil: use o modelo mais barato que decide com confiança e escale só a fração incerta. Isso exige que o primeiro estágio seja **calibrado**. Como fraude é rara, acurácia é inútil: um modelo que nunca acusa fraude acerta 99,9%. Use precisão, recall e PR-AUC.

Sistemas de recomendação: fundamentos

Para começar

O livreiro do bairro que diz "se gostou daquele, vai adorar este" conhece seu gosto e os livros. Um **sistema de recomendação** faz isso para milhões de pessoas usando duas pistas: como os itens são e o que pessoas parecidas com você consumiram.

Um recomendador estima, para cada par (usuário, item), uma utilidade (probabilidade de clique, de compra, de não pular a música) e ordena os itens por ela. A entrada costuma ser uma matriz usuário × item enorme e esparsa, com feedback **explícito** (notas) ou, muito mais comum, **implícito** (cliques, tempo, pulos).

- **Baseada em conteúdo**: descreve itens por atributos ou embeddings e recomenda os parecidos com o que o usuário consumiu, pela similaridade de cosseno. Funciona para itens novos e é explicável, mas tende à bolha.
- **Filtragem colaborativa**: usa só o comportamento coletivo. Na variante por vizinhança, "quem comprou isto também comprou aquilo" (a Amazon popularizou a versão por item). Na **fatoração de matrizes**, $R \approx PQ^T$, cada usuário e item vira um vetor latente, e a predição é o produto escalar. Ficou famosa no Netflix Prize.
- **Híbridos e two-tower**: a colaborativa sofre de **cold start** (usuários e itens novos sem histórico) e ignora o contexto da sessão. O modelo **two-tower**, padrão atual, codifica usuário e item no mesmo espaço vetorial, e uma busca vetorial aproximada (ANN) acha os vizinhos.

Você sabia?

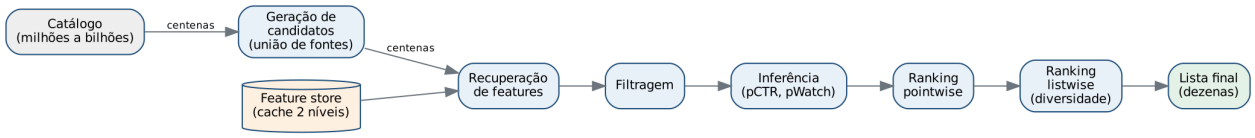
O Netflix Prize (2006–2009) pagou US$ 1 milhão a quem melhorasse em 10% o recomendador da empresa. A solução vencedora combinava centenas de modelos, e a Netflix nunca a pôs inteira em produção: o ganho não compensava a complexidade.

Arquitetura de um recomendador online

Para começar

Escolher 20 vídeos entre bilhões em menos de um segundo funciona como um processo seletivo: triagem rápida de currículos, entrevistas caras com poucos candidatos e escolha final pensando no time. O recomendador é esse **funil**.

O guia prático da Databricks, de Jake Noble (que liderou recomendações da home do YouTube), descreve seis etapas num funil que vai de bilhões de itens a dezenas, usando o contexto do momento: catálogo atual, comportamento recente, sessão.



Os seis componentes de um sistema de recomendação online, em funil. Fonte: adaptado de Noble, "A Practical Guide to Building an Online Recommendation System", Databricks Blog, 2025.

- **Geração de candidatos**: reduz bilhões a centenas, rápido e barato (candidatos pré-computados em batch, busca vetorial, filtragem colaborativa). Boa prática: unir várias fontes, inclusive "populares recentes" contra o cold start; nenhum ranking cria variedade que não entrou no funil.
- **Recuperação de features**: de item, de usuário e cruzadas (ex.: "já comprou este produto"). O ponto sutil é a **paridade online-offline**: se o treino usou gasto antes de impostos e a produção serve gasto depois de impostos, as predições quebram em silêncio. Feature stores existem para resolver isso.
- **Filtragem**: remove o inelegível (fora de estoque, já visto, impróprio), com limites para não remover todos os candidatos.
- **Inferência e calibração**: modelos preveem CTR, compra, tempo de exibição. Dois modelos com o mesmo AUC podem ter saídas em escalas diferentes; calibrar **desacopla** a troca de modelo do resto do sistema.
- **Ranking pointwise**: combina as predições em uma pontuação por item.
- **Ranking listwise**: considera a lista inteira, penalizando itens parecidos com os já escolhidos para garantir diversidade.

A fórmula pointwise citada do YouTube mostra uma decisão de produto codificada como número:

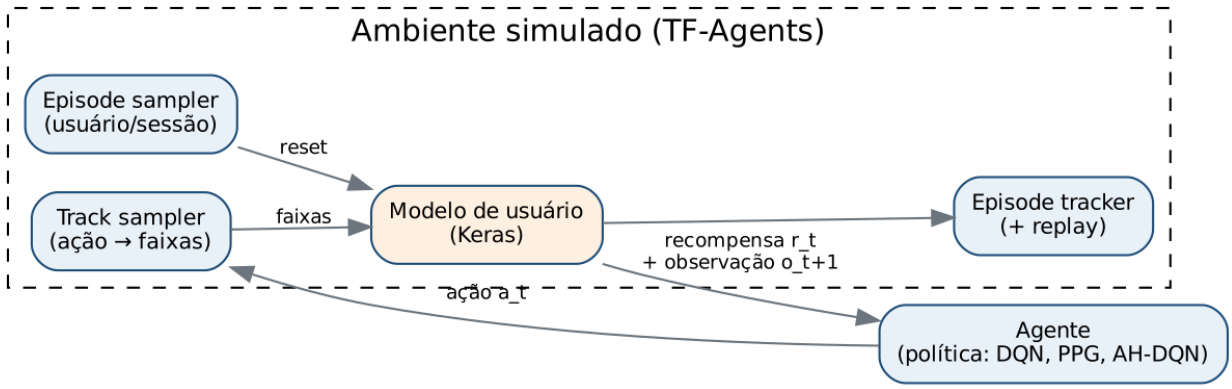
```
ranking_score = pCTR^X × pWatchTime^Y × (isSmallChannel ? Z : 1.0)
```

Os expoentes ponderam clique contra tempo assistido (evitando o *clickbait*), e Z dá impulso a canais pequenos. Sistemas reais são **híbridos**: modelos e embeddings treinados em batch sobre o histórico; features de sessão, filtros e ranking calculados online.

Recomendação como decisão sequencial: RL no Spotify

> **Para começar**
> Um bom DJ observa a pista, toca algo, vê a reação e ajusta a próxima música. Isso é **aprendizado por reforço** (RL): aprender por tentativa e erro com recompensas. Como treinar errando com milhões de ouvintes seria um desastre, o Spotify criou um público simulado.

Uma playlist não é uma lista de itens independentes: cada faixa muda o estado do ouvinte. A equipe do Spotify, num post de 2023 no blog do TensorFlow baseado num artigo do KDD 2023, trata o problema como **decisão sequencial**. A saída para não treinar com usuários reais foi **RL baseado em modelo**: um **modelo de usuário** prevê reações a listas de faixas e serve de recompensa num **simulador offline**.



Loop de RL com o modelo de usuário no lugar do usuário real e os quatro módulos do simulador do Spotify. Fonte: adaptado de Spotify/TensorFlow Blog, "Simulated Spotify Listening Experiences for Reinforcement Learning with TensorFlow and TF-Agents", 2023.

O simulador tem quatro módulos: um amostrador de sessões, um tradutor da ação do agente em faixas reais (escolher diretamente uma lista ordenada tornaria o espaço de ação combinatório), o modelo de usuário (ouvir ou pular) e um registro de episódios. O episódio termina por uma regra empírica, como vários pulos consecutivos. O agente **Action-Head DQN** recebe estado e *uma* faixa candidata e produz um valor Q; repete para cada candidata, escolhe a melhor, acrescenta à lista e continua. Assim lida com um catálogo que muda o tempo todo.

O resultado mais importante: as estimativas offline tiveram **alinhamento direcional** com os testes online. Se o simulador diz que a política A é melhor que a B, isso tende a se confirmar no teste A/B.

Atenção

O agente otimiza o modelo de usuário, não o usuário. Qualquer viés do simulador vira brecha a explorar (*reward hacking*). Por isso é preciso validar contra resultados online. O mesmo vale para o pós-treino de LLMs: o desenho da recompensa define o comportamento.

Métricas: como saber se está funcionando

Para começar

Um termômetro só serve se medir a coisa certa. Um detector de fraude que nunca acusa ninguém acerta 99,9% das vezes e não serve para nada.

Métrica	O que mede, em palavras simples
Precisão	Dos alarmes dados, quantos eram reais; importa quando alarme falso custa caro
Recall	Dos casos reais, quantos foram pegos; importa quando deixar passar custa caro
PR-AUC	Resumo de precisão e recall; melhor que ROC-AUC para casos raros
Log-loss, Brier, ECE	Se "80% de chance" acerta mesmo 8 em cada 10 vezes
Precision@k, Recall@k	Quantos dos 10 primeiros filmes sugeridos você queria ver
NDCG@k	Perde pontos se o filme certo aparece lá embaixo na lista
Cobertura, diversidade, novidade	Se as sugestões variam ou mostram sempre os mesmos sucessos

No fim, o juiz são métricas online em testes A/B: CTR, conversão, retenção, receita por sessão. Defina o tamanho de amostra antes, não interrompa o teste ao primeiro $p < 0{,}05$ (*peeking* infla falsos positivos) e acompanhe métricas de proteção, como latência e reclamações.

Atenção

Recomendadores avaliados em logs sofrem **viés de exposição**: o usuário só pôde clicar no que o sistema antigo mostrou. Separe treino e teste *no tempo*, nunca aleatoriamente, e reserve tráfego de exploração.

Você sabia?

Claude Shannon definiu o bit como unidade de informação em "A Mathematical Theory of Communication" (1948). A entropia que ele formalizou reaparece hoje na entropia cruzada, a função de perda mais usada para treinar classificadores e LLMs.

Juntando tudo: um checklist

Em resumo

- **Armazenamento**
  : relacional normalizado por padrão, índices, consultas parametrizadas, transações; documentos quando o padrão de acesso pede.
- **Movimento**
  : pipelines idempotentes com validação, retry e checkpoint; batch para profundidade, streaming para velocidade.
- **Governança**
  : documente, automatize ingestões, rastreie mudanças, marque qual dado foi para qual modelo.
- **Exploração**
  : estatística robusta, gráficos sempre, PCA para estrutura, várias famílias de detecção de anomalias.
- **Decisão**
  : probabilidades calibradas, limiares conforme custo, cascatas do barato ao caro.
- **Recomendação**
  : funil de candidatos, paridade de features, filtros, modelos calibrados, ranking com diversidade; RL com simulador validado quando a sequência importa.
- **Métricas**
  : PR-AUC e calibração para eventos raros, NDCG offline, A/B bem desenhado como juiz final.

Fontes deste capítulo

Fonte	URL	Canal / autor	Ponto principal
CS50x em Português - Aula 7 - SQL	https://www.youtube.com/watch?v=Samqz7iHwPM	CS50	SQL, modelagem, índices, injeção e transações
Scaling Data Pipelines: Memory Optimization & Failure Control	https://www.youtube.com/watch?v=A6x5y8yQRHY	IBM Technology	Chunking, validação, retry e checkpoint externo
Big Data vs Fast Data: Optimize Your AI Strategy	https://www.youtube.com/watch?v=vWVOMV_vxxs	IBM Technology	Treinar em big data, aplicar em fast data
Master NoSQL: Scalable Databases for Modern Applications	https://www.youtube.com/watch?v=pYK4No7ACRE	IBM Technology	Modelar documentos para o padrão de acesso
What Is Real-Time Data Streaming? AI & Machine Learning Applications	https://www.youtube.com/watch?v=aBIxpJ1_EyY	IBM Technology	Filtrar, enriquecer, analisar; valor decai com tempo
Data Products Explained: Key Characteristics & Benefits	https://www.youtube.com/watch?v=7w7_QWPS9L8	IBM Technology	Características de um produto de dados
What is Apache Hadoop?	https://www.youtube.com/watch?v=JWX5Inb--ig	IBM Technology	HDFS, YARN, MapReduce; Hadoop versus Spark
Big Data Rules For AI: Essential Data Management Principles	https://www.youtube.com/watch?v=AtXqpveCWQU	IBM Technology	Documentar, automatizar, rastrear, marcar dado e modelo
Fraud Detection with AI: Ensemble of AI Models Improve Precision & Speed	https://www.youtube.com/watch?v=Mo7JMC_oDII	IBM Technology	Cascata: modelo tabular, depois encoder
LLM + Data: Building AI with Real & Synthetic Data	https://www.youtube.com/watch?v=DOhvtcjl1ac	IBM Technology	Curadoria define representatividade dos dados
A $15 GPU Solves the Traveling Salesman Problem (and Your Staff Roster) | VectorFall Ep. 3	https://www.youtube.com/watch?v=2ILc4WY86us	Deep Tech with Amir Husain	Fluxo exploratório: estatística, PCA, anomalias

Fonte	URL	Canal / autor	Ponto principal
What Is Jev? The AI Model That Doesn't Generate Text	https://www.youtube.com/watch?v=YGgNBcIgI4s	IBM Technology	Decisões tipadas calibradas e limiares
The Problem with Jev: Can You Trust AI Decisions?	https://www.youtube.com/watch?v=_yO_Q7qo0HQ	IBM Technology	Probabilidade não é evidência
New frontier AI models, TypeSafe's Jev AI, & NASA's IBM collab	https://www.youtube.com/watch?v=O4n1jtWzt30	IBM Technology	Contexto do Jev: decisões em paralelo
LLMs + Agentes = competente o suficiente (e posts de 03/10 e 10/10/2026 sobre o Jev e classificadores)	https://akitaonrails.com/2026/10/12/llms-agentes-competente-o-suficiente/	AkitaOnRails (Fabio Akita)	Classificadores pequenos competem com hospedados
Simulated Spotify Listening Experiences for Reinforcement Learning with TensorFlow and TF-Agents	—	TensorFlow Blog (equipe do Spotify), 19/10/2023	Simulador de usuário e alinhamento offline–online
How to Build an Online Recommendation System (A Practical Guide to Building an Online Recommendation System)	—	Databricks Blog (Jake Noble), 2025	Seis etapas, paridade de features, calibração

CAPÍTULO 4 ·

Engenharia Reversa e Segurança

> *Engenharia reversa e segurança são duas faces da mesma disciplina: entender um sistema melhor do que seu autor esperava. Este capítulo apresenta, em nível conceitual, o que é engenharia reversa, para que a indústria a usa e como a IA mudou sua velocidade. Depois passa ao lado defensivo: falhas clássicas de aplicações, senhas e criptografia, phishing, segurança de agentes de IA e governança.*

Para começar

Imagine receber um bolo pronto e tentar descobrir a receita. Engenharia reversa é isso aplicado a programas. Segurança é o outro lado: saber como alguém descobriria a receita, ou estragaria o bolo, para impedir que isso aconteça.

O que é engenharia reversa e por que ela voltou à moda

Para começar

Compilar um programa é traduzi-lo para uma língua que quase ninguém lê. Engenharia reversa é traduzir de volta o suficiente para entender o que ele faz. É assim que se analisam vírus, se auditam fornecedores e se preserva software antigo.

Engenharia reversa (RE, de *reverse engineering*) é reconstruir conhecimento de projeto a partir de um artefato pronto: um binário compilado, um bytecode Java ou .NET, um pacote JavaScript minificado, um aplicativo móvel ou um firmware. Seus usos legítimos são antigos e comuns na indústria:

- **Análise de malware**: empresas de antivírus e equipes de resposta a incidentes estudam amostras para criar detecções.
- **Interoperabilidade**: formatos de arquivo e protocolos não documentados (o LibreOffice lendo arquivos do Office, o Samba falando com redes Windows).
- **Auditoria**: verificar o que um software de terceiros realmente faz antes de implantá-lo.
- **Preservação**: manter jogos e programas antigos funcionando quando o fabricante já não existe (Livro 6, Capítulo 2 (IA e games: preservação)).

Nada disso é novo: disassemblers existem desde os anos 1980, e decompiladores de bytecode Java e C# tornam esses binários quase legíveis há mais de vinte anos. O que mudou em 2025–2026, segundo o debate em A NOVA ONDA DE CLONES DE SOFTWARE | COMO O REA FUNCIONA (Código Fonte TV), foi a **velocidade**: modelos de código leem

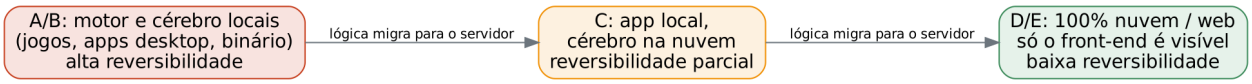
saída de decompilador, nomeiam funções e documentam comportamentos. Daí a onda viral de “clones” de produtos comerciais feitos com IA.

A tese “a Adobe morreu?” e seus limites

No post *Engenharia reversa com IA: a Adobe morreu?*, Fabio Akita contesta a ideia de que agentes vão substituir qualquer software comercial. As ferramentas “de IA para RE” continuam dependendo de ferramental clássico, como o Ghidra, para o trabalho pesado; a IA é uma camada de interpretação. Reescritas grandes continuam caras, porque décadas de casos de borda não cabem numa interface parecida. E o valor de produtos como Photoshop ou Excel está também em ecossistema, compatibilidade e suporte. Reimplementações abertas como LibreOffice e GIMP mostram que alternativas são possíveis, mas clones idênticos ou feitos a partir do binário de terceiros entram em terreno jurídico arriscado.

O que pode (e o que não pode) ser revertido

Uma classificação útil, apresentada no canal Gabriel Kendy, olha para **onde fica a lógica** de uma solução. Quando ela está toda no dispositivo (jogos, apps desktop), quase tudo é recuperável. Quando o app é local mas a lógica valiosa roda na nuvem, o analista vê só o cliente e o protocolo. Quando tudo é web, resta apenas o front-end.



Reversibilidade conforme a localização da lógica. Quanto mais lógica no servidor, menos o binário revela.
Fonte: elaboração própria com base no canal Gabriel Kendy.

A lição defensiva é direta: **lógica de negócio valiosa, segredos e decisões de autorização pertencem ao servidor**. Ofuscação eleva o custo, mas não muda a classe de reversibilidade. O Livro 3, Capítulo 5 (Segurança cloud-first) desenvolve essa ideia como arquitetura.

Legalidade e sala limpa

A legalidade varia por país. Muitos ordenamentos admitem RE para interoperabilidade, pesquisa de segurança e correção de erros, mas licenças frequentemente a proíbem, e leis anticircunvenção (como a DMCA nos EUA) restringem quebrar proteções técnicas. Os termos de uso dos provedores de IA também vedam usos que infrinjam direitos de terceiros. O padrão histórico para reimplementações defensáveis é a **sala limpa**: uma equipe documenta o comportamento observado; outra, que nunca viu o código original, implementa a partir da especificação. A regra profissional é simples: *analise apenas o que você tem autorização para analisar*.

Você sabia?

Nos anos 1980, a Phoenix Technologies recriou a BIOS do IBM PC com uma equipe de sala limpa: engenheiros que nunca tinham lido o código da IBM implementaram a partir de uma especificação escrita por outro grupo. Isso viabilizou legalmente a indústria de PCs compatíveis.

Formatos executáveis e ferramentas

Para começar

Um executável tem uma "ficha de identificação" no começo e compartimentos definidos, como uma caixa de remédio com rótulo e bula. Conhecer esse arranjo é o primeiro passo de qualquer análise.

Os formatos nativos dominantes são **PE** (Windows: .exe, .dll), **ELF** (Linux, Android nativo, sistemas embarcados) e **Mach-O** (macOS e iOS). Todos têm um cabeçalho que declara arquitetura, ponto de entrada, seções de código e dados e bibliotecas importadas. Para quem defende, esses campos alimentam detecção: assinaturas digitais, listas de hashes e regras YARA usadas por antivírus. Outros ecossistemas têm formatos próprios, como APK (Android), assemblies .NET, pacotes Electron e WebAssembly.

As ferramentas se agrupam por camada: **disassembly** (bytes → instruções), **decompilação** (instruções → pseudocódigo), **análise dinâmica** (observar a execução) e bibliotecas de apoio.

Ferramenta	Tipo	Licença
Ghidra	Suíte completa (disassembler + decompilador)	Open source (NSA)
IDA Pro	Suíte completa, padrão histórico da indústria	Comercial (Hex-Rays)
Binary Ninja	Suíte completa	Comercial
radare2 / Rizin	Framework de linha de comando	Open source
JADX / ILSpy	Decompiladores de bytecode Android e .NET	Open source
Capstone / LIEF	Bibliotecas de disassembly e de leitura de formatos	Open source

Você sabia?

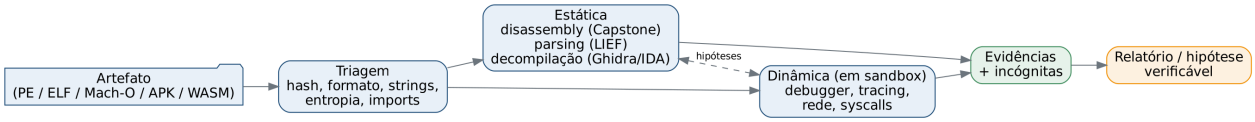
O Ghidra foi desenvolvido internamente pela NSA e liberado como software livre em 2019, na conferência RSA. Hoje é a principal alternativa gratuita ao IDA Pro e base de muitas ferramentas de RE assistida por IA.

Análise estática versus dinâmica

Para começar

Uma encomenda suspeita pode ser examinada por fora (raio-X, peso) ou aberta num local seguro. Na análise de software, o primeiro jeito é a análise estática; o segundo, a dinâmica.

Análise estática examina o artefato sem executá-lo. É segura e cobre, em tese, todos os caminhos de código, mas sofre com ofuscação e código gerado em tempo de execução. **Análise dinâmica** executa o alvo em ambiente isolado e observa chamadas de sistema, rede e arquivos. Vê o que de fato acontece, mas só nos caminhos exercitados, e exige isolamento rigoroso, porque o alvo pode ser malicioso.



Fluxo típico de investigação combinando análise estática e dinâmica, com hipóteses circulando entre as duas. Fonte: elaboração própria.

Critério	Estática	Dinâmica
Executa o alvo?	Não	Sim, num ambiente isolado (sandbox)
Cobertura	Todos os caminhos, em teoria	Só os que rodaram no teste
Ofuscação	Atrapalha muito	Atrapalha menos
Risco	Baixo	Alto se o isolamento falhar
Resultado	Mapa do programa e suspeitas	O que o programa fez: arquivos, conexões
No dia a dia	Raio-X de uma encomenda suspeita	Abrir a caixa numa sala isolada

Na prática, as duas se alimentam: a análise estática levanta hipóteses (“esta função parece cifrar dados”), e a dinâmica as confirma ou descarta observando o comportamento real. Equipes de resposta a incidentes fazem esse ciclo diariamente para extrair indicadores de comprometimento e criar regras de detecção que protegem outras empresas.

Uma terceira família, a **execução simbólica**, trata entradas como variáveis e calcula que condições levam a cada caminho; a indústria a usa em *fuzzing* defensivo para encontrar bugs antes do lançamento.

IA aplicada à engenharia reversa

> **Para começar**
>
> O decompilador entrega código com nomes sem sentido, como num livro em que todos os personagens se chamam “Pessoa 1”. Modelos de linguagem são bons em sugerir o que cada trecho faz, mas também inventam com convicção (veja Livro 1, Capítulo 2 (LLMs, Transformers e MoE)).

Os modelos ajudam a **nomear e explicar** funções, **recuperar tipos**, **resumir** grandes volumes de código e **correlacionar** evidências de várias ferramentas. A arquitetura comum é um agente de código conectado por MCP (Livro 2, Capítulo 4 (Agentes e MCP)) a ferramentas como Ghidra ou IDA. Três dificuldades permanecem:

- **Alucinação**: uma explicação sem evidência rastreável é só hipótese.
- **Contexto**: binários grandes geram milhões de linhas de pseudocódigo.
- **Conteúdo hostil**: strings dentro do alvo podem conter instruções para o agente (prompt injection).

Projetos maduros respondem com **proveniência**: cada afirmação aponta para a ferramenta e o trecho que a sustentam, o que não foi provado fica registrado como incógnita, o conteúdo do alvo é marcado como não confiável e o agente opera com privilégio mínimo. Com essa disciplina, até um modelo modesto produz resultados auditáveis.

Segurança de aplicações: falhas clássicas e defesas corretas

> **Para começar**
> Mudamos de lado da mesa. A maioria dos problemas de uma aplicação acontece quando ela confia demais no que o usuário entrega, como um bilhete que, em vez de pedido, contém uma ordem para o caixa.

A referência canônica é o **OWASP Top 10**, que lista controle de acesso quebrado, falhas criptográficas, injeção, configuração incorreta e componentes vulneráveis, entre outros. A OWASP mantém também um Top 10 para aplicações com LLM, liderado por prompt injection.

SQL injection

SQL injection ocorre quando dados do usuário são **concatenados** no texto de uma consulta: o banco não distingue comando de dado. A defesa recomendada pela OWASP são as **consultas parametrizadas**, em que valores seguem separados do SQL.

```
# ERRADO: concatenação
cur.execute(f"SELECT id FROM users WHERE email = '{email}'")

# CERTO: parâmetros
cur.execute("SELECT id FROM users WHERE email = ?", (email,))
```

> **Atenção**
> ORMs protegem porque usam parametrização por baixo, e voltam a ser vulneráveis se você concatenar SQL bruto. “Sanitizar” strings manualmente é frágil; bibliotecas como *bleach* tratam HTML (XSS), não SQL. Nomes de tabela e colunas de ORDER BY devem vir de uma lista de permissões, como mostra COMO AUMENTAR A SEGURANÇA DO SEU BACKEND? | Conhecendo ataques mais comuns.

Camadas complementares: usuário de banco com privilégios mínimos, banco acessível só pelo back-end e erros de SQL nunca exibidos ao usuário.

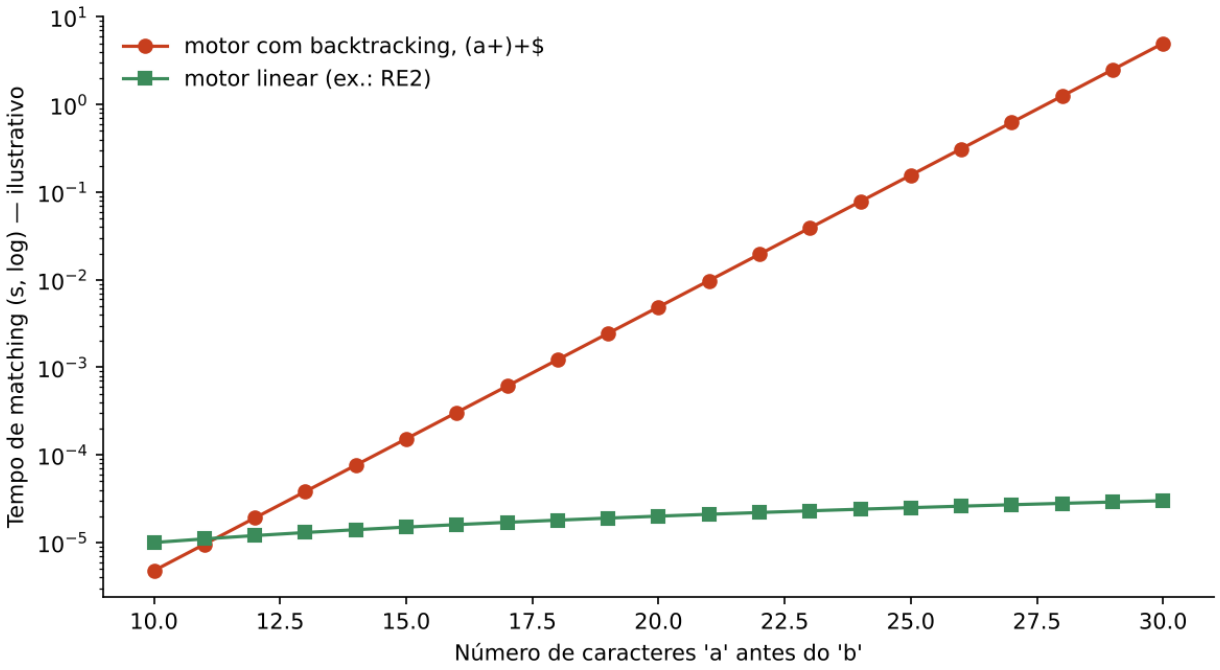
CSRF

Cross-Site Request Forgery explora o fato de o navegador anexar cookies automaticamente: um site malicioso dispara uma requisição ao seu banco e ela chega com sua sessão. Defesas: cookies `SameSite=Lax` ou `Strict`; token anti-CSRF aleatório vinculado à sessão; header customizado em APIs, que força preflight CORS; e verificação do header `Origin` em ações sensíveis. Erros comuns: colocar o token na URL, onde vaza em logs e his-

tórico; confiar só no *double submit cookie*, que tem contornos conhecidos; e desabilitar CORS "para funcionar". Em operações financeiras, vale exigir reautenticação ou MFA. Se houver XSS, o script roda dentro da sua origem e lê o token, por isso escape de saída e Content Security Policy continuam necessários.

ReDoS: quando uma regex derruba o servidor

Motores de regex com **backtracking** (Python, JavaScript, Java, PCRE) podem levar tempo exponencial em padrões com quantificadores aninhados, como `(a+)+$`. Em Regex pode TRAVAR o seu servidor! CUIDADO!, cerca de 30 caracteres bastam para travar uma máquina por segundos.



Crescimento ilustrativo do tempo de matching de `(a+)+$` num motor com backtracking versus um motor linear. Fonte: elaboração própria, ordem de grandeza de "Regex pode TRAVAR o seu servidor!".

Mitigações: evitar quantificadores aninhados, preferir motores lineares (RE2, Go, a crate `regex` do Rust), limitar o tamanho da entrada e impor timeout.

Princípios de engenharia segura

O vídeo O MÍNIMO que um dev precisa saber sobre segurança, de Augusto Galego, resume: **minimizar a superfície de ataque**, **menor privilégio**, **defaults seguros**, **criptografia com padrões estabelecidos** e **atualizações rápidas**. Some a isso: segredos nunca no repositório (um segredo commitado deve ser revogado, não só apagado) e comparação de tokens em tempo constante (`hmac.compare_digest`), para não vazar informação pelo tempo de resposta.

Rede: VPN e Tor

HTTPS cifra o conteúdo, mas o provedor de internet ainda vê com quem você fala. Segundo Personal VPNs: Encryption Myths and Data Security Explained, uma VPN é **trans-**

ferência de confiança: o provedor da VPN passa a ver o que o ISP via, e não há anonimato se você faz login. O **Tor** usa três saltos cifrados, nenhum dos quais conhece a rota inteira; é melhor para anonimato e pior para velocidade, e o trecho final até o site depende de HTTPS. O podcast The PHP bug threatening the web. Plus, who's auditing AI? lembra que nós de saída já foram comprometidos em massa: assuma que podem ser hostis.

Caso de uso	VPN	Tor
Esconder tráfego do provedor de internet	Sim	Sim
Anonimato	Fraco (confiança no provedor)	Forte
Velocidade	Boa	Ruim
Wi-Fi público	Adequada	Desnecessário

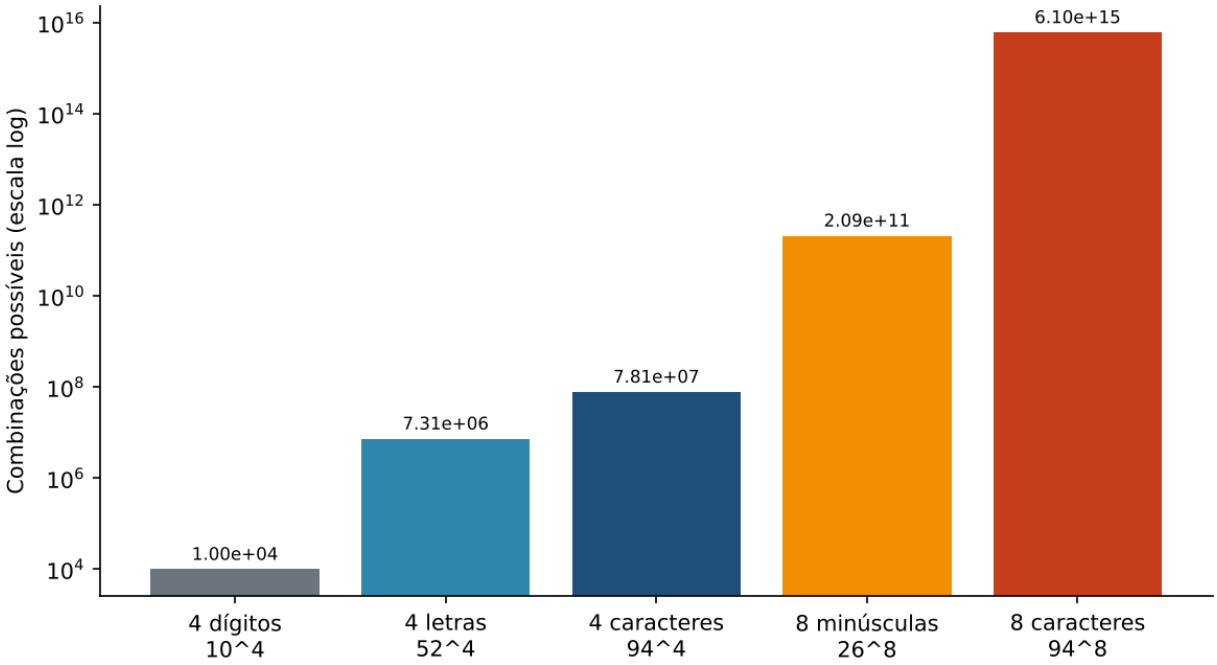
Outras ameaças de rede seguem a mesma lógica de camadas. **Man-in-the-middle** em Wi-Fi público é neutralizado por TLS em tudo e HSTS. **DDoS** pede CDN com filtragem do provedor, rate limiting no gateway e banco aceitando conexões só do back-end.

> **Você sabia?**
> Em 1988, o Morris Worm infectou cerca de 10% dos computadores da internet de então. Robert Tappan Morris foi o primeiro condenado pela lei americana de fraude por computador, e o episódio levou à criação do CERT/CC.

Senhas, hashing e criptografia

> **Para começar**
> O site precisa conferir sua senha sem guardar uma cópia que um ladrão possa usar. Para isso existe o **hash**, uma impressão digital de mão única. Já a **criptografia** é um cofre: quem tem a chave abre.

A aula CS50 for Business - Lecture 8 - Securing Systems separa **autenticação** (quem você é) de **autorização** (o que pode fazer). O espaço de busca de uma senha cresce exponencialmente com o comprimento: alfabeto A e comprimento L dão A^L combinações.



Número de combinações para diferentes políticas de senha (escala logarítmica). Fonte: elaboração própria com dados de “CS50 for Business — Securing Systems”.

Num **ataque online**, rate limiting e MFA tornam a força bruta impraticável. Num **ataque offline**, após vazamento do banco, GPUs testam bilhões de hashes rápidos por segundo, e por isso o algoritmo importa. Comprimento vence complexidade: uma frase-senha longa num gerenciador é melhor que oito caracteres “complexos”.

Como armazenar senhas

Password Hashing, Salting e Peppering e A Forma Correta de Criptografar Senhas: O Guia que Todo Dev Precisa Ver! convergem: nunca texto puro nem criptografia reversível; **salt** único por usuário; e algoritmo **lento e caro em memória**, como **Argon2id** (recomendação da OWASP), scrypt ou bcrypt. MD5 e SHA-256 puros são rápidos demais. Use a biblioteca do ecossistema ou um provedor de identidade, nunca uma função caseira.

```
from argon2 import PasswordHasher
ph = PasswordHasher()               # Argon2id, salt automático
h = ph.hash("frase-senha longa e única")
ph.verify(h, attempt)                # lança exceção se não confere
```

Atenção

É comum ouvir que o SHA-256 é “inquebrável”. Para colisões, na prática, é. Para **senhas**, não: um SHA-256 sem salt de uma senha comum é descoberto em instantes por dicionário, porque o atacante não inverte a função, apenas testa candidatos prováveis. O que importa para senhas é o hash ser *lento*. Se um site lhe envia a senha original por e-mail, ele a guarda de forma recuperável; sistemas corretos só permitem *redefinir*.

O **pepper**, um segredo da aplicação guardado fora do banco, é uma camada opcional: protege contra vazamento só do banco, mas perdê-lo invalida todas as senhas.

Criptografia simétrica e assimétrica

Simétrica (AES, ChaCha20): a mesma chave cifra e decifra; é rápida, mas exige compartilhar a chave. **Assimétrica** (RSA, curvas elípticas): cada parte tem um par de chaves.

Objetivo	Quem usa qual chave
Confidencialidade	Cifra-se com a **pública do destinatário**; só a **privada** dele decifra
Assinatura digital	Assina-se com a **própria privada**; qualquer um verifica com a pública

No **TLS 1.3**, uma troca Diffie-Hellman efêmera permite que as duas partes combinem uma chave simétrica de sessão, e o servidor assina a troca para provar identidade. Nos **passkeys**, o dispositivo guarda uma chave privada por site e assina desafios; não há segredo para ser digitado num site falso, o que os torna resistentes a phishing. Na criptografia **ponta a ponta** (Signal, WhatsApp), só os aparelhos das pontas têm as chaves. A aula do CS50 lembra a diferença entre segurança e privacidade: com HTTPS o trajeto até o Gmail está protegido, mas o servidor lê seus e-mails; com ponta a ponta, não.

Dizer que se "cifra com a chave privada" para assinar é uma simplificação do RSA de livro-texto que confunde; o termo correto é *assinar*. O vídeo Privacidade em Risco? Entendendo Criptografia e Algoritmos Hash, de Fernanda Kipper, troca os papéis das chaves num trecho e depois se corrige; a tabela acima é a forma correta. Dois corolários práticos: com **criptografia de disco completo**, "apagar" um aparelho é descartar a chave; e o ransomware usa a mesma criptografia contra você, por isso a resposta é backup.

> **Dica**
> Base64 é codificação (reversível sem chave); AES é criptografia (reversível com chave); SHA-256 é hash (não reversível, mas testável); Argon2 é hash de senha (lento de propósito).

Engenharia social: phishing na era da IA

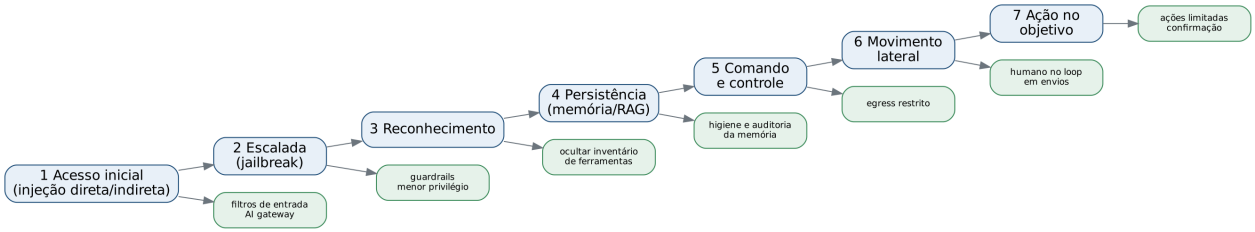
Phishing explora a confiança humana com promessas ou medo, por e-mail, SMS, voz ou QR code. Segundo o relatório IBM Cost of a Data Breach 2024, citado em Phishing Defenses: Top Cybersecurity Strategies to Protect Your Data, o phishing foi o segundo vetor de ataque mais comum. A IA permite mensagens personalizadas e sem erros, então procurar erros de português deixou de ser defesa. O que funciona: não clicar em links de mensagens, MFA e sobretudo passkeys, DNS filtrante e SPF/DKIM/DMARC no domínio.

Segurança de agentes de IA

> **Para começar**
> Um agente de IA é como um estagiário prestativo que obedece a qualquer bilhete que encontra, inclusive um bilhete escondido numa página. Esse é o problema da **prompt injection**.

Em sistemas tradicionais, código e dados viajam por canais separados, e é isso que consultas parametrizadas garantem. Em LLMs, **tudo é token**: instruções, pedido do usuário, conteúdo de uma página. Segundo What Is a Prompt Injection Attack?, há **injeção direta** (o usuário manipula o modelo), **indireta** (a instrução está nos dados lidos, como o agente de compras que pagou caro por um livro por causa de texto oculto na página) e **jailbreak**.

Bruce Schneier e coautores descrevem o **promptware** (The Promptware Kill Chain: How Prompt Injection Becomes AI Malware) como nova classe de malware, com uma kill chain de sete elos. Os mais traiçoeiros são a **persistência** (instrução gravada na memória do agente) e o **movimento lateral** (um assistente de e-mail que reencaminha o payload).



Kill chain de promptware e um controle defensivo associado a cada elo. Fonte: elaboração própria com base em “The Promptware Kill Chain”, IBM Technology.

Camada	Controle, num exemplo simples
Entrada	Marcar o que veio de fora: texto de uma página nunca vira ordem
Capacidades	Dar só as chaves necessárias, como estagiário sem acesso ao cofre
Ações	Pedir confirmação de uma pessoa antes de comprar, enviar ou apagar
Memória	Anotar a origem de cada lembrança; bilhete achado na rua não vira regra
Observabilidade	Registrar tudo o que o agente leu e fez, como um diário de bordo

Vale também a lição de The Problem with Jev: Can You Trust AI Decisions?: “sim, 97%” é *confiança*, não *evidência*. Decisões de alto risco pedem perguntas menores e verificáveis, sinais independentes e registro de tudo. Agentes com terminal, navegador e mensageiro devem rodar isolados, com acesso restrito a usuários autorizados.

No caso do agente de compras, discutido em Securing AI Agents: How to Prevent Hidden Prompt Injection Attacks, a proposta é um AI gateway que inspeciona três pontos: o pedido do usuário, as requisições do agente e o conteúdo devolvido pelos sites. Um estudo da Meta citado no vídeo registrou sucesso parcial dos ataques em 86% dos casos, mas os agentes muitas vezes não conseguiam concluir o objetivo do atacante: uma “segurança por incompetência” que tende a desaparecer conforme os agentes melhoram.

Você sabia?

Em “Reflections on Trusting Trust” (1984), Ken Thompson descreveu um compilador que inseria um backdoor e escondia o truque até de quem lesse seu código-fonte. É a raiz das discussões atuais sobre cadeia de suprimentos e builds reprodutíveis.

Vulnerabilidades, zero-days e a corrida com a IA

Um **zero-day** é uma falha explorada antes de o fabricante corrigi-la. Separe **vulnerabilidade** (a fraqueza), **exploit** (o código que a aproveita) e **ataque** (o uso, que continua até o patch ser *aplicado*). Um experimento citado em What is a Zero Day Threat? mostrou o GPT-4 gerando exploits a partir de descrições de CVE em cerca de 87% dos casos, contra quase zero do GPT-3.5. A consequência é que **velocidade de patch virou controle de segurança número um**: o WannaCry atingiu mais de 100 mil sistemas explorando uma falha cuja correção já existia. A contrapartida otimista é usar os mesmos modelos para auditar e corrigir antes do lançamento.

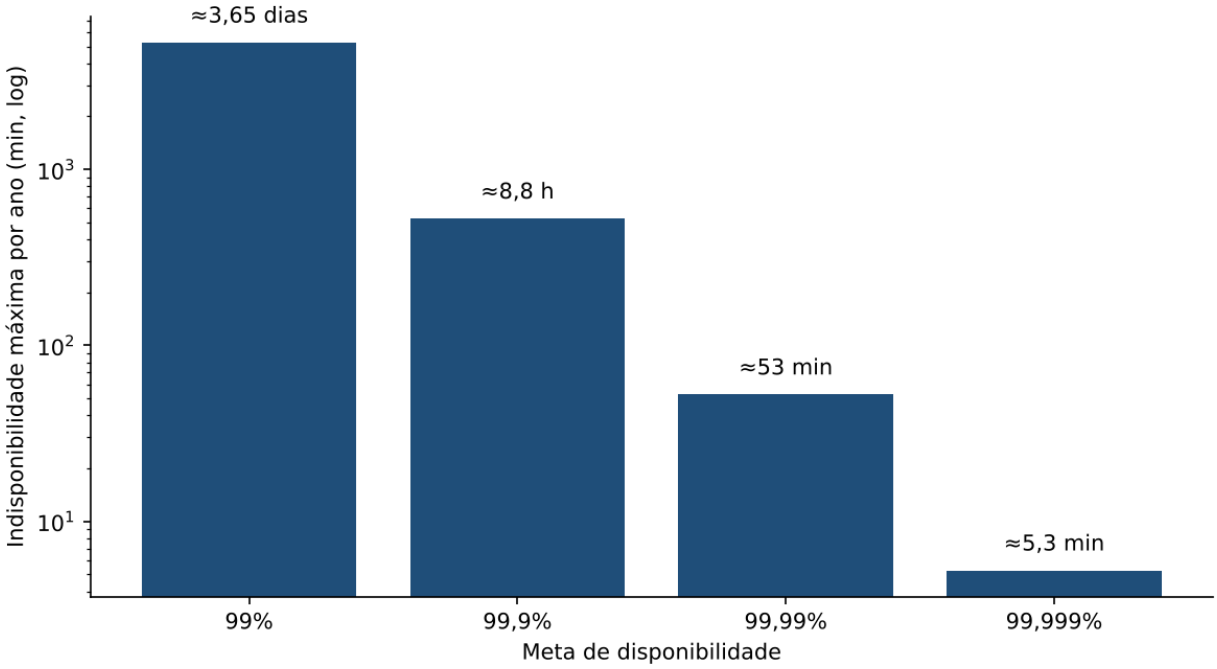
O podcast da IBM Technology citado acima traz um exemplo: vulnerabilidades no PHP, que sustenta boa parte da web, encontradas com IA numa auditoria externa. A lição é organizacional: saber onde cada componente roda, ter gestão ágil de patches e aceitar janelas de manutenção planejadas. Casos históricos lembram o impacto: Stuxnet usou vários zero-days, Heartbleed vazou chaves da memória de servidores OpenSSL e o Pegasus foi usado contra jornalistas. As defesas são as de sempre, combinadas: patch rápido, defesa em profundidade, segmentação e monitoramento (EDR, SIEM).

Governança, resiliência e estratégia

> **Para começar**
> Uma casa segura não depende só de uma tranca: precisa de plano, rotina e cópia das chaves em outro lugar. Governança é esse plano.

O **NIST Cybersecurity Framework 2.0** (2024), apresentado em Building a Cybersecurity Framework, organiza um programa em seis funções: **Governar** (risco, papéis, políticas), **Identificar** (inventário de ativos), **Proteger** (criptografia, MFA, backup), **Detectar** (EDR, SIEM), **Responder** (playbooks) e **Recuperar** (restauração e comunicação, no Brasil conforme a LGPD).

A regra de backup **3-2-1-1-0** (3-2-1 Backup Rule Explained: Protect Your Data from Disaster) pede 3 cópias, em 2 mídias, 1 fora do local, 1 imutável e 0 erros em testes de restauração. Um backup nunca restaurado é uma hipótese. E, como lembra Disaster Recovery vs Operational Resilience: Protecting Your Data, recuperação de desastres não é resiliência a ataques: num ataque, infectar os backups faz parte da estratégia, por isso são necessárias cópias imutáveis e uma “cópia limpa conhecida”.



Indisponibilidade máxima anual por meta de disponibilidade (escala log). Cada "nove" adicional reduz o tempo tolerado em 10×. Fonte: cálculo próprio.

Por fim, An Ancient Guide to Cybersecurity: 8 Lessons from The Art of War traduz Sun Tzu para a defesa: conhecer os próprios ativos e o adversário, vencer pela prevenção e usar o engano a favor do defensor, com **honey tokens** (credenciais falsas que disparam alarme quando usadas). O engano se conecta à engenharia reversa: atacantes, humanos ou agentes, vasculham configurações e binários atrás de segredos, e um segredo falso bem posicionado transforma essa curiosidade em alarme.

Vale ainda o princípio da **auditoria independente**. O mesmo podcast discute avaliadores externos com acesso privilegiado a laboratórios de IA e sugere revisão cruzada entre empresas, nos moldes das grandes auditorias contábeis. Para qualquer organização, olhos de fora desafiam suposições internas.

Em resumo

- Tudo que roda no cliente pode ser lido; a IA só tornou isso mais rápido. Proteja lógica e segredos no servidor.
- RE com IA confiável exige proveniência, autorização e alvo tratado como hostil.
- Injeção (SQL ou prompt) nasce de misturar instrução e dado; para SQL, parametrização resolve; para LLMs, defesa em camadas.
- Senhas: Argon2id com salt, ou melhor, passkeys. Governança, backups testados e patch rápido decidem a maioria dos incidentes.

Fontes deste capítulo

Título	URL	Canal / autor	Ponto principal
CS50 for Business - Lecture 8 - Securing Systems	https://www.youtube.com/watch?v=9KoYnFuHYL8	CS50	Autenticação, senhas, criptografia, passkeys

Título	URL	Canal / autor	Ponto principal
Engenharia Reversa Black: copie qlqer aplicativo. Ghidra, IDA, frida, análise binária+ REA.	https://www.youtube.com/watch?v=iCAtSC2CrVw	Gabriel Kendy	Reversibilidade conforme localização da lógica
Personal VPNs: Encryption Myths and Data Security Explained	https://www.youtube.com/watch?v=2fCTrjmDqyg	IBM Technology	VPN é transferência de confiança
A NOVA ONDA DE CLONES DE SOFTWARE	COMO O REA FUNCIONA	https://www.youtube.com/watch?v=-soAkLdbNfY	Código Fonte TV	IA acelera RE; sala limpa, termos de uso
Regex pode TRAVAR o seu servidor! CUIDADO!	https://www.youtube.com/watch?v=wuVXBiJoIKU	Augusto Galego	Backtracking exponencial; motores lineares e timeout
The Problem with Jev: Can You Trust AI Decisions?	https://www.youtube.com/watch?v=_yO_Q7qo0HQ	IBM Technology	Confiança não é evidência; verificar antes
What Is a Prompt Injection Attack?	https://www.youtube.com/watch?v=jrHRe9lSqqA	IBM Technology	Injeção direta, indireta e jailbreak
The Promptware Kill Chain: How Prompt Injection Becomes AI Malware	https://www.youtube.com/watch?v=K68sqG18270	IBM Technology	Sete elos do malware baseado em prompts
Securing AI Agents: How to Prevent Hidden Prompt Injection Attacks	https://www.youtube.com/watch?v=5ZA1lTxTH3c	IBM Technology	Texto oculto engana agente; AI gateway
Privacidade em Risco? Entendendo Criptografia e Algoritmos Hash	https://www.youtube.com/watch?v=5ve4MkHmALc	Fernanda Kipper	Hash, simétrica vs. assimétrica (com correção)
The PHP bug threatening the web. Plus, who's auditing AI?	https://www.youtube.com/watch?v=VoSQZBguB1A	IBM Technology	Falhas achadas por IA; patch ágil; auditoria
3-2-1 Backup Rule Explained: Protect Your Data from Disaster	https://www.youtube.com/watch?v=WJzsX32qMJY	IBM Technology	Cópia imutável e restauração testada
Building a Cybersecurity Framework	https://www.youtube.com/watch?v=Ula3TG8QS7g	IBM Technology	NIST CSF 2.0 com a função Governar
Disaster Recovery vs Operational Resilience: Protecting Your Data	https://www.youtube.com/watch?v=kr6fc869Ugs	IBM Technology	Ataques miram backups; snapshots imutáveis
An Ancient Guide to Cybersecurity: 8 Lessons from The Art of War	https://www.youtube.com/watch?v=kjoQPn--F7A	IBM Technology	Prevenção, engano e risco
A Forma Correta de Criptografar Senhas: O Guia que Todo Dev Precisa Ver!	https://www.youtube.com/watch?v=sC-rrOdwsIs	Attekita Dev	Hash lento com salt; só redefinir senha
Password Hashing, Salting e Peppering	https://www.youtube.com/watch?v=SZb7yyhTWBg	Augusto Galego	Ataque online vs. offline; Argon2id
O MÍNIMO que um dev precisa saber sobre segurança	https://www.youtube.com/watch?v=aGVN6aHKkE0	Augusto Galego	Superfície mínima, menor privilégio, segredos
COMO AUMENTAR A SEGURANÇA DO SEU BACKEND?	Conhecendo ataques mais comuns	https://www.youtube.com/watch?v=JqCH1uUntuQ	Fernanda Kipper	Consultas parametrizadas e mitigação em camadas
Phishing Defenses: Top Cybersecurity Strategies to Protect Your Data	https://www.youtube.com/watch?v=nSGQkE67jcg	IBM Technology	Tipos de phishing; passkeys, DMARC
What is a Zero Day Threat?	https://www.youtube.com/watch?v=w5MV1Jeo76g	IBM Technology	Vulnerabilidade, exploit, patch; IA gera exploits
Engenharia reversa com IA: a Adobe morreu? (13/10/2026)	https://akitaonrails.com/2026/10/13/engenharia-reversa-com-ia-a-adobe-morreu/	Fabio Akita (AkitaOnRails)	Ferramentas dependem de Ghidra; reescritas seguem caras